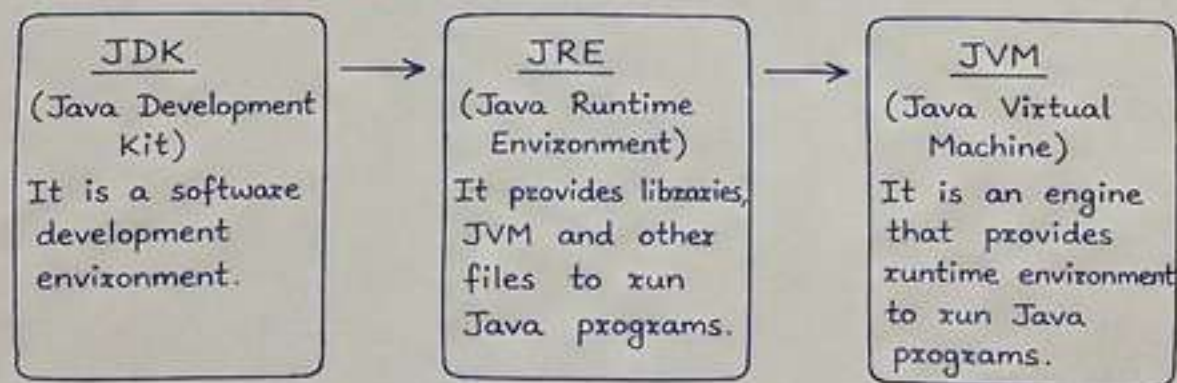


1. Introduction to Java

- Java is a high level, object oriented, platform independent programming language.
- Developed by James Gosling in 1995 at Sun Microsystems.
- Java follows Write Once, Run Anywhere.

Features of Java

1. Simple
2. Object Oriented
3. Platform Independent
4. Secure
5. Robust
6. Architecture Neutral
7. Portable
8. High Performance
9. Multithreaded
10. Distributed

JDK, JRE, JVM

Sandeep Chaurasiya

2. Path Setup

- Set JAVA_HOME to JDK installation path.
- Add %JAVA_HOME%\bin to Path environment variable.
- Open cmd and type java -version to check installation.

3. First Java Program

```

class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
  
```

Output:

Hello World

4. Data Types

- Java has two types of data types.

1. Primitive Data Types : byte, short, int, long, float, double, char, boolean
2. Non-Primitive Data Types : String, Arrays, Classes, Interfaces etc.

Example:

```

int age = 22;
char grade = 'A';
boolean isJavaFun = true;
  
```

Sandeep Chaurasiya

5. Type Casting

- Converting one data type into another is called type casting.

1. Implicit Casting (Widening)
byte → short → int → long → float → double
2. Explicit Casting (Narrowing)
double → float → long → int → short → byte

Example:

```

int a = 10;
double d = a; // implicit
double x = 10.5;
int y = (int) x; // explicit
  
```

6. Operators

1. Arithmetic : +, -, *, /, %
2. Relational : ==, !=, >, <, >=, <=
3. Logical : &&, ||, !
4. Assignment : =, +=, -=, *=, /=
5. Unary : ++, --, +, -, !
6. Bitwise : &, |, ^, ~, <<, >>

Sandeep Chaurasiya

7. Number System

1. Binary (Base 2)
→ uses 0 and 1
Example: $1010_2 = 10_{10}$
2. Octal (Base 8)
→ uses 0 to 7
Example: $17_8 = 15_{10}$
3. Decimal (Base 10)
→ uses 0 to 9
Example: $25_{10} = 25_{10}$
4. Hexadecimal (Base 16)
→ uses 0 to 9 and A to F
Example: $1A_{16} = 26_{10}$

8. Flow Control

1. if Statement
if (condition) {
 // code
}
2. if-else Statement
if (condition) {
 // code
} else {
 // code
}
3. if-else if Ladder
if (cond1) {
 // code
} else if (cond2) {
 // code
} else {
 // code
}

Sandeep Chaurasiya

9. Transfer Statements

1. break → terminates the loop
2. continue → skips current iteration
3. return → exits from method
4. switch → multiple choices

Example:

```

for (int i = 1; i <= 5; i++) {
    if (i == 3)
        continue;
    if (i == 5)
        break;
    System.out.println(i);
}
  
```

10. Pattern ProgramsExample:

```

  *
 * *
* * *
* * * *
  
```

```

    1
   1 2
  1 2 3
 1 2 3 4
1 2 3 4 5
  
```

Sandeep Chaurasiya

11. Methods

- A method is a block of code that performs a specific task.
- It improves reusability and readability.

Example:

```

class Demo {
    // method without return type
    void show() {
        System.out.println("Hello Method");
    }

    // method with return type
    int add(int a, int b) {
        return a + b;
    }
}

class Test {
    public static void main(String[] args) {
        Demo d = new Demo();
        d.show();
        int sum = d.add(10, 20);
        System.out.println("Sum = " + sum);
    }
}
  
```

Sandeep Chaurasiya

Sandeep Chaurasiya

1. Variable Types in Java

A variable is a named memory location used to store data.

Types of Variables :-

1. Local Variable → Declared inside a method/ constructor/block.
2. Instance Variable → Declared inside class but outside method.
3. Static Variable → Declared inside class with static keyword.

Type	Belongs to	Default Value	Lifetime
Local	Method / Block	No	Method execution
Instance	Object	Yes	Object lifetime
Static	Class	Yes	Class lifetime

Example :-

```
class Test {  
    int a = 10; // instance variable  
    static int b = 20; // static variable  
    void show() {  
        int c = 30; // local variable  
        System.out.println(a + " " + b + " " + c);  
    }  
}
```

Sandeep Chaurasiya

Sandeep Chaurasiya

2. Classes & Objects

- Class is a blueprint or template.
- Object is an instance of class.
- We can access members (variables/methods) through object.

Example :-

```
class Student {  
    String name;  
    int age;  
    void study() {  
        System.out.println(name + " is studying");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student(); // object creation  
        s1.name = "Sandeep";  
        s1.age = 22;  
        s1.study();  
    }  
}
```

Sandeep Chaurasiya

Sandeep Chaurasiya

3. Constructors

- Constructor is a special method used to initialize object.
- Name is same as class name.
- No return type.
- Called automatically when object is created.

Example :-

```
class Student {  
    String name;  
    int age;  
    // No-argument Constructor  
    Student() {  
        System.out.println("No-arg Constructor");  
    }  
    // Parameterized Constructor  
    Student(String name, int age) {  
        this.name = name; // this keyword  
        this.age = age;  
    }  
}  
  
// Object creation  
Student s1 = new Student();  
Student s2 = new Student("Sandeep", 22);
```

Sandeep Chaurasiya

Sandeep Chaurasiya

4. Instance Block

- Runs every time when object is created.
- Used for initialization.

Example :-

```
class Test {  
    {  
        System.out.println("Instance Block");  
    }  
    Test() {  
        System.out.println("Constructor");  
    }  
}  
  
Test t = new Test();
```

5. Static Block

- Runs when class is loaded in memory.
- Used for static initialization.
- Executes only once.

Example :-

```
class Test {  
    static {  
        System.out.println("Static Block");  
    }  
    public static void main(String[] args) {  
        System.out.println("Main Method");  
    }  
}
```

Output :-

Static Block
Main Method

Sandeep Chaurasiya

Sandeep Chaurasiya

6. this Keyword

- Refers to current object.
- Used to resolve variable conflict.
- Can call current class constructor.

Example :-

```
class Student {  
    String name;  
    int age;  
    Student(String name, int age) {  
        this.name = name; // current object variable  
        this.age = age;  
    }  
    void display() {  
        System.out.println(this.name + " " + this.age);  
    }  
    Student() {  
        this("Sandeep", 22); // constructor chaining  
    }  
}
```

Sandeep Chaurasiya

7. Packages

- Package is a group of related classes and interfaces.
- Helps in organization and avoids naming conflicts.

Example :-

```
package com.example.myapp;  
public class Hello { }
```

8. Arrays

- Array is a data structure that stores multiple values of same type.
- Fixed size.
- Index starts from 0.

Example :-

```
int[] arr = {10, 20, 30, 40, 50};  
for (int i = 0; i < arr.length; i++) {  
    System.out.println(arr[i]);  
}
```

Sandeep Chaurasiya

1. Classes & Objects

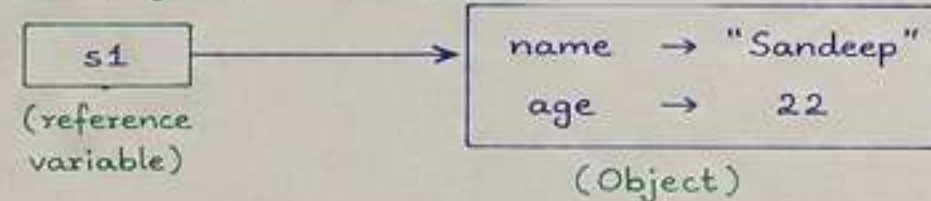
- Class is a blueprint or template.
- Object is an instance of a class.
- Class contains data (variables) and behaviour (methods).

Example :

```
class Student {
    String name;
    int age;
    void study() {
        System.out.println(name + " is studying");
    }
}
```

Creating Objects :

```
Student s1 = new Student();
s1.name = "Sandeep";
s1.age = 22;
s1.study(); // Output: Sandeep is studying
```

Memory Representation

Sandeep Chaurasiya

2. Constructors

- Constructor is a special method used to initialize objects.
- Name is same as class name.
- No return type.
- Called automatically when object is created.
- Can be overloaded.

Example :

```
class Student {
    String name;
    int age;
    // No-argument Constructor
    Student() {
        System.out.println("No-arg Constructor");
    }
    // Parameterized Constructor
    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Usage :

```
Student s1 = new Student();
Student s2 = new Student("Sandeep", 22);
```

Sandeep Chaurasiya

3. this Keyword

- 'this' refers to the current object.
- Used to resolve variable conflict.

Example :

```
class Student {
    String name;
    int age;
    Student(String name, int age) {
        this.name = name; // instance variable
        this.age = age; // instance variable
    }
    void show() {
        System.out.println(this.name + ":" + this.age);
    }
}
Student s = new Student("Sandeep", 22);
s.show();
```

4. Instance Block

- Runs when object is created, before constructor.
- Used for initialization.

Example :

```
class Test {
    {
        System.out.println("Instance Block");
    }
    Test() {
        System.out.println("Constructor");
    }
}
Test t = new Test();
```

5. Static Block

- Runs when class is loaded, before main method.
- Used for static initialization.

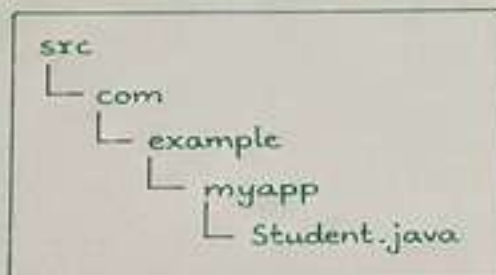
Example :

```
class Demo {
    static {
        System.out.println("Static Block");
    }
    public static void main(String[] args) {
        System.out.println("Main Method");
    }
}
// Output:
// Static Block
// Main Method
```

Sandeep Chaurasiya

6. Packages

- Package is a collection of related classes and interfaces.
- Used to organize code and avoid name conflicts.
- Built-in packages: java.lang, java.util, java.io, etc.
- User-defined packages start with reverse domain name.

Creating a Package :

At the top of Student.java
package com.example.myapp;

```
public class Student {
    // class body
}
```

Using a class from another package :

```
import com.example.myapp.Student;

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
    }
}
```

Sandeep Chaurasiya

7. Arrays

- Array is a data structure that stores multiple values of same type.
- Fixed size.
- Index starts from 0.

1. One Dimensional Array

```
int[] arr = {10, 20, 30, 40, 50};
Index : 0 1 2 3 4
Value : 10 20 30 40 50
Traverse :
for(int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

2. Two Dimensional Array

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
Traverse :
for(int i = 0; i < matrix.length; i++) {
    for(int j = 0; j < matrix[i].length; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
```

3. Jagged Array

```
int[][] jagged = new int[3][];
jagged[0] = new int[2];
jagged[1] = new int[3];
jagged[2] = new int[4];
```

Example Values :

```
jagged[0][0] = 10; jagged[0][1] = 20;
jagged[1][0] = 30; jagged[1][1] = 40; jagged[1][2] = 50;
jagged[2][0] = 60; jagged[2][1] = 70; jagged[2][2] = 80; jagged[2][3] = 90;
```

4. Array of Objects

```
class Student { String name;
    Student(String name) { this.name = name; } }

Student[] students = new Student[2];
students[0] = new Student("Sandeep");
students[1] = new Student("Rahul");
```

```
System.out.println(students[0].name); // Sandeep
System.out.println(students[1].name); // Rahul
```

Sandeep Chaurasiya

Java Abstract Class

1. What is Abstract Class?

- An abstract class is a class that is declared with the abstract keyword.
- It can have abstract methods (without body) and concrete methods (with body).
- We cannot create an object of abstract class.
- It must be inherited by a subclass.

Syntax :

```
abstract class ClassName {
    abstract void methodName(); // abstract method
    void normalMethod() {      // concrete method
        // body
    }
}
```

Example :

```
abstract class Animal6 {
    abstract void sound();

    void eat() {
        System.out.println(
            "Animal is eating");
    }
}

class Dog6 extends Animal6 {
    @Override
    void sound() {
        System.out.println(
            "Dog is barking");
    }
}
```

Output :

Dog is barking
Animal is eating

2. Abstract Class Constructor

- An abstract class can have a constructor.
- When child class object is created, abstract class constructor is called automatically.
- Abstract class constructor is used to initialize its instance variables.

Example :

```
abstract class Animal7 {
    String name;
    int age;

    Animal7(String name, int age) {
        this.name = name;
        this.age = age;
        System.out.println("Animal7 Constructor");
    }

    abstract void sound();
    void eat() {
        System.out.println(name + " is eating");
    }
}

class Dog7 extends Animal7 {
    String breed;

    Dog7(String name, int age, String breed) {
        super(name, age);
        this.breed = breed;
        System.out.println("Dog7 Constructor");
    }

    @Override
    void sound() {
        System.out.println(name + " is barking");
    }
}
```

Output :

Animal7 Constructor
Dog7 Constructor

Tommy is barking
Tommy is eating

When Dog7 object is created →
1. Animal7() constructor called (by super)
2. Then Dog7() constructor called

3. Complete Abstraction (Using Interface)

- In interface, all methods are abstract by default.
- A class implements interface and provides implementation.
- This is called complete abstraction.

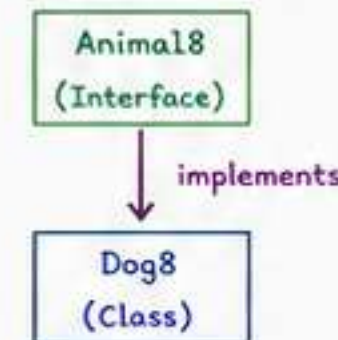
Example :

```
interface Animal8 {
    void sound();
    void eat();
    void sleep();
}

class Dog8 implements Animal8 {
    public void sound() {
        System.out.println("Dog is barking");
    }
    public void eat() {
        System.out.println("Dog is eating");
    }
    public void sleep() {
        System.out.println("Dog is sleeping");
    }
}
```

Output :

Dog is barking
Dog is eating
Dog is sleeping



Interface
→ complete abstraction
→ methods have no body
→ implementation provided by class

4. Interface in Java

Example :

```
interface Animal9 {
    void sound();
    void eat();
}

class Dog9 implements Animal9 {
    public void sound() {
        System.out.println("Dog is barking");
    }
    public void eat() {
        System.out.println("Dog is eating");
    }
}
```

Output :

Dog is barking
Dog is eating

Important Rules

1. Use 'implements' keyword.
2. All methods in interface are public and abstract by default.
3. Implemented methods must be public.
4. Interface object cannot be created directly.
(Animal9 a = new Animal9(); ✗)
5. Reference can be created.
Animal9 a = new Dog9(); ✓

Example :

```
interface Animal10 {
    void sound();

    default void eat() {
        System.out.println("Animal is eating");
    }

    static void sleep() {
        System.out.println("Animal is sleeping");
    }

    private void msg() {
        System.out.println("Private method");
    }

    default void show() {
        msg(); // calling private method
    }
}

class Dog10 implements Animal10 {
    public void sound() {
        System.out.println("Dog is barking");
    }
    public void eat() {
        System.out.println("Dog is eating");
    }
}
```

Example :

```
interface Animal10 {
    void sound();
    default void eat() {
        System.out.println("Animal is eating");
    }
    static void sleep() {
        System.out.println("Animal is sleeping");
    }
    private void msg() {
        System.out.println("Private method");
    }
}
```

5. Interface Methods (Java 8+)

Type	Keyword	Body	Override?	How to call
Abstract Method	(default)	No	Compulsory	By object
Default Method	default	Yes	Optional	By object
Static Method	static	Yes	No	By Interface name
Private Method	private	Yes	No	Inside Interface only

Output :

Dog is barking
Dog is eating
Animal is sleeping
Private method

Animal10.sleep();
d.show();

6. Interface Variables

- All variables in interface are by default public, static and final.
- Value cannot be changed.

Example :

```
interface Animal12 {
    int AGE = 5;
    String NAME = "Tommy";
    String COLOR = "Brown";
}

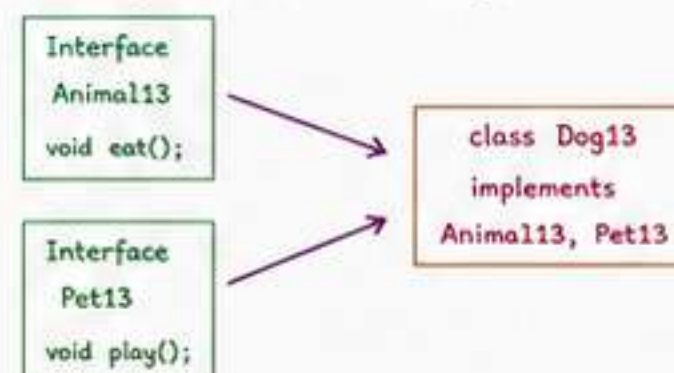
class Dog12 implements Animal12 {
    void display() {
        System.out.println("Name : " + NAME);
        System.out.println("Age : " + AGE);
        System.out.println("Color : " + COLOR);
    }
}
```

Output :

Name : Tommy
Age : 5
Color : Brown

Effectively,
int AGE;
becomes
public static final int AGE;
(automatically)

7. Multiple Inheritance using Interface



```
class Dog13 implements Animal13, Pet13 {
    public void eat() {
        System.out.println("Dog is eating");
    }
    public void play() {
        System.out.println("Dog is playing");
    }
}
```

Output :

Dog is eating
Dog is playing

Important Points (Summary)

- * Abstract class → partial abstraction (can have concrete methods).
- * Abstract class object cannot be created.
- * Abstract class can have constructor.
- * Interface → complete abstraction.
- * Interface methods → abstract, default, static, private.
- * Interface variables → public static final.
- * Multiple inheritance → possible through interface.

Note :

Interface cannot be declared final.

✗

Java OOP Concepts - Part 1

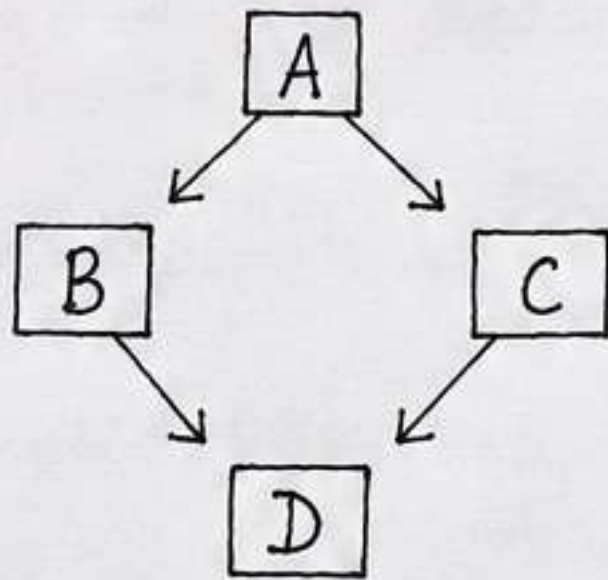
Name : SANDEEP

Date : ___/___/2024

Page No. : 1

1. Multiple Inheritance through Interface

→ Java does not support multiple inheritance through classes because it can create ambiguity (**Diamond Problem**).



If both B and C have a method show(), then class D will get confused which show() to use.

→ To avoid this ambiguity, Java does not allow multiple inheritance using classes.

→ But Java supports multiple inheritance through interfaces.

A class can **implement** multiple interfaces.

```
interface A {  
    void show();  
}  
interface B {  
    void display();  
}  
class C implements A, B {  
    public void show() {  
        System.out.println("Show");  
    }  
    public void display() {  
        System.out.println("Display");  
    }  
}  
C obj = new C();  
obj.show();           // Show  
obj.display();        // Display
```

Important Rules

- ① A class can implement multiple interfaces.
- ② A class cannot extend multiple classes.
- ③ A class first extends one class (if any) and then implements multiple interfaces.

Not Allowed

```
class C extends A, B {  
    // Error  
}
```

Advantages of Interface Multiple Inheritance

- ★ It achieves 100% abstraction.
- ★ It avoids ambiguity because interfaces only declare methods (no implementation in abstract methods).
- ★ It provides flexibility to implement multiple behaviours.

★ Interface = contract, Class = implementation 😊

Java OOP Concepts - Part 2

Name : SANDEEP
Date : ___/___/2024
Page No. : 2

2. Default Methods in Interface (Java 8)

Before Java 8, interface methods were abstract (no body).

From Java 8, we can provide a default implementation in interface using the keyword **default**.

```
interface Vehicle {  
    void start(); // abstract method  
    default void stop(){  
        System.out.println("Vehicle stopped");  
    }  
}
```

```
class Car implements Vehicle {  
    public void start(){  
        System.out.println("Car started");  
    }  
    // stop() is not implemented, still we can use it  
}  
  
public class Test {  
    public static void main(String[] args){  
        Car c = new Car();  
        c.start(); // Car started  
        c.stop(); // Vehicle stopped  
    }  
}
```

Why default methods?

- ★ To provide backward compatibility,
- ★ To add new functionality in existing interfaces without breaking old classes.
- ★ Implementing classes are not forced to override the method.

Important Point

If a class implements an interface, it must implement all **abstract methods**, but it is not compulsory to implement **default methods**.

3. Conflict in Default Methods

If two interfaces have same default method, implementing class must override it. Otherwise, compilation error occurs.

```
interface A {  
    default void show(){  
        System.out.println("A");  
    }  
}
```

```
interface B {  
    default void show(){  
        System.out.println("B");  
    }  
}
```

Without overriding:

```
class C implements A, B { }  
// Error: duplicate default  
method show()
```

Solution: Override the method.

```
class C implements A, B {  
    @Override  
    public void show(){  
        System.out.println("C");  
    }  
}
```

Calling specific interface default method

We can call a particular interface default method using **InterfaceName.super.methodName()**;

```
class C implements A, B {  
    @Override  
    public void show(){  
        A.super.show(); // prints A  
        B.super.show(); // prints B  
    }  
}
```

Output:

A
B

4. Static Methods in Interface

- Interfaces can have static methods.
- These methods belong to interface, not to implementing classes.
- We can call them using **InterfaceName.methodName()**;
- They cannot be overridden.

Note:

We cannot call static method through object.
MathUtil obj = new MathUtil(); → **Not allowed**
Always use → **InterfaceName.methodName()**;

```
interface MathUtil {  
    static int add(int a, int b){  
        return a + b;  
    }  
}  
  
public class Demo {  
    public static void main(String[] args){  
        int sum = MathUtil.add(10, 20);  
        System.out.println(sum); // 30  
    }  
}
```

Java OOP Concepts - Part 3

Name : SANDEEP

Date : __/__/2024

Page No. : 3

5. Encapsulation

Encapsulation means binding data (variables) and methods together and restricting direct access to some of the object's components.

Why Encapsulation?

- ★ To protect data from being modified by accident.
- ★ To achieve data hiding.
- ★ To control access to data.
- ★ To achieve better validation and flexibility.

Important Points

- ① Variables should be declared as private.
- ② Provide public getter and setter methods.
- ③ We can perform validation inside setter methods.
- ④ Encapsulation helps in data hiding and making the code more secure and maintainable.

Example :

```
class Student {  
    private int age;    // private data  
  
    // getter method  
    public int getAge(){  
        return age;  
    }  
  
    // setter method with validation  
    public void setAge(int age){  
        if(age >= 0 && age <= 100){  
            this.age = age;  
        } else {  
            System.out.println("Invalid age");  
        }  
    }  
}
```

6. Abstraction

Abstraction is the process of hiding implementation details and showing only essential features to the user.

Ways to achieve Abstraction in Java

1. Abstract Class (0 to 100% abstraction)
2. Interface (100% abstraction)

Using Abstract Class

```
abstract class Shape {  
    abstract void draw(); // abstract method  
  
    void color(){  
        System.out.println("Coloring shape");  
    }  
}  
  
class Circle extends Shape {  
    @Override  
    void draw(){  
        System.out.println("Drawing Circle");  
    }  
}
```

Using Interface

```
interface Shape {  
    void draw(); // abstract method  
}  
  
class Rectangle implements Shape {  
    @Override  
    public void draw(){  
        System.out.println("Drawing Rectangle");  
    }  
}
```

Key Difference

Abstraction focuses on what an object does, not how it does.

7. Composition vs Inheritance

Inheritance (IS-A Relationship)

One class inherits another class.

Example :

Animal (Parent Class)

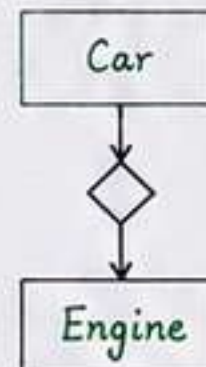
Dog (Child Class)

```
class Animal {  
    void eat(){...}  
}  
class Dog extends Animal {  
    void bark(){...}  
}
```

Composition (HAS-A Relationship)

One class has another class as a part.

Example :



```
class Engine {  
    void start(){...}  
}  
class Car {  
    Engine engine; // HAS-A  
    void drive(){...}  
}
```

Key Differences

Feature	Inheritance (IS-A)	Composition (HAS-A)
Relationship	IS-A (is a type of)	HAS-A (has a type of)
Achieved using	extends keyword	object reference
Coupling	Tight coupling	Loose coupling
Flexibility	Less flexible	More flexible
Reuse	Code reuse through inheritance hierarchy	Code reuse through object composition
Best use	When child is a specialized form of parent	When class contains another as a part

★ Conclusion : Use inheritance when you have IS-A relationship.
Use composition when you have HAS-A relationship.



Java OOP Concepts - Part 4

Name : SANDEEP

Date : __/__/2024

Page No. : 4

8. Sealed Classes (Java 17)

Sealed classes allow us to restrict which classes can extend or implement a class or interface.

Why Sealed Classes?

- ★ To provide controlled inheritance.
- ★ To increase type safety.
- ★ To express intent clearly.
- ★ To make code more secure and maintainable.

Sealed classes are used when we know all possible subtypes at compile time and we want to restrict others.

9. Rules of Sealed Classes

- ① A sealed class or interface must specify permitted subclasses using the 'permits' keyword.
- ② Every direct subclass of a sealed class must be declared as one of the following:
 - final
 - sealed
 - non-sealed
- ③ A non-sealed class can be extended by any class.
- ④ A final class cannot be extended.
- ⑤ A sealed class can be extended only by permitted classes.
- ⑥ If a sealed class extends another class, that class must be either sealed or non-sealed.

Example with all three options:

```
sealed class Shape permits Circle, Rectangle, Line { }
final class Circle extends Shape { } // final
sealed class Rectangle extends Shape permits Square { } // sealed
final class Square extends Rectangle { } // final
non-sealed class Line extends Shape { } // non-sealed
class StraightLine extends Line { } // allowed (Line is non-sealed)
```

10. Sealed Interface

```
sealed interface Payment permits UPI, Card {
    void pay();
}
final class UPI implements Payment {
    public void pay(){
        System.out.println("Paid through UPI");
    }
}
final class Card implements Payment {
    public void pay(){
        System.out.println("Paid through Card");
    }
}
```

Only UPI and Card can implement Payment. Other classes cannot implement it.

Sealed interface is also useful to restrict implementation.

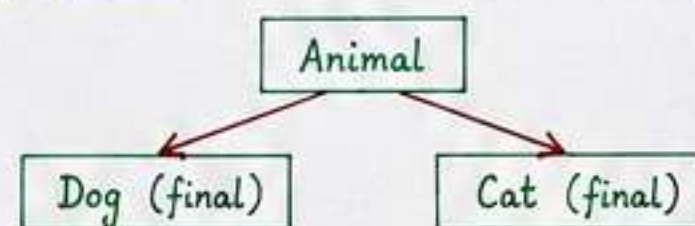
Simple Example :

```
sealed class Animal permits Dog, Cat {
    void sound(){
        System.out.println("Animal Sound");
    }
}
final class Dog extends Animal { } // cannot be extended
final class Cat extends Animal { } // cannot be extended
class Tiger extends Animal { } // Compile Error (Not permitted)
```

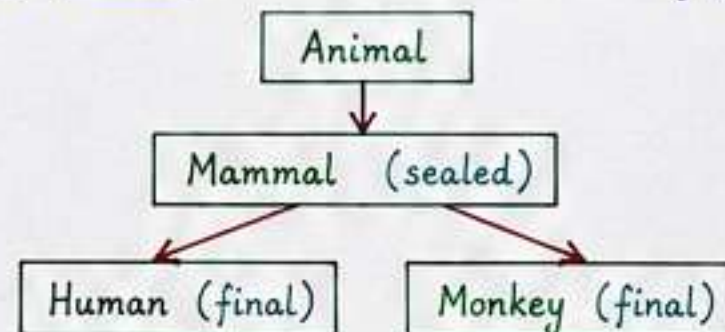
Only Dog and Cat can extend Animal because they are listed in the permits clause.

Types of Permitted Subclasses

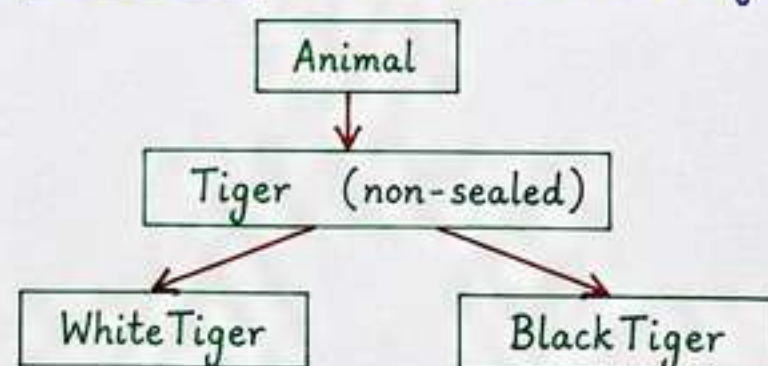
- ① final → Cannot be extended further.



- ② sealed → Can be extended by permitted classes only.



- ③ non-sealed → Can be extended by any class.



11. Sealed vs Final Classes

Feature	Sealed Class	Final Class
Inheritance	Can be extended by permitted classes only.	Cannot be extended at all.
Purpose	Control who can extend.	Complete restriction.
Permits	Must explicitly specify permitted subclasses.	No need.
Flexibility	More flexible.	Less flexible.
Example	sealed class A permits B, C { }	final class A { }

Key Takeaways

- Sealed classes help in creating controlled and predictable class hierarchies.
- They are very useful in frameworks, libraries and domain modeling.
- They improve code readability, safety and maintainability.

"Design your hierarchy today, to avoid bugs tomorrow."



- Sandeep

Java OOP Concepts - Part 5

Name : SANDEEP

Date : ___/___/2024

Page No. : 5

12. Summary of All Concepts

1. Multiple Inheritance through Interface

- ★ A class can implement multiple interfaces.
- ★ It provides the ability of multiple inheritance.

```
interface A { void showA(); }
interface B { void showB(); }
class C implements A, B {
    public void showA() { System.out.println("A"); }
    public void showB() { System.out.println("B"); }
}
```

2. Default / Static Methods in Interface

Default Method :

- ★ Has body and can be inherited by implementing class.
- ★ Used to provide default behaviour.

Static Method :

- ★ Belongs to interface, not to implementing class.
- ★ Called using interface name.

```
interface Test {
    default void hello() { System.out.println("Hello"); }
    static void info() { System.out.println("Static Method"); }
}
```

Call → Test.info(); // Hello() can be used through object

3. Encapsulation

Encapsulation means binding data and methods together and restricting direct access to some of the object's components.

Benefits :

- ★ Protects data from being modified by accident.
- ★ Achieves data hiding.
- ★ Controls access to data.
- ★ Achieves better validation and flexibility.

```
class Account {
    private double balance;
    public double getBalance() { return balance; }
    public void deposit(double amt) { if(amt > 0) balance += amt; }
    public boolean withdraw(double amt) {
        if(amt > 0 && amt <= balance) { balance -= amt; return true; }
        return false;
    }
}
```

Data is hidden and accessed through methods.

4. Abstraction

Abstraction means hiding implementation details and showing only essential features to the user.

Ways to achieve Abstraction :

- ★ Abstract Class (0 to 100% abstraction)
- ★ Interface (100% abstraction)

Abstract Class

```
abstract class Shape {
    abstract void draw();
    void display() {
        System.out.println("Shape");
    }
}
```

Interface

```
interface Shape {
    void draw();
}
```

Key Difference

Abstraction focuses on what an object does, not how it does.

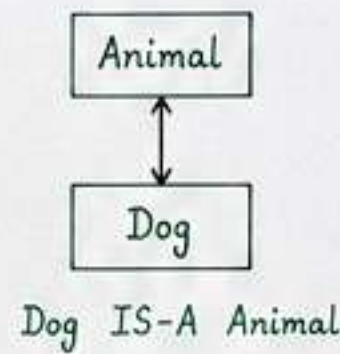
Quick Revision Table

Concept	Main Idea	Key Benefit	Example / Use
Multiple Inheritance	Implement multiple interfaces	Avoids diamond problem	class A implements X, Y
Default Method	Has body in interface	Backward compatibility	default void show() { }
Static Method	Belongs to interface	Utility method in interface	InterfaceName.method()
Encapsulation	Hide data, control access	Data protection	private variables + getters/setters
Abstraction	Hide implementation details	Simplifies complexity	abstract class / interface
Composition	HAS-A relationship	More flexible, loose coupling	class Car { Engine engine; }
Inheritance	IS-A relationship	Code reusability	class Dog extends Animal
Sealed Classes	Controlled inheritance	More predictable hierarchy	sealed class Shape permits ...

5. Composition vs Inheritance

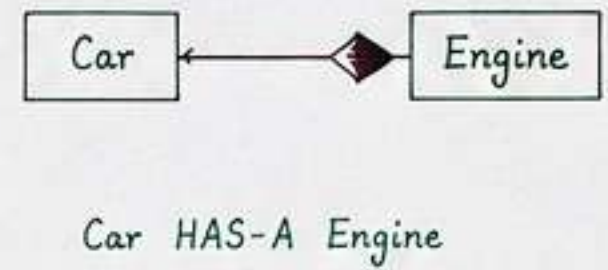
Inheritance (IS-A)

Represents IS-A relationship.



Composition (HAS-A)

Represents HAS-A relationship.



Feature	Inheritance (IS-A)	Composition (HAS-A)
Relationship	IS-A (is a type of)	HAS-A (has a type of)
Coupling	Tight Coupling	Loose Coupling
Flexibility	Less Flexible	More Flexible
Reuse	Code reuse through inheritance hierarchy	Code reuse through object composition
Best Use	When child is a specialized form of parent	When class contains another as a part

Always prefer Composition over Inheritance when possible. 😊

6. IS-A vs HAS-A

IS-A (Inheritance)

- ★ "Is a" relationship.
- ★ Dog is an Animal.
- ★ Manager is an Employee.
- ★ Use inheritance.

HAS-A (Composition)

- ★ "Has a" relationship.
- ★ Car has an Engine.
- ★ Student has an Address.
- ★ Use composition.

7. Sealed Classes (Java 17)

Sealed classes restrict which classes can extend or implement a class or interface.

Example :

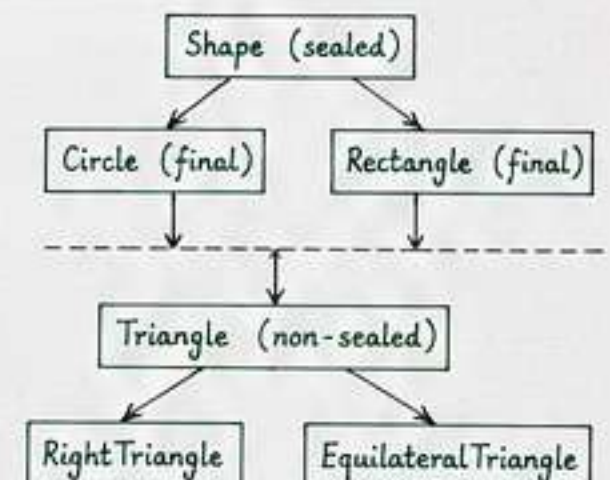
```
sealed class Shape permits Circle, Rectangle {
    void area() { System.out.println("Area"); }
}
final class Circle extends Shape { }
final class Rectangle extends Shape { }
// class Square extends Shape { } // Compile Error
```

Points

Only Circle and Rectangle can extend Shape.

Types of Permitted Subclasses :

- ① final → Cannot be extended further.
- ② sealed → Can be extended by permitted classes only.
- ③ non-sealed → Can be extended by any class.



Final Takeaway

Good OOP design is not just about writing code, but about building flexible, maintainable and secure systems.

😊 - Sandeep

1. String in Java

- Sandeep Chaurasiya

- * String is a sequence of characters.
- * It is used to store text such as names, words, sentences, etc.
- * String is not a primitive data type, it is a class in Java.
- * String is part of java.lang package. So we don't need to import it.

Example :-

String name = "Sandeep Chaurasiya";

↑ ↑ ↑

class reference String object / literal

variable

In Java, String is represented by a class.

Ways to create String :-

- ① Using String literal (stored in String Pool)
String s1 = "Hello";
- ② Using new keyword (creates new object in Heap)
String s2 = new String("Hello");
- ③ Using char array
char[] ch = {'J', 'a', 'v', 'a'};
String s3 = new String(ch);

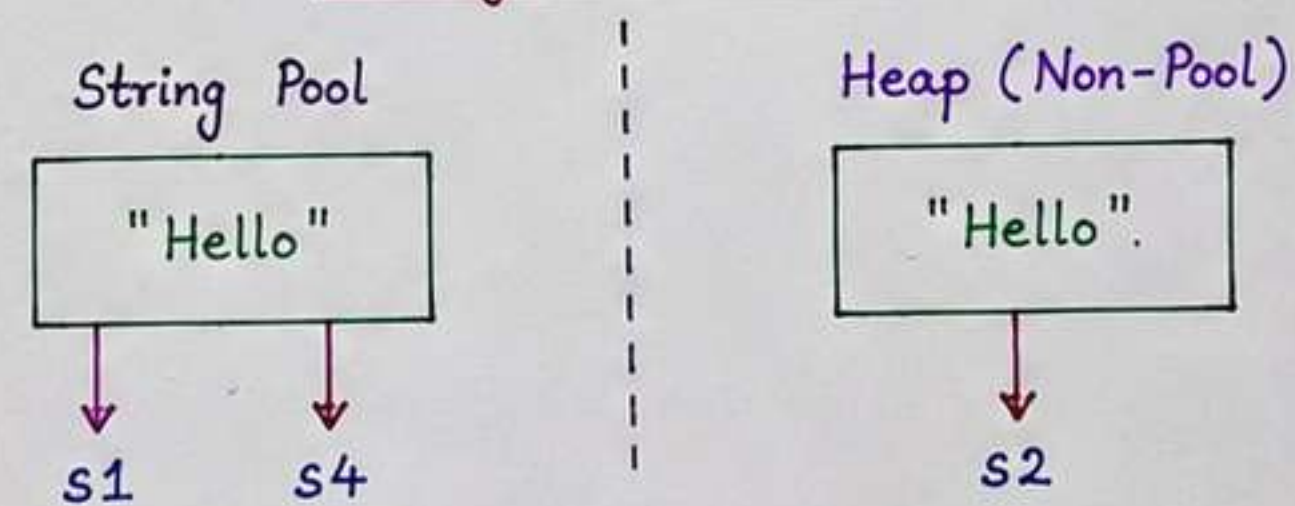
Note :-

In most cases, we use String literals because they are memory efficient due to String Pool.

Important characteristics of String :-

- ① String is immutable.
- ② String literals are stored in String Pool.
- ③ String objects created using new are stored in Heap.
- ④ String class is final (cannot be inherited).

Memory Representation



So, String literal (s1, s4) share same object from String Pool, but new keyword (s2) creates new object in Heap.

2. String Pool

- Sandeep Chaurasiya

- * Java has a special memory area called **String Pool**.
- * It is part of Heap memory.
- * String literals are stored in String Pool.
- * If same literal is created again, existing pooled object is reused.
- * This helps in **memory saving**.

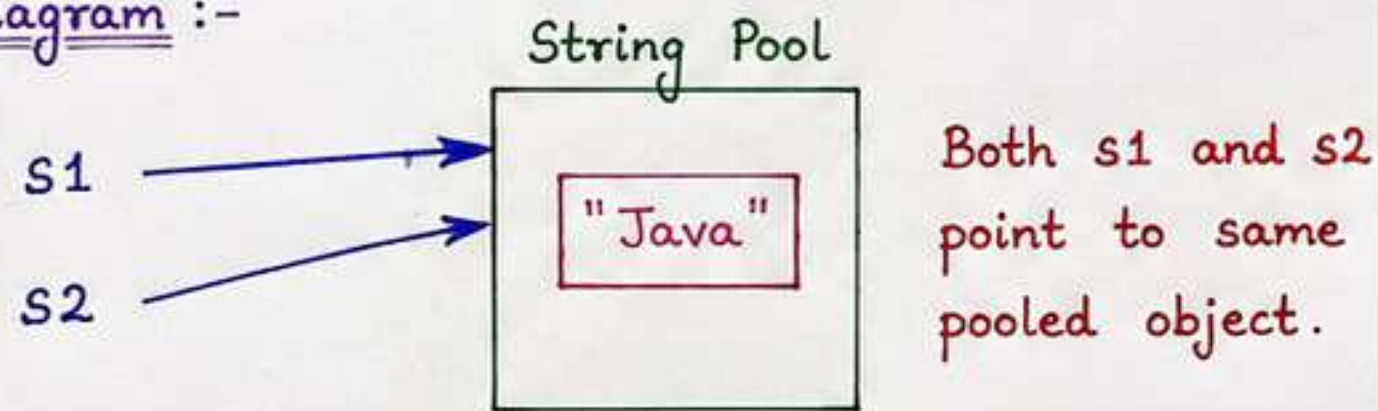
Key Point :-

Only one object for each string literal exists in String Pool.

Example 1 : Using String literal

```
String s1 = "Java";  
String s2 = "Java";  
System.out.println(s1 == s2); // true
```

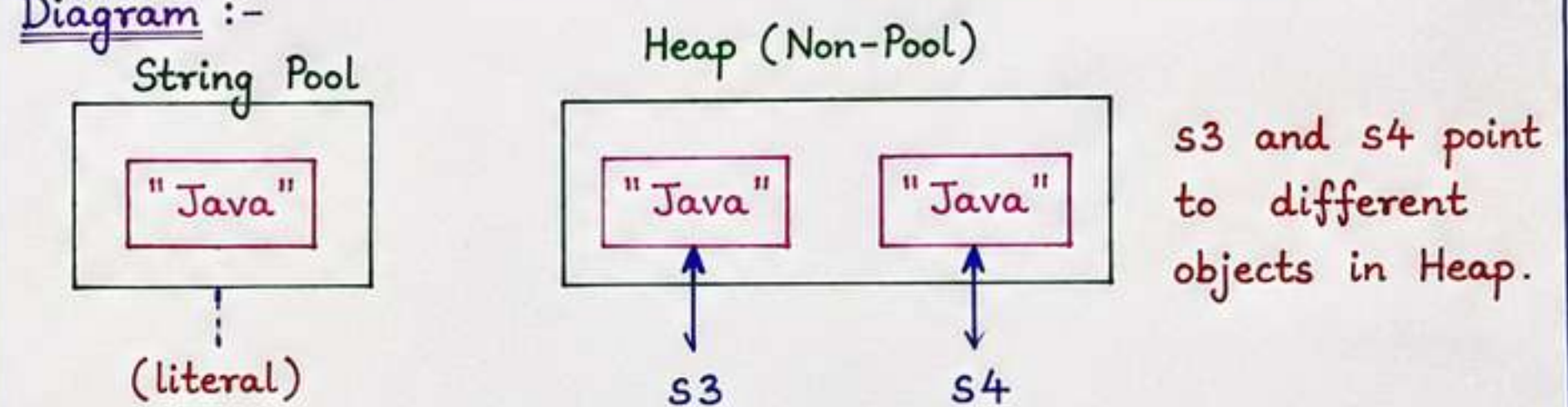
Diagram :-



Example 2 : Using new keyword

```
String s3 = new String("Java");  
String s4 = new String("Java");  
System.out.println(s3 == s4); // false
```

Diagram :-

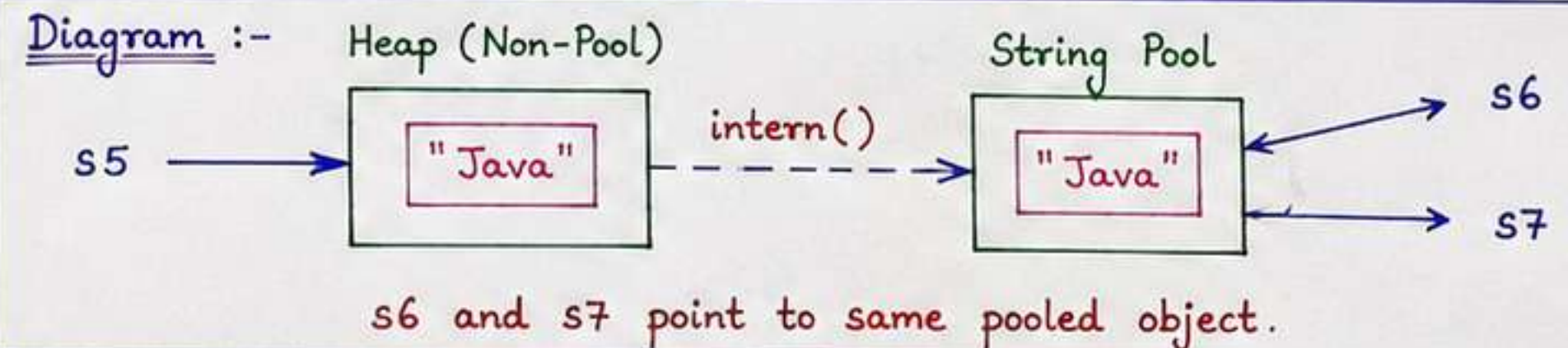


Intern() method :-

- * Intern() returns the **canonical representation** of the string from String Pool.
- * If the string already exists in pool, it returns that reference.
- * Otherwise, it adds string to pool and returns it.

```
String s5 = new String("Java");  
String s6 = s5.intern();  
String s7 = "Java";  
System.out.println(s6 == s7); // true
```

Diagram :-



Remember : Always prefer String literals instead of new String() to utilize String Pool and save memory.

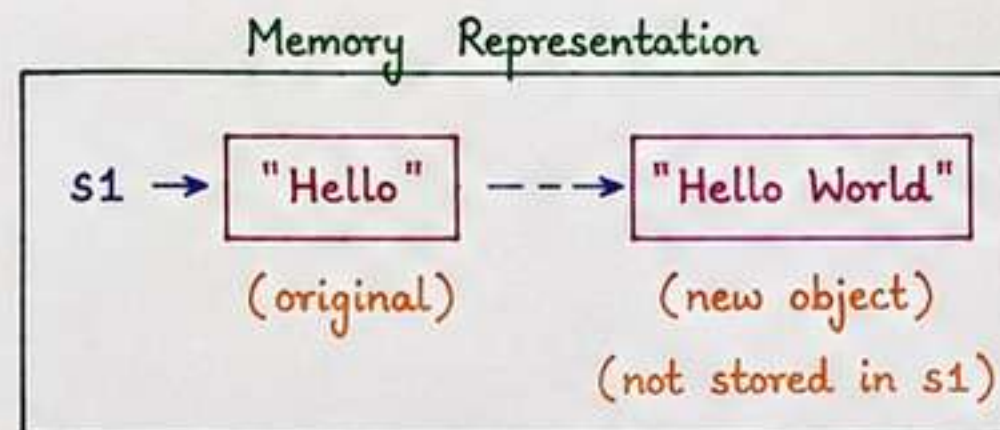
3. Immutability in String

- Sandeep Chaurasiya

- * String in Java is **immutable**.
- * Once String object is created, its content (value) cannot be changed.
- * Any operation that seems to modify String actually creates a **new String object**.
- * This is why String is safe to share (used in **String Pool**).

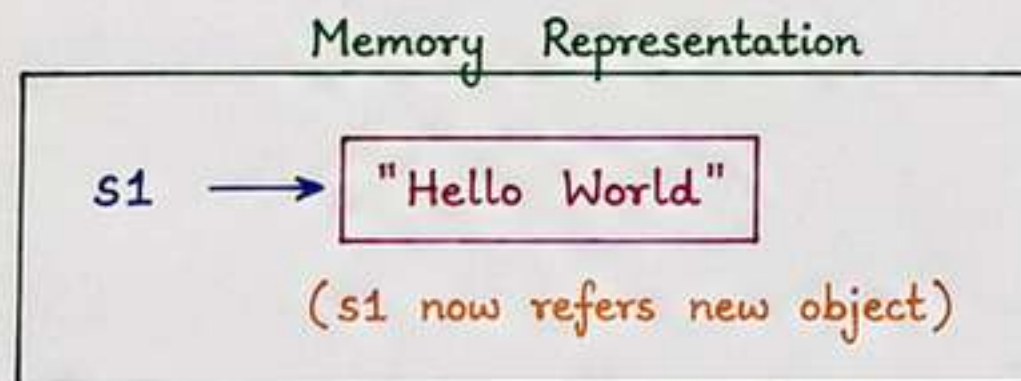
Example 1 : concat() does not change original String

```
String s1 = "Hello";  
s1.concat(" World");  
System.out.println(s1);  
// Hello
```



Example 2 : store the returned value

```
String s1 = "Hello";  
s1 = s1.concat(" World");  
System.out.println(s1); // Hello World
```



Important Note :-

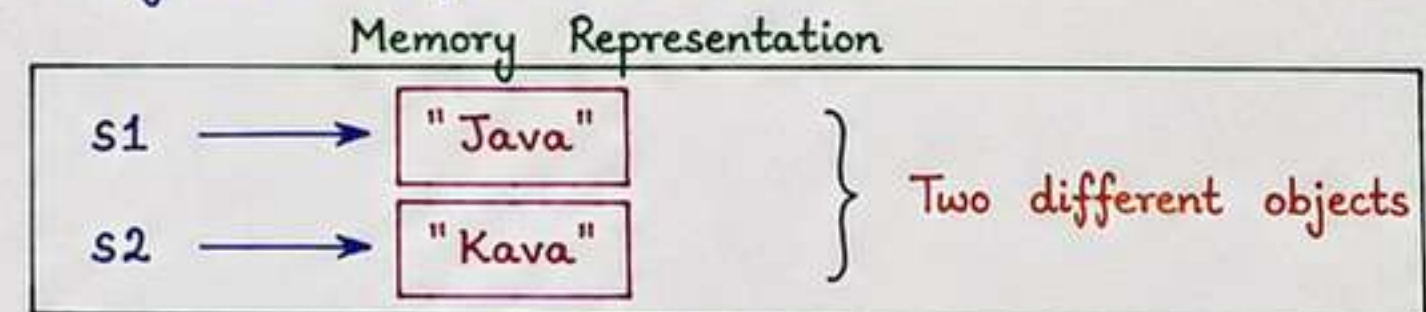
In String, old objects are never changed.
A new object is created for every modification.
This is called **Immutability**.

Why String is Immutable?

- ① **Security** → Strings are used in passwords, file paths, URLs, class names etc. If change is allowed, security issues may occur.
- ② **String Pool** → Immutable String can be shared safely. If mutable, change by one reference affects others.
- ③ **Hashing** → Used in HashMap, HashSet etc. If String changes, its hashCode may change and data structure may break.

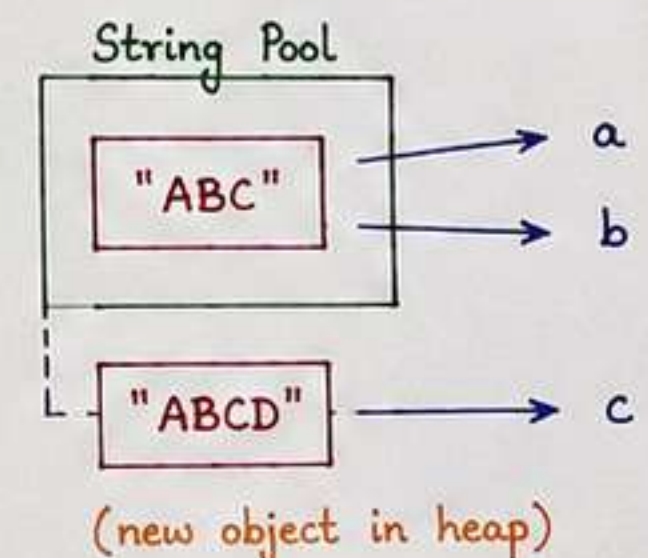
Example 3 : even replace() creates new object

```
String s1 = "Java";  
String s2 = s1.replace('J', 'K');  
System.out.println(s1); // Java  
System.out.println(s2); // Kava
```



Quick Example

```
String a = "ABC";  
String b = "ABC"; // both from pool  
String c = a + "D"; // new object  
System.out.println(a); // ABC  
System.out.println(b); // ABC  
System.out.println(c); // ABCD
```



4. == vs equals()

- Sandeep Chaurasiya

* == (reference compare)

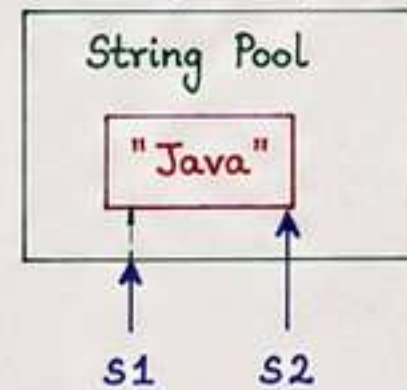
It checks whether two references point to same object or not.

It compares memory address.

Example 1 : Using String literals (from String Pool)

```
String s1 = "Java";
String s2 = "Java";
System.out.println(s1 == s2); // true
System.out.println(s1.equals(s2)); // true
```

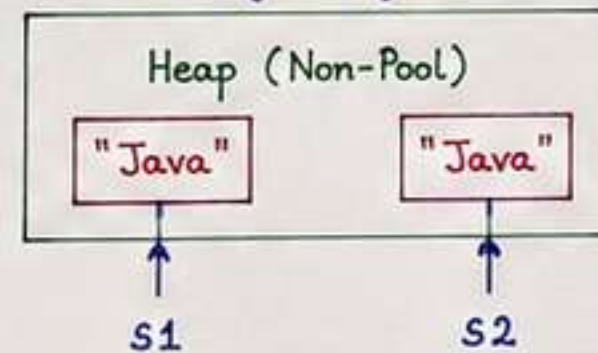
Memory Diagram



Example 2 : Using new keyword

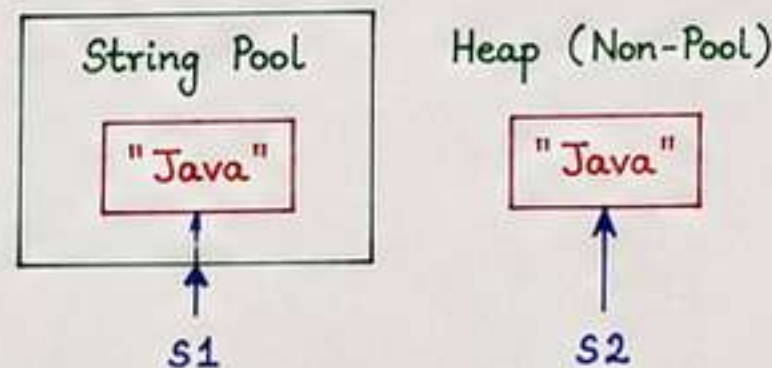
```
String s1 = new String("Java");
String s2 = new String("Java");
System.out.println(s1 == s2); // false
System.out.println(s1.equals(s2)); // true
```

Memory Diagram



Example 3 : Mixed (literal + new)

```
String s1 = "Java";
String s2 = new String("Java");
System.out.println(s1 == s2); // false
System.out.println(s1.equals(s2)); // true
```



Summary :-

- * Use == only for reference comparison.
- * Use equals() for content comparison.
- * Always prefer equals() while comparing Strings.

Other Important Points :-

- * == is not overridden in String class (it comes from Object).
- * equals() is overridden in String class to compare content.
- * equals() is case-sensitive.
- * For case-insensitive comparison, use equalsIgnoreCase().

More Examples :-

① Case sensitive

```
String a = "Java";
String b = "java";
System.out.println(a == b); // false
System.out.println(a.equals(b)); // false
System.out.println(a.equalsIgnoreCase(b)); // true
```

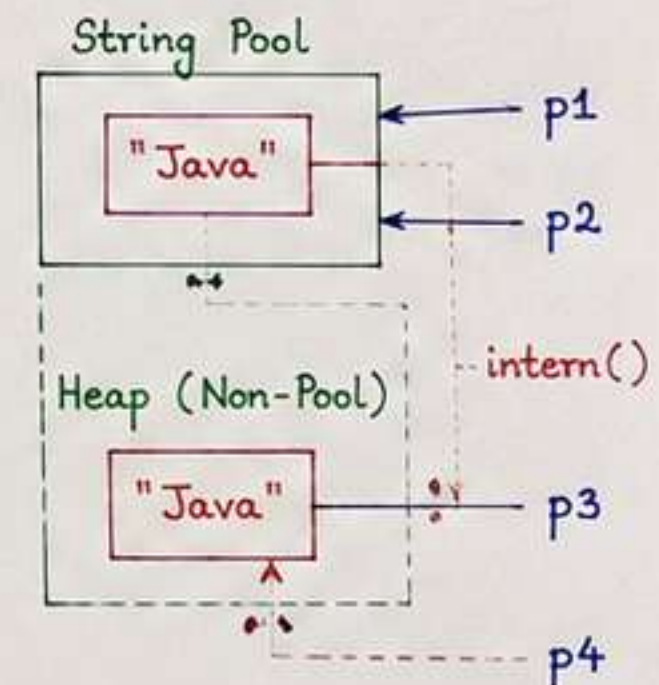
② Using equals() with NULL

```
String x = null;
String y = "Java";
// System.out.println(x.equals(y)); // NullPointerException
System.out.println(Objects.equals(x, y)); // false (safe way)
```

③ Interview Style Question

```
String p1 = "Java";
String p2 = "Java";
String p3 = new String("Java");
String p4 = p3.intern();

System.out.println(p1 == p2); // true
System.out.println(p1 == p3); // false
System.out.println(p3 == p4); // false
System.out.println(p1 == p4); // true
System.out.println(p1.equals(p3)); // true
```



Remember :- == checks "Is Same Object?" (address)
equals() checks "Is Same Content?" (value)

5. String Methods (Important Ones)

- Sandeep Chaurasiya

Method	Description	Example
① <code>length()</code>	Returns number of characters.	<code>"Java".length(); // 4</code>
② <code>charAt(index)</code>	Returns character at specified index (0-based).	<code>"Java".charAt(2); // 'v'</code>
③ <code>substring(start)</code>	Returns substring from start index to end.	<code>"Programming".substring(3); // "gramming"</code>
④ <code>substring(start, end)</code>	Returns substring from start to end-1 index.	<code>"Programming".substring(0, 4); // "Prog"</code>
⑤ <code>toUpperCase()</code>	Converts all characters to uppercase.	<code>"java".toUpperCase(); // "JAVA"</code>
⑥ <code>toLowerCase()</code>	Converts all characters to lowercase.	<code>"JAVA".toLowerCase(); // "java"</code>
⑦ <code>trim()</code>	Removes leading and trailing spaces.	<code>" hello ".trim(); // "hello"</code>
⑧ <code>contains(seq)</code>	Checks if sequence is present or not.	<code>"I love Java".contains("Java"); // true</code>
⑨ <code>startsWith(prefix)</code>	Checks if String starts with given prefix.	<code>"Hello World".startsWith("Hello"); // true</code>
⑩ <code>endsWith(suffix)</code>	Checks if String ends with given suffix.	<code>"Hello World".endsWith("World"); // true</code>
⑪ <code>equals(obj)</code>	Compares content (case-sensitive).	<code>"Java".equals("Java"); // true</code>
⑫ <code>equalsIgnoreCase(obj)</code>	Compares content (ignores case).	<code>"Java".equalsIgnoreCase("java"); // true</code>
⑬ <code>indexOf(seq)</code>	Returns first index of sequence.	<code>"banana".indexOf('a'); // 1</code>
⑭ <code>lastIndexOf(seq)</code>	Returns last index of sequence.	<code>"banana".lastIndexOf('a'); // 5</code>
⑮ <code>replace(old, new)</code>	Replaces all occurrences of old with new.	<code>"Java is easy".replace("easy", "fun"); // "Java is fun"</code>
⑯ <code>replaceAll(regex, repl)</code>	Replaces using regular expression.	<code>"a1b2c3".replaceAll("\\d", ""); // "abc"</code>
⑰ <code>split(regex)</code>	Splits string into array by regex.	<code>"a,b,c".split(","); // ["a", "b", "c"]</code>

⑱ <code>concat(str)</code>	Joins two strings.	<code>"Hello".concat(" World"); // "Hello World"</code>
⑲ <code>isEmpty()</code>	Checks if length is 0.	<code>"".isEmpty(); // true</code>
⑳ <code>isBlank()</code>	Checks if empty or only white spaces.	<code>" ".isBlank(); // true</code>

Conversions :-

- * String to char[] :

```
String s = "Java";
char[] arr = s.toCharArray(); // ['J', 'a', 'v', 'a']
```
- * String to int :

```
String numStr = "100";
int num = Integer.parseInt(numStr); // 100
```
- * int to String :

```
int n = 100;
String str1 = String.valueOf(n); // "100"
String str2 = Integer.toString(n); // "100"
```

Because String is Immutable :-



String Concatenation :-

- * Using + operator

```
String a = "Hello";
String b = "Java";
String c = a + " " + b; // "Hello Java"
```
- * Using concat()

```
String c = a.concat(" " + b); // "Hello Java"
```

Note :-

For many concatenations inside loop, String creates many temporary objects. Use StringBuilder for better performance.

6. StringBuilder in Java

- Sandeep Chaurasiya

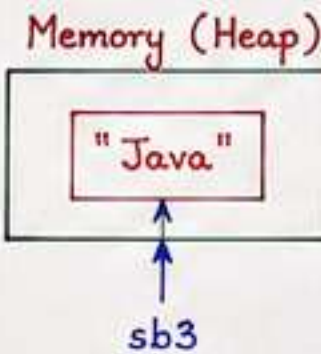
- * StringBuilder is a **mutable** sequence of characters.
- * It is used to create and modify String.
- * It is **not synchronized** (not thread-safe).
- * It provides better performance than String when we modify String repeatedly.

Key Point :-

Use StringBuilder when we are working in a single-threaded environment.

Creating StringBuilder

```
StringBuilder sb1 = new StringBuilder();
StringBuilder sb2 = new StringBuilder(" ");
StringBuilder sb3 = new StringBuilder("Java");
```



Important Methods of StringBuilder

No.	Method	Description	Example
①	append(value)	Appends the given value at the end.	sb.append(" World"); // Hello World
②	insert(index, value)	Inserts value at the specified position.	sb.insert(5, ","); // Hello, World
③	delete(start, end)	Deletes characters from start (inclusive) to end (exclusive).	sb.delete(5, 11); // Hello
④	deleteCharAt(index)	Deletes character at specified index.	sb.deleteCharAt(1); // Helo
⑤	replace(start, end, str)	Replaces characters in range with new String.	sb.replace(0, 5, "Hi"); // Hi World
⑥	reverse()	Reverses the current sequence.	sb.reverse(); // dlroW olleH
⑦	length()	Returns the number of characters.	sb.length(); // 11
⑧	charAt(index)	Returns character at given index.	sb.charAt(0); // H
⑨	toString()	Converts StringBuilder to String.	String s = sb.toString(); // "Hello World"

- Remember :-
1. Use String for immutable (unchangeable) data.
 2. Use StringBuilder for mutable data in single-threaded environment.
 3. Use StringBuffer for mutable data in multi-threaded environment.

Example :-

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");           // Hello World
sb.insert(5, ",");            // Hello, World
sb.replace(7, 12, "Java");     // Hello, Java
sb.reverse();                 // avaJ , olleH
sb.deleteCharAt(0);           // vaJ , olleH
System.out.println(sb);       // vaJ , olleH
System.out.println(sb.length()); // 9
System.out.println(sb.charAt(0)); // v
System.out.println(sb.toString()); // vaJ , olleH
```

7. StringBuffer in Java

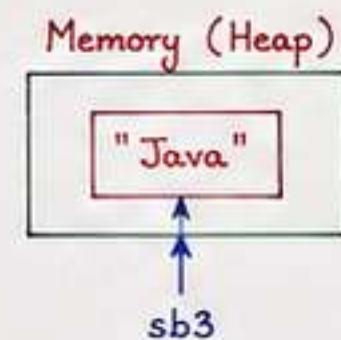
- * StringBuffer is also a **mutable** sequence of characters.
- * It is **synchronized** (thread-safe).
- * It is safe to use in multi-threaded environment.
- * It is slightly slower than StringBuilder due to synchronization.

Key Point :-

Use StringBuffer when we are working in multi-threaded environment.

Creating StringBuffer

```
StringBuffer sb1 = new StringBuffer();
StringBuffer sb2 = new StringBuffer(10);
StringBuffer sb3 = new StringBuffer("Java");
```



Important Points (StringBuilder vs StringBuffer)

Feature	StringBuilder	StringBuffer
Thread Safety	Not synchronized	Synchronized (thread-safe)
Performance	Faster	Relatively slower
Use Case	Single-threaded environment	Multi-threaded environment
Class	java.lang.StringBuilder	java.lang.StringBuffer
Methods	Same methods (append, insert, delete, reverse, etc.)	Same methods (append, insert, delete, reverse, etc.)

Quick Tip :-

For heavy String manipulation in loops, prefer StringBuilder.



8. String vs StringBuilder vs StringBuffer

- Sandeep Chaurasiya

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread Safety	Yes (Immutable)	No (Not synchronized)	Yes (Synchronized)
Synchronization	—	No	Yes (All methods are synchronized)
Performance	Slower for modifications (create new objects)	Fastest	Slower than StringBuilder
Memory	Creates many temporary objects for changes	Uses single object (modifies in place)	Uses single object (modifies in place)
Use Case	When text is fixed (unchanged)	Frequent modifications (single-threaded)	Frequent modifications (multi-threaded)
Class	java.lang.String	java.lang.StringBuilder	java.lang.StringBuffer
Methods	Many utility methods	append, insert, delete, reverse, etc.	Same as StringBuilder (but synchronized)

When to Use What ?

- * Use **String** → when String value is constant (no frequent changes).
- * Use **StringBuilder** → when you need high performance in single-threaded environment.
- * Use **StringBuffer** → when you need thread-safe mutable String in multi-threaded environment.

Performance Example

Using **String** in loop
(slower)

```
String s = "";
for(int i=1; i<=10000; i++) {
    s = s + i; // creates many
}              // new objects
```

Using **StringBuilder** in loop
(faster)

```
StringBuilder sb = new StringBuilder();
for(int i=1; i<=10000; i++) {
    sb.append(i); // modifies same
}                // object
String result = sb.toString();
```

Remember

1. String is immutable → any change creates a new object.
2. StringBuilder is mutable → not synchronized → fastest.
3. StringBuffer is mutable → synchronized → thread-safe.

9. String Comparison Summary

Expression	Compares	Example	Result
== (reference compare)	Memory address (reference)	String a = "Java"; String b = "Java";	true
.equals()	Content (case-sensitive)	a.equals(b)	true
.equalsIgnoreCase()	Content (case-insensitive)	a.equalsIgnoreCase("java")	true
.compareTo()	Lexicographical order	"abc".compareTo("abd")	-1
.compareToIgnoreCase()	Lexicographical order (ignore case)	"abc".compareToIgnoreCase("ABC")	0

10. Miscellaneous Important Points

- * Strings are **immutable** due to security, caching (String Pool), and thread-safety.
- * String literals are stored in String Pool.
- * `new String("text")` creates object in **Heap** (not in String Pool).
- * `intern()` returns the reference of String in String Pool.
- * Always prefer String literals over `new String()` to save memory.
- * String is final class in Java (cannot be inherited).
- * **StringBuilder** and **StringBuffer** are final classes as well.

11. Quick Recap (One Line Summary)

String	→	Immutable sequence of characters.
String Pool	→	Stores String literals for memory reusability.
StringBuilder	→	Mutable, not synchronized, fast.
StringBuffer	→	Mutable, synchronized, thread-safe.
==	→	Compares references (addresses).
equals()	→	Compares content (values).

Tip Box

In interview, always explain String Pool, immutability and **==** vs **equals** first. These are the most asked concepts!

Practice Suggestion

Solve output based questions on String Pool, immutability, **==**, **equals()**, **intern()**, **StringBuilder**, **StringBuffer** to master these concepts.

1. Wrapper Classes

- Wrapper classes in Java convert primitive data types into objects.
- All wrapper classes are in `java.lang` package.

Primitive	byte	short	int	long	float	double	char	boolean
Wrapper	Byte	Short	Integer	Long	Float	Double	Character	Boolean

- Wrapper classes are `immutable` (value cannot be changed).

2. Autoboxing

- Automatic conversion of primitive into corresponding wrapper class object.

Example:

```
int a = 10;
Integer obj = a; // autoboxing
```

3. Unboxing

- Automatic conversion of wrapper class object into primitive.

Example:

```
Integer obj = 20;
int a = obj; // unboxing
```

4. Integer Caching

- Integer caching is a technique where Integer objects from `-128` to `127` are cached in `IntegerCache`.
- For these values, same object is returned every time.

Example:

```
Integer a = 100;
Integer b = 100;
System.out.println(a == b); // true (cached)

Integer x = 1000;
Integer y = 1000;
System.out.println(x == y); // false (not cached)
```

5. Object Class

- Object class is the root (superclass) of all classes.
- It is in `java.lang` package.
- Every class implicitly extends Object class.

Methods of Object class (Important ones)

- | | |
|----------------------------|------------------------------------|
| 1. <code>toString()</code> | 2. <code>equals(Object obj)</code> |
| 3. <code>hashCode()</code> | 4. <code>getClass()</code> |
| 5. <code>clone()</code> | (protected) |

"Practice is the key of success."

- Sandeep Chaurasiya

6. toString()

- Returns string representation of the object.
- Default implementation returns `ClassName@hashCode`.

Example:

```
Object obj = new Object();
System.out.println(obj.toString());

Output: java.lang.Object@1b6d3586
```

7. equals(Object obj)

- Compares content of two objects.
- Default implementation compares references (`==`).
- We should override it for meaningful comparison.

Example:

```
String s1 = new String("Sandeep");
String s2 = new String("Sandeep");

System.out.println(s1.equals(s2)); // true
```

8. hashCode()

- Returns hash code (int value) of object.
- Objects that are equal must have same hashCode.

Example:

```
String s = "Sandeep";
System.out.println(s.hashCode());
```

9. getClass()

- Returns Class object associated with the object.
- We can get class name using `getClass().getName()`.

Example:

```
String s = "Hello";
System.out.println(s.getClass().getName());

Output: java.lang.String
```

10. clone()

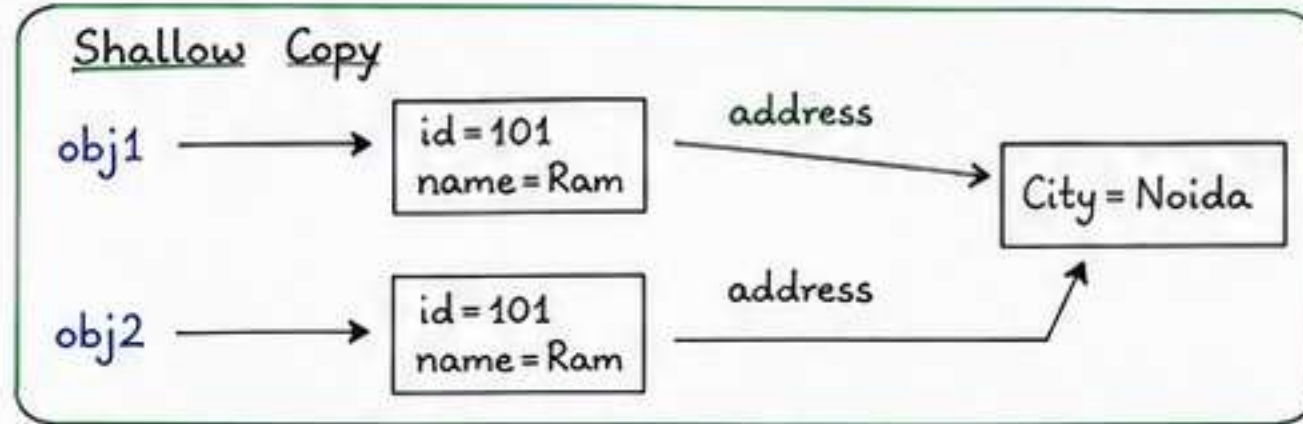
- Creates a copy of the object.
- Defined in `Object` class as protected method.
- Class must implement `Cloneable` interface to use `clone()`.

Example:

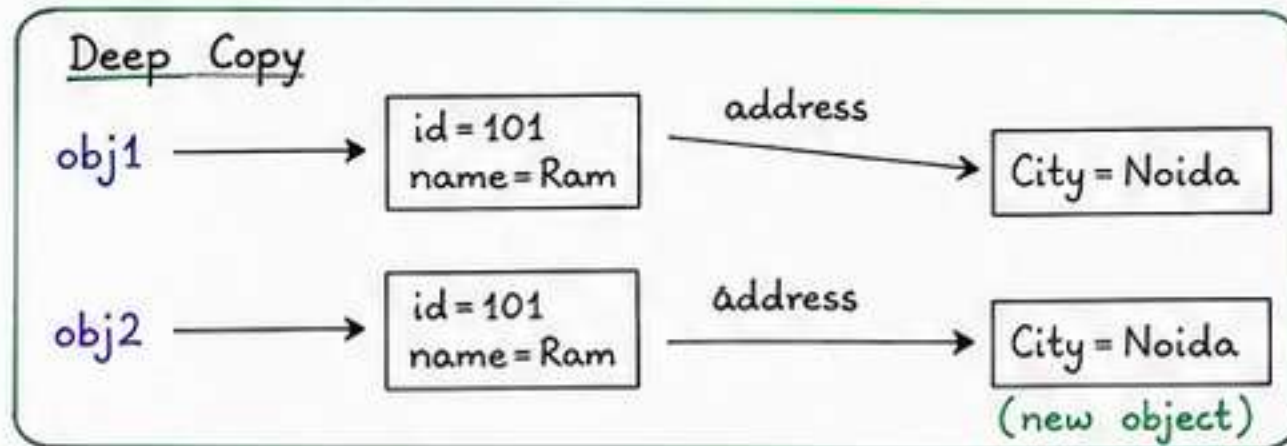
```
class Student implements Cloneable {
    int id;
    String name;
    Student(int id, String name){
        this.id = id; this.name = name;
    }
    public Object clone() throws CloneNotSupportedException{
        return super.clone();
    }
}
```

11. Shallow Copy vs Deep Copy

- Shallow Copy** : Creates a copy of object but if object has reference type members, references are copied not actual objects.



- Deep Copy** : Creates completely new copy including referenced objects.



12. Marker Interface

- An interface with no methods and no fields is called marker interface.
- It provides information (marks) to the JVM.

Examples: Cloneable, Serializable, RandomAccess

13. Enum in Java

- Enum is a special data type that represents a fixed set of constants.
- Improves code readability and type safety.

Example:

```
enum Day {
    MONDAY, TUESDAY, WEDNESDAY,
    THURSDAY, FRIDAY, SATURDAY, SUNDAY
}
```

```
Day d = Day.MONDAY;
System.out.println(d); // Output : MONDAY
```

14. Enum Constructor and Fields

Example:

```
enum Size {
    SMALL(10), MEDIUM(20), LARGE(30);
    private int value;
    Size(int value){
        this.value = value;
    }
    public int getValue(){
        return value;
    }
}
```

Use:

```
Size s = Size.MEDIUM;
System.out.println(s);
System.out.println(s.getValue());
```

Output:

```
MEDIUM
20
```

15. Important Enum Methods

- values()** : Returns all enum constants as an array.

Example:

```
for(Day d : Day.values()){
    System.out.print(d + " ");
}
```

Output:

```
MONDAY TUESDAY
WEDNESDAY THURSDAY
FRIDAY SATURDAY SUNDAY
```

- valueOf(String name)** : Returns enum constant of given name.

Example:

```
Day d = Day.valueOf("MONDAY");
System.out.println(d);
```

Output:

```
MONDAY
```

- ordinal()** : Returns position (index) of enum constant.

Example:

```
System.out.println(Day.MONDAY.ordinal());
System.out.println(Day.WEDNESDAY.ordinal());
```

Output:

```
0
2
```

- name()** : Returns exact name of enum constant.

Example:

```
System.out.println(Day.FRIDAY.name());
```

Output:

```
FRIDAY
```

16. EnumSet

- EnumSet is a special Set implementation for enum types.
- It is highly optimized and very efficient.

Example:

```
EnumSet<Day> days = EnumSet.of(Day.SATURDAY, Day.SUNDAY);
System.out.println(days);
EnumSet<Day> allDays = EnumSet.allOf(Day.class);
System.out.println(allDays);
```

Output:

```
[SATURDAY, SUNDAY]
[MONDAY, TUESDAY, WEDNESDAY, THURSDAY,
FRIDAY, SATURDAY, SUNDAY]
```

17. EnumMap

- EnumMap is a Map implementation where keys must be of enum type.
- It is fast and efficient.

Example:

```
EnumMap<Day, String> schedule = new EnumMap<>(Day.class);
schedule.put(Day.MONDAY, "Java");
schedule.put(Day.TUESDAY, "Spring Boot");
schedule.put(Day.WEDNESDAY, "SQL");
System.out.println(schedule);
```

Output:

```
{MONDAY=Java, TUESDAY=Spring Boot, WEDNESDAY=SQL}
```

EnumSet vs EnumMap

EnumSet	EnumMap
Implements Set	Implements Map
Stores only enum values	Stores enum keys with values
No duplicate elements	Keys are unique
Example: EnumSet<Day>	Example: EnumMap<Day, String>
Highly efficient for enum	Highly efficient for enum keys

18. Inner Class (Non-static Inner Class)

- A class defined inside another class is called Inner Class.
- It has access to all members (including private) of outer class.

Syntax :

```
class Outer {
    private int x = 100;
    class Inner {
        void display(){
            System.out.println("x = " + x);
            // can access
        }
    }
}
```

Example :

```
public class Test {
    public static void main(String[] args){
        Outer o = new Outer();
        Outer.Inner i = o.new Inner();
        i.display();
    }
}
```

Output : x = 100

19. Static Nested Class

- A static class inside another class is called Static Nested Class.
- It can access only static members of outer class directly.
- No object of outer class is required to access static nested class.

Syntax :

```
class Outer {
    static int x = 50;
    static class Nested {
        void display(){
            System.out.println("x = " + x);
        }
    }
}
```

Example :

```
public class Test {
    public static void main(String[] args){
        Outer.Nested n = new Outer.Nested();
        n.display();
    }
}
```

Output : x = 50

20. Anonymous Inner Class

- A class without name. Used when we need to override method of class or implement interface only once.
- It is written at the time of object creation.

Example with Abstract Class :

```
abstract class Animal{
    abstract void sound();
}
```

Use :

```
Animal a = new Animal(){
    void sound(){
        System.out.println("Dog barks");
    }
};
a.sound();
```

Output : Dog barks

Example with Interface :

```
interface Greeting{
    void sayHello();
}
```

Use :

```
Greeting g = new Greeting(){
    public void sayHello(){
        System.out.println("Hello Sandeep");
    }
};
g.sayHello();
```

Output : Hello Sandeep

Inner Class vs Static Nested Class

Inner Class	Static Nested Class
1. Non-static class inside another class	1. Static class inside another class
2. Needs outer class object	2. No need of outer class object
3. Can access all members of outer	3. Can access only static members
4. Syntax : outerObj.new Inner()	4. Syntax : Outer.Nested obj = new Outer.Nested();

"Keep practicing, Keep growing!"

- Sandeep Chaurasiya

21. Exception Handling

- Exception is an unwanted or unexpected event that disrupts the normal flow of program.
- Exception Handling helps in maintaining normal flow.

21.1. try-catch

```
try {
    // risky code
} catch (Exception e){
    // handle exception
}
```

Example :

```
try {
    int a = 10/0;
} catch (ArithmeticException e){
    System.out.println("Cannot divide by zero");
}
```

Output : Cannot divide by zero

21.2. Multiple catch

```
try {
    int arr[] = {1,2,3};
    System.out.println(arr[5]);
} catch (ArithmeticException e){
    System.out.println("Arithmetic");
} catch (ArrayIndexOutOfBoundsException e){
    System.out.println("Invalid Index");
}
```

Output : Invalid Index

21.3. try-catch-finally

```
try {
    System.out.println("In try");
    int a = 10/0;
} catch (ArithmeticException e){
    System.out.println("In catch");
} finally {
    System.out.println("In finally");
}
```

Output :

In try
In catch
In finally

Note :

finally block
always executes
whether exception
occurs or not.

21.4. throw vs throwsthrow (inside method)

- Used to explicitly throw an exception.
- Used inside method body.
- Example :

```
if (age < 18){
    throw new IllegalArgumentException("Age  
must be 18 or above");
}
```

throws (in method signature)

- Used to declare an exception.
- Used in method signature.
- Example :

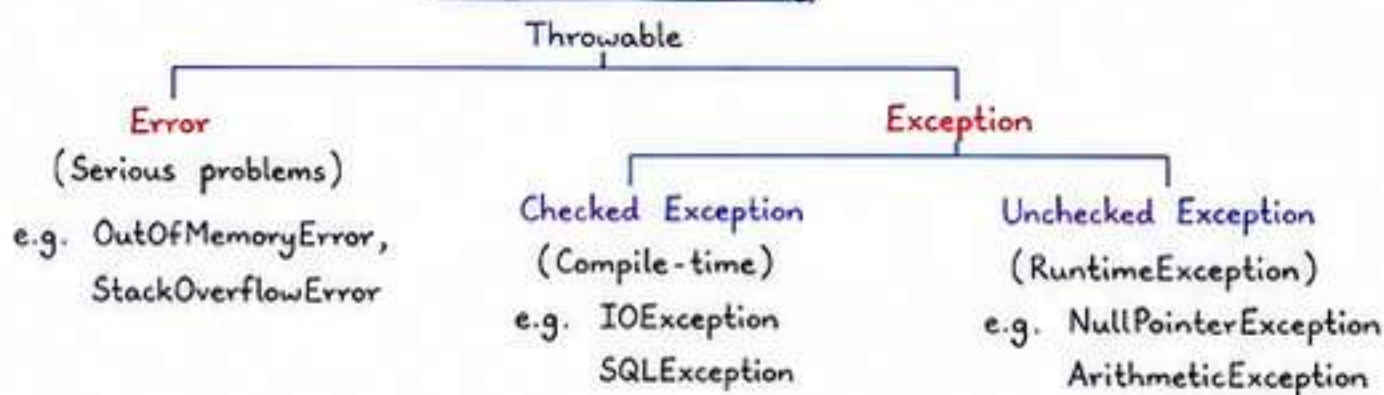
```
public void readFile()
    throws IOException{
    // code
}
```

21.5. Checked vs Unchecked ExceptionChecked Exception

- Checked at compile-time.
- Must handle or declare using throws.
- Subclasses of Exception (except RuntimeException).
- Examples : IOException, SQLException, FileNotFoundException

Unchecked Exception

- Not checked at compile-time.
- Occurs at runtime.
- Subclasses of RuntimeException.
- Examples : NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException

Exception Hierarchy

22. Summary Table (Quick Revision)

Topic	Key Points
Wrapper Classes	Convert primitive into object.
Autoboxing	Primitive → Wrapper (automatic)
Unboxing	Wrapper → Primitive (automatic)
Integer Caching	-128 to 127 cached in IntegerCache.
Object Class	Parent of all classes in Java.
clone()	Creates copy of object. Shallow by default.
Shallow Copy	Copies object but not inner referenced objects.
Deep Copy	Copies both object and referenced objects.
Marker Interface	Interface with no methods/fields (just a tag).
Enum	Fixed set of constants.
EnumSet	Set implementation for enum, very efficient.
EnumMap	Map with enum keys, very efficient.
Inner Class	Class inside another class (non-static).
Static Nested Class	Static class inside another class.
Anonymous Inner Class	Class without name, used once.
Exception Handling	try-catch, throws, throw, finally, etc.

23. Important Points to Remember

- Wrapper classes are immutable.
- == compares reference for objects, equals() compares content.
- hashCode() must be consistent with equals().
- Always override equals() and hashCode() together.
- clone() gives shallow copy, for deep copy we need custom logic.
- Checked Exception → compile time, Unchecked → runtime.
- Enum improves code readability and type safety.
- EnumSet & EnumMap are specialized & highly optimized.

"Practice is the key to success."

- Sandeep Chaurasiya

24. Bonus - Small Example Collection

(1) Autoboxing & Unboxing

```
Integer a = 10;    // autoboxing
int b = a;         // unboxing
System.out.println(a + b); // 20
```

(2) equals() vs ==

```
String s1 = new String("Hello");
String s2 = new String("Hello");
System.out.println(s1 == s2);    // false
System.out.println(s1.equals(s2)); // true
```

(3) Enum Example

```
enum Status { SUCCESS, FAILURE, PENDING }
Status s = Status.SUCCESS;
System.out.println(s);           // SUCCESS
System.out.println(s.ordinal()); // 0
```

(4) EnumSet Example

```
EnumSet<Status> set = EnumSet.of(Status.SUCCESS,
                                   Status.FAILURE);
System.out.println(set);
EnumSet<Status> all = EnumSet.allOf(Status.class);
System.out.println(all);
```

(5) EnumMap Example

```
EnumMap<Status, Integer> map = new EnumMap<>(Status.class);
map.put(Status.SUCCESS, 100);
map.put(Status.FAILURE, 0);
System.out.println(map);
```

(6) Inner Class Example

```
class Outer {
    int x = 10;
    class Inner {
        void show(){
            System.out.println("x = " + x);
        }
    }
}
Outer o = new Outer();
Outer.Inner i = o.new Inner();
i.show();    // x = 10
```

(7) Static Nested Class Example

```
class Outer {
    static int x = 50;
    static class Nested {
        void show(){
            System.out.println(x);
        }
    }
}
Outer.Nested n = new Outer.Nested();
n.show();    // 50
```

(8) Anonymous Inner Class Example

```
interface Msg {
    void display();
}
Msg m = new Msg(){
    public void display(){
        System.out.println("Hello Sandeep");
    }
};
m.display();    // Hello Sandeep
```

"Hard work + Consistency + Smart work"
= Success



25. Exception Handling - Complete Notes

- It is a mechanism to handle runtime errors and maintain normal program flow.

26. Types of Exceptions

Checked Exceptions	Unchecked Exceptions
- Checked at compile-time. - Must handle or declare using throws. - Subclasses of Exception (except RuntimeException). - Examples : IOException, SQLException, FileNotFoundException	- Not checked at compile-time. - Occurs at runtime. - Subclasses of RuntimeException. - Examples : NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException

27. try - catch

- Used to catch and handle exception.
- try block contains risky code.
- catch block handles exception.

Example :

```
try {
    int a = 10, b = 0;
    int c = a / b;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
```

Output : Cannot divide by zero

29. try - catch - finally

- finally block always executes whether exception occurs or not.
- Used for cleanup code (closing resources, files, connections etc.)

Example :

```
try {
    System.out.println("In try");
    int x = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("In catch");
} finally {
    System.out.println("In finally");
}
```

Output :

In try
In catch
In finally

28. Multiple catch

Example :

```
try {
    int arr[] = {1, 2, 3};
    System.out.println(arr[5]);
} catch (ArithmeticException e) {
    System.out.println("Arithmetic");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Invalid Index");
}
```

Output : Invalid Index

30. throw vs throws

throw	throws
- Used inside method to actually throw an exception. - Used to throw custom or specific exception. - Example : throw new IllegalArgumentException("Invalid age");	- Used in method signature to declare possible exception. - Does not throw, just declares. - Example : public void read() throws IOException { }

31. throw Example

Example :

```
public void checkAge(int age) {
    if (age < 18) {
        throw new IllegalArgumentException(
            "Age must be 18 or above");
    }
}
```

Output (if age < 18) :

Exception in thread "main" java.lang.IllegalArgumentException:
Age must be 18 or above

33. try - with - resources (Java 7+)

- Automatically closes resources.
- Used with implements AutoCloseable.

Example :

```
try (FileReader fr = new FileReader("test.txt");
    BufferedReader br = new BufferedReader(fr)) {
    System.out.println((char) br.read());
} catch (IOException e) {
    System.out.println("File error");
}
```

32. throws Example

Example :

```
import java.io.*;
public void readFile() throws IOException {
    FileReader file = new FileReader("test.txt");
}
```

Caller Code :

```
public static void main(String[] args)
    throws IOException {
    readFile();
}
```

34. Custom Exception

Example :

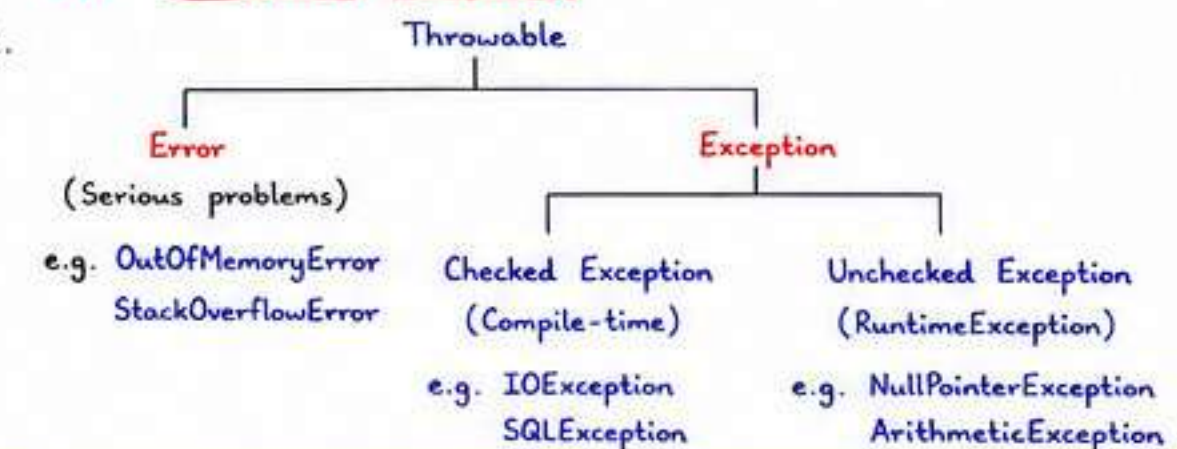
```
class InvalidAgeException extends Exception {
    InvalidAgeException(String msg) {
        super(msg);
    }
}
```

Use :

```
public void validate(int age) throws InvalidAgeException {
    if (age < 18) throw new InvalidAgeException(
        "Age must be 18 or above");
}
```

35. Important Points

- Checked Exceptions must be handled or declared.
- Unchecked Exceptions occur at runtime.
- finally block always executes.
- throw is used to throw exception.
- throws is used to declare exception.
- Custom Exceptions improve readability.

36. Exception HierarchyQuick Revision

- Wrapper Classes - Convert primitives into objects.
- Autoboxing - Primitive to Wrapper (automatic)
- Unboxing - Wrapper to Primitive (automatic)
- Integer Caching - -128 to 127 cached.
- Object Class - Parent of all classes.
- toString(), equals(), hashCode(), getClass(), clone()

- Shallow Copy - Copies references.
- Deep Copy - Copies actual objects.
- Marker Interface - Empty interface (tag).
- Enum, EnumSet, EnumMap - Special features.
- Inner, Static Nested, Anonymous Inner Classes.
- Exception Handling - try, catch, finally, throw, throws.

Topics Covered in this Notes (Page 1)

1. Clone in Java
2. Shallow Copy
3. Deep Copy
4. Marker Interface
5. Enum

6. Enum Methods
7. EnumSet
8. Enum Map

① Clone in Java

- Clone means creating a copy of an existing object.
- To use clone(), a class must implement Cloneable interface.
- The clone() method of Object class is used to create a copy.

Output:

```
101 Aryan
101 Aryan
```

// Example

```
class Student implements Cloneable {
    int id;
    String name;
    Student(int id, String name){
        this.id = id;
        this.name = name;
    }

    public Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}

public class Main {
    public static void main(String[] args) throws CloneNotSupportedException {
        Student s1 = new Student(101, "Aryan");
        Student s2 = (Student) s1.clone();

        System.out.println(s1.id + " " + s1.name); // 101 Aryan
        System.out.println(s2.id + " " + s2.name); // 101 Aryan
    }
}
```

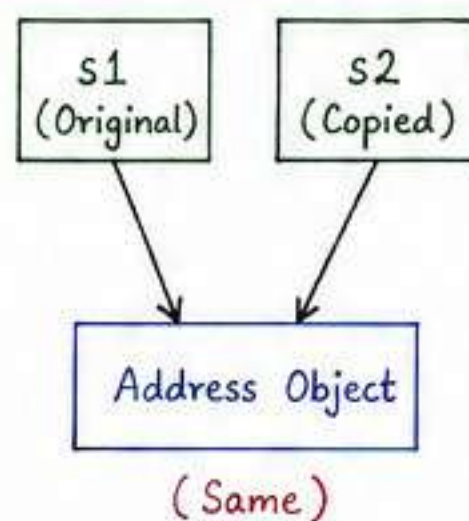
② Shallow Copy

- In shallow copy, primitive values are copied but reference variables copy the reference (address).
- Both objects point to the same referenced object.
- If we change the referenced object, both original and copied objects will be affected.

Output:

```
s1 address: Noida
s2 address: Noida
```

Shallow Copy Diagram



// Example

```
class Address {
    String city;
    Address(String city) {
        this.city = city;
    }
}

class Student implements Cloneable {
    int id;
    String name;
    Address address;
    Student(int id, String name, Address address){
        this.id = id; this.name = name; this.address = address;
    }

    public Object clone() throws CloneNotSupportedException {
        return super.clone(); // Shallow copy
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        Address a = new Address("Delhi");
        Student s1 = new Student(101, "Aryan", a);
        Student s2 = (Student) s1.clone();

        s2.address.city = "Noida";
        System.out.println("s1 address: " + s1.address.city); // Noida
        System.out.println("s2 address: " + s2.address.city); // Noida
    }
}
```

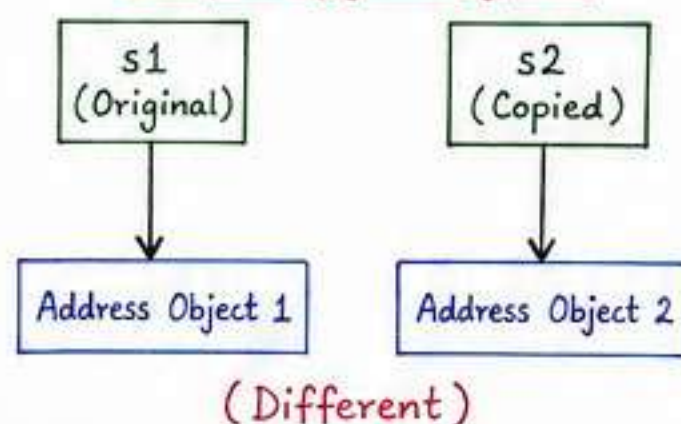
Why?

Because s1.address and s2.address are pointing to the same Address object.

③ Deep Copy

- In deep copy, a new copy of the referenced object is also created.
- Both objects point to different referenced objects.
- Changes in one object do not affect the other object.

Deep Copy Diagram



Key Difference

Shallow Copy	Deep Copy
Same referenced object	Different referenced object
Changes affect original	Changes do not affect original
Faster	Slower (more memory/time)

Note: Clone() alone gives shallow copy by default. For deep copy, we need to create new objects of referenced types manually. 😊

4) Marker Interface

- Marker Interface is an interface that does not contain any method.
- It is used to provide some special information to the JVM.
- Example: Cloneable, Serializable, Remote, RandomAccess etc.

Example:

```
// Marker Interface
interface Printable { }           // no method

class Student implements Printable {
    void show() {
        System.out.println("Student is printable");
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.show();
    }
}
```

Output:

Student is printable

Why Marker Interface?

It helps JVM to identify the capability of an object.

Example:

If class implements Cloneable, JVM knows object can be cloned.

5) Enum

- Enum is used to represent a group of fixed constants.
- It makes code more readable and type-safe.
- Enum internally extends java.lang.Enum class.

Example:

```
enum Day {
    MONDAY, TUESDAY, WEDNESDAY,
    THURSDAY, FRIDAY, SATURDAY, SUNDAY
}

public class Main {
    public static void main(String[] args) {
        Day today = Day.MONDAY;
        System.out.println(today);
    }
}
```

Output:

MONDAY

Benefits of Enum

- * Type safety
- * Readability
- * Easy to maintain
- * Prevents invalid values

Invalid Example:

```
String day = "Mondy";
// spelling mistake
// can't be caught
```

6) Enum Methods (Important Methods of Enum)

1. values() → returns all enum constants

```
enum Day { MONDAY, TUESDAY, WEDNESDAY }

public class Main {
    public static void main(String[] args) {
        for(Day d : Day.values()) {
            System.out.println(d);
        }
    }
}
```

Output:

MONDAY
TUESDAY
WEDNESDAY

2. valueOf(String name) → returns enum object for given name

```
enum Day { MONDAY, TUESDAY, WEDNESDAY }

public class Main {
    public static void main(String[] args) {
        Day d = Day.valueOf("TUESDAY");
        System.out.println(d);
    }
}
```

Output:

TUESDAY

3. ordinal() → returns position (index) of enum constant (starts from 0)

```
enum Day { MONDAY, TUESDAY, WEDNESDAY }

public class Main {
    public static void main(String[] args) {
        System.out.println(Day.MONDAY.ordinal());
        System.out.println(Day.TUESDAY.ordinal());
        System.out.println(Day.WEDNESDAY.ordinal());
    }
}
```

Output:

0
1
2

Note: We should not use ordinal() for permanent storage like database ids, because order of enum constants can be changed.

7) EnumSet

- EnumSet is a special Set implementation for enum constants.
- It is more efficient than HashSet.

Example:

```
import java.util.*;

enum Day { MONDAY, TUESDAY, WEDNESDAY,
    THURSDAY, FRIDAY, SATURDAY, SUNDAY }

public class Main {
    public static void main(String[] args) {
        EnumSet<Day> workingDays = EnumSet.of(
            Day.MONDAY, Day.TUESDAY,
            Day.WEDNESDAY, Day.THURSDAY, Day.FRIDAY
        );
        System.out.println("Working Days: " + workingDays);

        EnumSet<Day> allDays = EnumSet.allOf(Day.class);
        System.out.println("All Days: " + allDays);

        EnumSet<Day> noDays = EnumSet.noneOf(Day.class);
        System.out.println("No Days: " + noDays);
    }
}
```

Output:

Working Days: [MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY]
 All Days: [MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY]
 No Days: [I]

Note: EnumSet can use only enum type values.

8) EnumMap

- EnumMap is a Map implementation where keys are enum types.
- It is fast and efficient.

Example:

```
import java.util.*;

enum Day { MONDAY, TUESDAY, WEDNESDAY }

public class Main {
    public static void main(String[] args) {
        EnumMap<Day, String> schedule = new EnumMap<>(Day.class);
        schedule.put(Day.MONDAY, "Java");
        schedule.put(Day.TUESDAY, "SQL");
        schedule.put(Day.WEDNESDAY, "Spring");

        System.out.println(schedule);
        System.out.println("On Tuesday we study: " +
            schedule.get(Day.TUESDAY));
    }
}
```

Output:

{MONDAY=Java, TUESDAY=SQL, WEDNESDAY=Spring}
 On Tuesday we study: SQL

Note:

- EnumSet → works with enum values.
- EnumMap → works with enum keys.

9) Inner Class

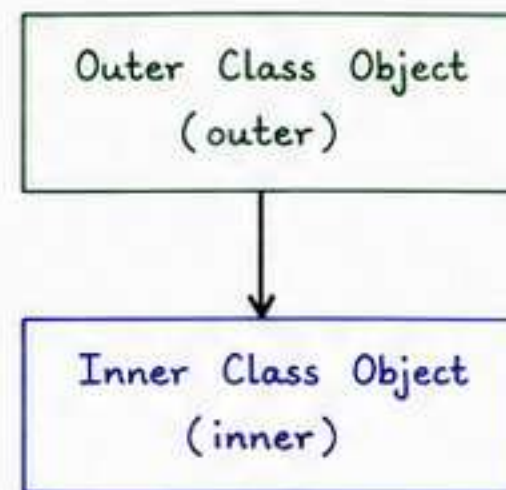
- A class defined inside another class is called Inner Class.
- Inner class can access all the members (private also) of outer class.
- To create object of inner class, we need object of outer class.

Example :

```
class Outer {
    private int number = 10;
    class Inner {
        void show() {
            System.out.println("Number = " + number);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Outer outer = new Outer();
        Outer.Inner inner = outer.new Inner();
        inner.show();
    }
}
```

How it works ?



Output :

Number = 10

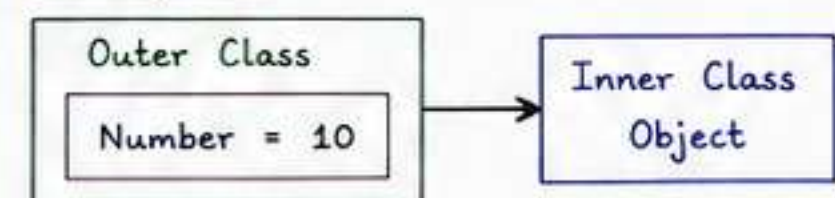
Key Points :

- Inner class is associated with an instance of outer class.
- It can access outer class members directly.
- Outer class cannot access inner class members directly.

Syntax to create object :

```
OuterClass.InnerClass obj =
    outerObject.new InnerClass();
```

Memory Idea :



Inner object holds reference to outer object.

10) Static Nested Class

- A class inside another class and declared with 'static' is called Static Nested Class.
- It can access only static members of outer class.
- To create object, we do NOT need outer class object.

Example :

```
class Outer {
    static int x = 100;
    static class Nested {
        void show() {
            System.out.println("x = " + x); // can access static
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Outer.Nested obj = new Outer.Nested();
        obj.show();
    }
}
```

Key Points :

- It behaves like a normal top-level class.
- It belongs to the outer class, but not to any outer object.

Syntax :

```
OuterClass.NestedClass obj =
    new OuterClass.NestedClass();
```

Output :

x = 100

11) Anonymous Inner Class

- A class without name is called Anonymous Inner Class.
- Mostly used to provide implementation at the time of object creation.

Example (with Interface) :

```
interface Greeting {
    void sayHello();
}

public class Main {
    public static void main(String[] args) {
        Greeting g = new Greeting() {
            public void sayHello() {
                System.out.println("Hello Aryan!");
            }
        };
        g.sayHello();
    }
}
```

Output :

Hello
Aryan!

Used when we need one-time implementation.

12) Exception Handling in Java (Complete Overview)

Exception is an unwanted event that disrupts the normal flow of the program.

1. try and catch

try → risky code
catch → handle exception

Example :

```
try {
    int result = 10 / 0;
    System.out.println(result);
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
```

Output :

Cannot divide by zero

2. finally

finally block always executes whether exception occurs or not.

Example :

```
try {
    int result = 10 / 2;
    System.out.println(result);
} catch (Exception e) {
    System.out.println("Error");
} finally {
    System.out.println("Finally executed");
}
```

Output :

5
Finally executed

3. Multiple catch

We can have multiple catch blocks for different exceptions.

Example :

```
try {
    int[] arr = {10, 20};
    System.out.println(arr[5]);
} catch (ArithmeticException e) {
    System.out.println("Arithmetic Error");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Array index error");
} catch (Exception e) {
    System.out.println("Some other error");
}
```

Output :

Array index error

4. throw

Used to throw exception manually.

Example :

```
int age = 15;
if (age < 18) {
    throw new ArithmeticException(
        "Age must be 18 or above");
}
System.out.println("Eligible");
```

Output : (Runtime Exception)

```
Exception in thread "main"
java.lang.ArithmeticException:
Age must be 18 or above
```

(Continued on next page...) →

13) Checked vs Unchecked ExceptionChecked Exception

- Checked at compile time.
- Must be handled using try-catch or throws.
- These are subclasses of Exception (but not subclasses of RuntimeException).
- Examples: IOException, SQLException, FileNotFoundException, ClassNotFoundException, Exception

Unchecked Exception

- Checked at runtime.
- Not mandatory to handle.
- These are subclasses of RuntimeException.
- Examples: ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException

Note: Error is also under Throwable but it is not meant to be handled (OutOfMemoryError, StackOverflowError).

14) Nested try

- A try block inside another try block is called Nested try.
- Inner try-catch handles its own exception.
- If not handled, it is passed to outer try.

Example:

```
public class Main {
    public static void main(String[] args) {
        try {
            int a = 10, b = 0;
            try {
                int result = a / b; // will throw ArithmeticException
            } catch (ArithmeticException e) {
                System.out.println("Inner catch: " + e.getMessage());
            }
            } catch (Exception e) {
                System.out.println("Outer catch: " + e.getMessage());
            }
        }
    }
}
```

Output:

Inner catch: / by zero

Flow:

Exception occurs in inner try
↓
Inner catch handles it
↓
Outer catch does not execute

15) Custom Exception

- We can create our own exception by extending Exception class or RuntimeException class.
- It is useful to show specific error messages.

Example:

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String message) {
        super(message);
    }
}

public class Main {
    static void checkAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or above");
        }
        System.out.println("You are eligible");
    }

    public static void main(String[] args) {
        try {
            checkAge(16);
        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output:

Age must be 18 or above

Why Custom Exception?

- * More meaningful error messages
- * Easy to debug
- * Business logic specific exceptions

16) Exception Propagation

- If an exception is not handled in a method, it is passed to the calling method.
- This continues up the call stack until it is handled.
- If not handled anywhere, program terminates abnormally.

Example:

```
public class Main {
    static void method3() {
        int result = 10 / 0; // ArithmeticException
        System.out.println(result);
    }

    static void method2() {
        method3();
    }

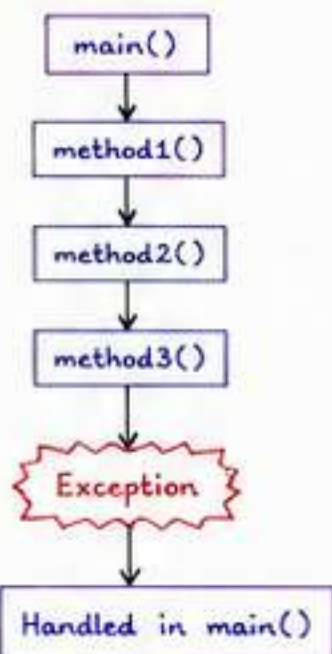
    static void method1() throws ArithmeticException {
        method2();
    }

    public static void main(String[] args) {
        try {
            method1();
        } catch (ArithmeticException e) {
            System.out.println("Exception handled in main");
        }
    }
}
```

Output:

Exception handled in main

Call Stack Flow:

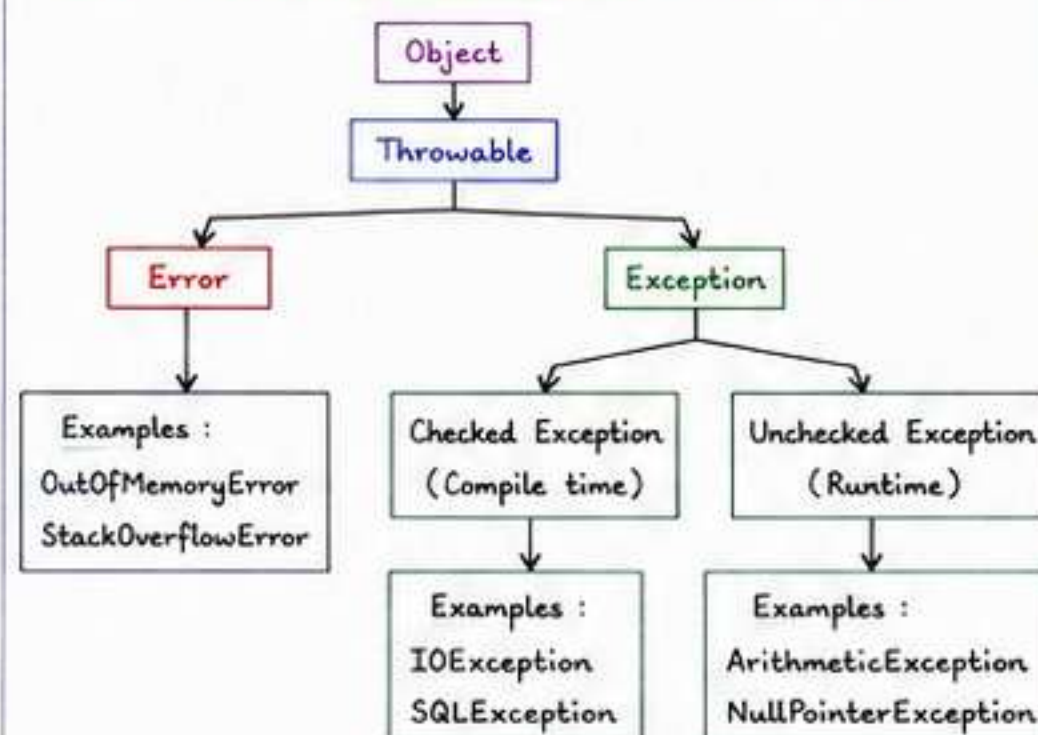


Note: Propagation helps in centralized exception handling.

17) throw vs throws

throw	throws
Actually throws an exception.	Declares that a method may throw an exception.
Used inside method body.	Used in method signature.
Throws single exception object.	Can declare multiple exceptions.
throw new ArithmeticException("Error");	void test() throws IOException

Note: Use throw to give error.
Use throws to pass responsibility.

18) Exception Hierarchy (Simplified)**19) Important Points**

- * Always handle specific exception first, then general exception.
- * finally block always executes (except in case of System.exit(0)).
- * Use meaningful messages in exceptions.
- * Prefer unchecked exceptions for programming errors and checked exceptions for recoverable conditions.

Exception Handling is not about hiding errors, it is about handling them gracefully. 😊

Phase 1 - Collection Framework Overview

* 1. What is Collection Framework?

Collection Framework is a part of java.util package. It provides architecture to store and manipulate group of objects as a single unit.

Why Collection Framework?

- Reduces programming effort
- Improves performance
- Provides standard data structures
- Highly reusable classes & interfaces

2. Collection vs Collections

Collection (Interface)	Collections (Class)
• It is an interface present in java.util package. • It is the root interface of Collection hierarchy. • It is implemented by List, Set, Queue etc.	• It is a utility class present in java.util package. • It contains only static methods. • It provides utility methods for Collection objects. • Example : sort(), reverse(), shuffle(), etc.

3. Iterable Interface

- It is the root interface of Collection Framework.
- It contains only one abstract method.

```
public interface Iterable<T> {
    Iterator<T> iterator();
}
```

Example:

```
ArrayList<String> list = new ArrayList<>();
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

Used to iterate elements.

4. Collection Interface

- It extends Iterable interface.
- It is the root interface of Collection hierarchy.
- It contains basic methods to perform operations like add, remove, size, clear, etc.

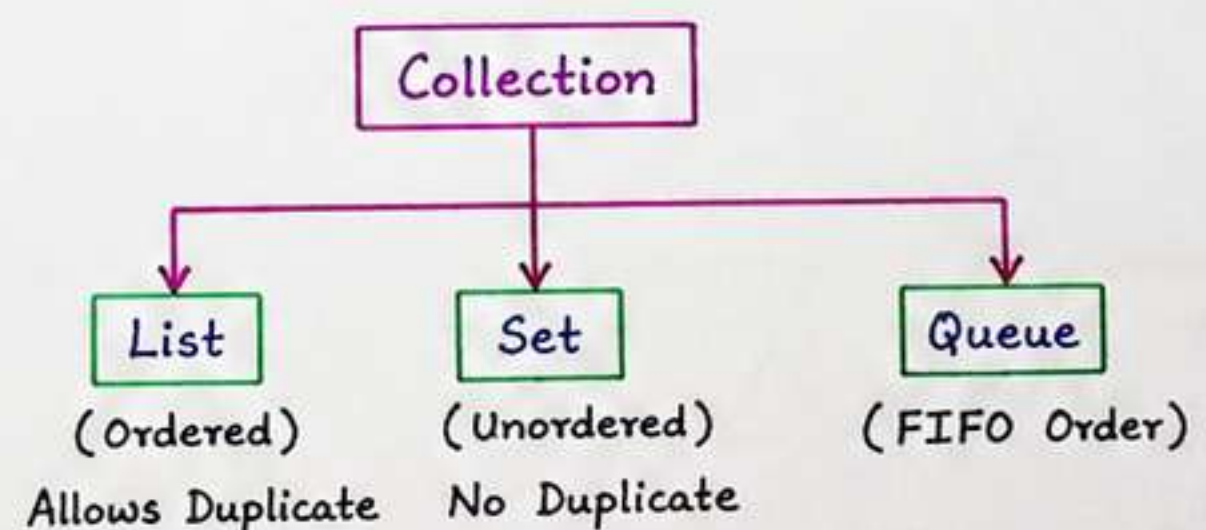
Common Methods in Collection Interface

- add(Object o)
- remove(Object o)
- contains(Object o)
- size()
- isEmpty()
- clear()
- iterator()

5. Main Sub-interfaces of Collection

Collection interface has three main sub-interfaces.

1. List → Ordered collection, allows duplicate.
2. Set → Unordered collection, does not allow duplicate.
3. Queue → Used to maintain FIFO order.



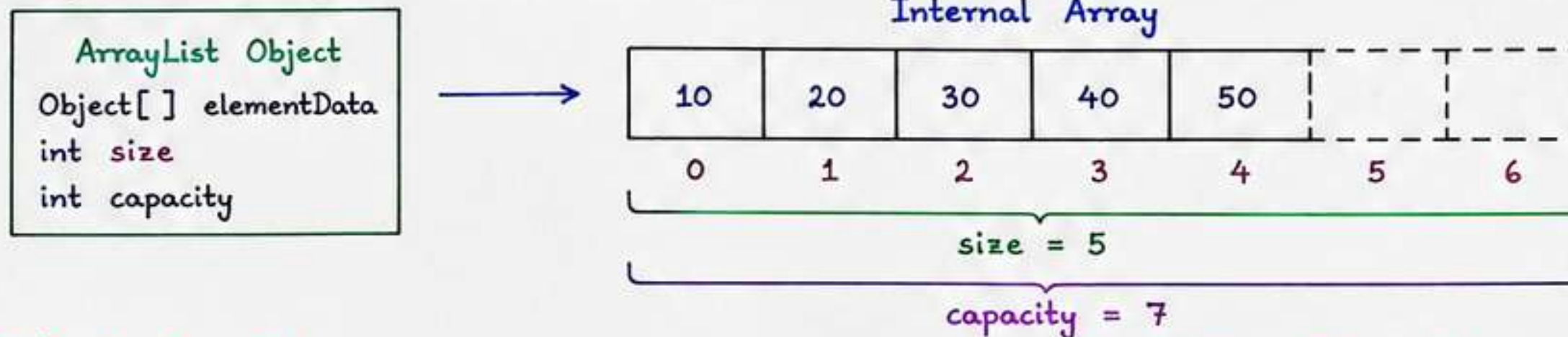
Phase 2 - ArrayList Deep Dive

1. What is ArrayList?

ArrayList is a resizable-array implementation of the List interface. It allows duplicate elements and maintains insertion order.

2. Internal Data Structure

ArrayList internally uses a dynamic array.



size → number of elements actually present
capacity → total space allocated

3. Default Capacity

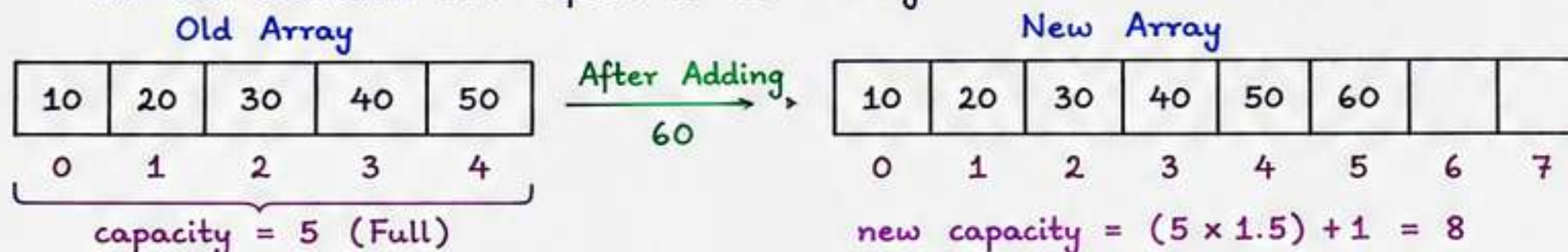
- Default initial capacity is 10.
- When we add first element, internal array of size 10 is created.

4. Adding Elements

- `add(E e)` adds element at the end.
- If `size < capacity` → simply add.
- If `size == capacity` → array grows (resizing).

5. Resizing (When Capacity is Full)

- When array is full, a new array of $(oldCapacity \times 1.5) + 1$ is created.
- All old elements are copied to new array.



Growth Formula

$$newCapacity = (oldCapacity \times 1.5) + 1$$

Example :

$$oldCapacity = 5$$

$$newCapacity = (5 \times 1.5) + 1 = 8$$

6. Common Operations

- `add(E e)` → Add element
- `get(int index)` → Get element at index
- `set(int index, E e)` → Update element
- `remove(int index)` → Remove by index
- `remove(Object o)` → Remove by value
- `contains(Object o)` → Search element
- `size()` → Number of elements
- `clear()` → Remove all elements

7. Time Complexity

- `add()` at end → $O(1)$ (amortized)
- `add()` at index → $O(n)$
- `get()` → $O(1)$
- `set()` → $O(1)$
- `remove()` at index → $O(n)$
- `remove(Object o)` → $O(n)$
- `contains()` → $O(n)$

n = number of elements

8. ArrayList vs Array

Array	ArrayList
Fixed size	Dynamic size
Stores primitives & objects	Stores only objects
Size cannot grow	Size grows automatically
Less features	Many utility methods
Better memory (less overhead)	More memory (extra overhead)

9. ArrayList vs LinkedList

ArrayList	LinkedList
Based on dynamic array	Based on doubly linked list
Random access is fast ($O(1)$)	Random access is slow ($O(n)$)
Add/remove at end is fast	Add/remove at middle is fast
Uses more memory	Uses less memory
Better for search operations	Better for frequent insert/delete

☆ **Note:** Use ArrayList when → more read/search operations.
Use LinkedList when → more insert/delete operations in middle.

Interview Tip :

Always mention time complexity, internal working & real life use cases.



Phase 3 - Hashing + hashCode() + equals() Deep Dive

1. What is Hashing?

- Hashing is a technique used to convert any large data into a fixed size integer value (hash code).
- This hash code is used to store and retrieve objects very fast.
- It is the foundation of HashMap, HashSet, Hashtable, etc.

2. Why Hashing is Required?

- To search, insert and delete data in constant time complexity.
- To handle large amount of data efficiently.
- To avoid linear search.
- Used in data structures like HashMap, HashSet, Hashtable.

Key Point :



Hashing does not guarantee uniqueness, it only helps to quickly locate the bucket.

3. hashCode() Method

- Every Object class in Java has a hashCode() method.
- It returns an integer value (hash code).
- Default implementation is provided by Object class.
- We can override it in our own class.

4. equals() Method

- equals() method is used to check logical equality between two objects.
- Default implementation of equals() in Object class is reference equality (==).
- We should override equals() for our own class.

5. Relationship between equals() and hashCode()

If two objects are equal (equals() returns true), then their hashCode() must be same.

But if hashCode() is same, equals() may or may not be true.

6. How Hashing Works Internally (Example)

Let's take a HashMap with initial capacity = 8 (index 0 to 7)
Load Factor = 0.75

capacity = 8
index = hashCode % 8
So index will be in range 0 to 7

Key → hashCode() → Hash Code (int value) → Index [hashCode % capacity] → Bucket (Store Entry)

Example :

Key	hashCode()	index = hashCode % 8
Amit	1001	1001 % 8 = 1
Rahul	1002	1002 % 8 = 2
Sandeep	1003	1003 % 8 = 3
John	1009	1009 % 8 = 1 (Collision with Amit)
Peter	1017	1017 % 8 = 1 (Collision again)

Buckets (Array of size 8)

0	1	2	3	4	5	6	7
	Amit ↓ John ↓ Peter	Rahul	Sandeep				

Collision is handled using Linked List (separate chaining)

7. Hash Collision

- When two different keys generate same index, it is called collision.
- Java resolves collision using : Separate Chaining (Linked List)
- Java 8+ uses Tree (Red-Black Tree) when bucket size > 8 and capacity >= 64.

Collision is normal in Hashing. It cannot be 100% avoided.

8. Contract of equals() and hashCode()

- If o1.equals(o2) is true, then o1.hashCode() must be equal to o2.hashCode().
- If o1.hashCode() != o2.hashCode(), then o1.equals(o2) must be false.
- If o1.equals(o2) is false, then o1.hashCode() may or may not be equal to o2.hashCode().

9. Custom Class Example (Correct Implementation)

```
class Student {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, name);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Student s = (Student) obj;
        return this.id == s.id && Objects.equals(this.name, s.name);
    }
}
```

Why Override?

- If we don't override equals() and hashCode(), HashMap/HashSet will use default implementation from Object class (reference based).
- That means two different objects with same data will be treated as different.

Common Mistakes

1. Overriding equals() but not hashCode()
2. Using mutable fields in hashCode()
3. Not following equals-hashCode contract
4. Using == for object comparison in equals()



☆ Remember : Good hashCode() + Good equals() = Reliable Collections 😊

Phase 2 - ArrayList Deep Dive (Continue...)

1. Internal Working of ArrayList

- ArrayList internally uses a dynamic array.
- It resizes automatically when it becomes full.

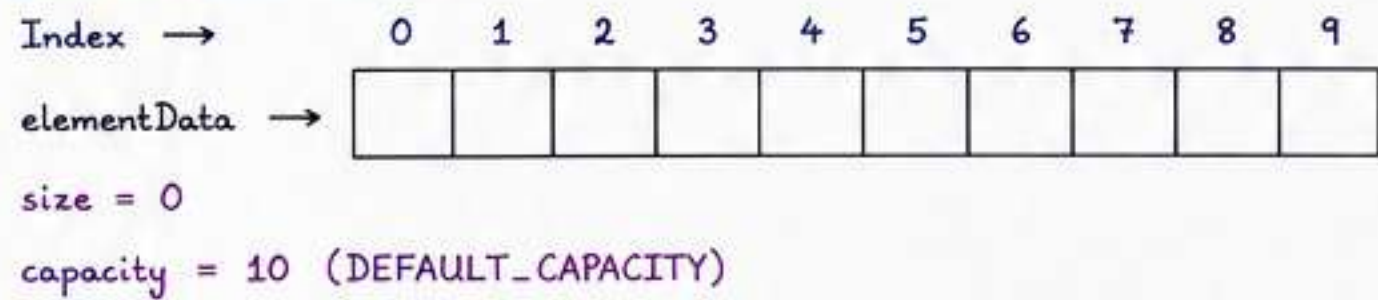
Internal Structure :

```
class ArrayList<E> {  
    transient Object[] elementData; // internal array  
    private int size;                // number of elements  
    private static final int DEFAULT_CAPACITY = 10;  
}
```

size = number of elements actually present

capacity = length of internal array

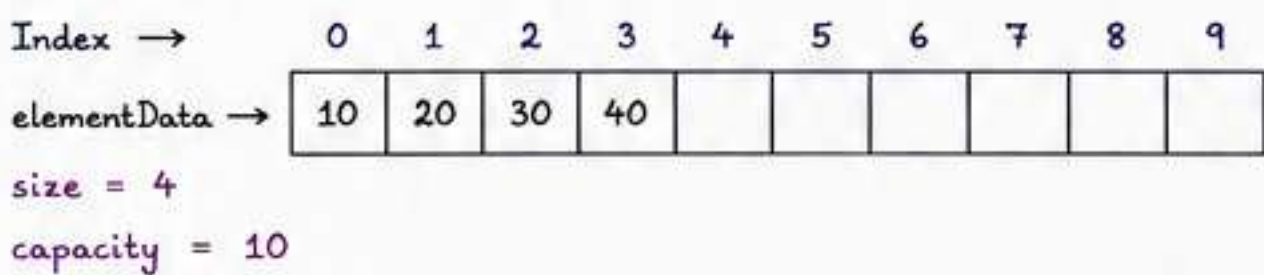
Example (Initial State) :



2. Adding Elements

- When we add elements, they are placed at the end.
- size increases by 1.
- If size == capacity → array grows (resizing).

Example : Adding 4 elements



3. Resizing / Growing ArrayList

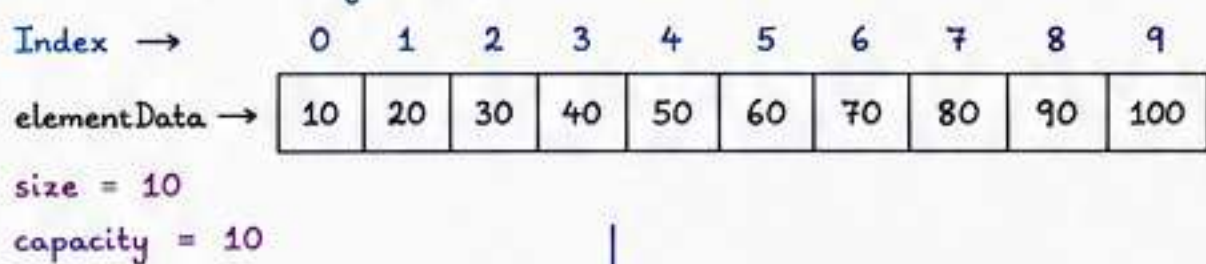
- When size == capacity, a new array of (oldCapacity × 1.5) + 1 is created.
- All old elements are copied to new array.

Growth Formula : newCapacity = (oldCapacity × 1.5) + 1

Example :

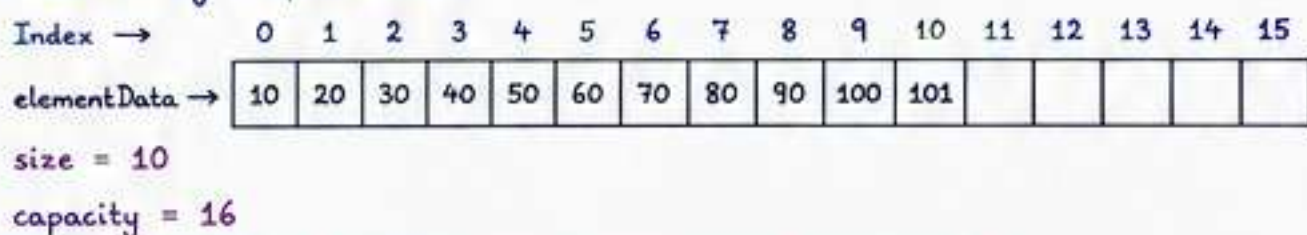
If oldCapacity = 10 newCapacity = (10 × 1.5) + 1 = 16

Old Array (Full)



After Adding 101

New Array (After Grow)



☆ Important Points :

- Resizing happens automatically.
- Old elements order is maintained.
- Copying elements during resizing takes O(n) time.

7. Real Project Use Cases of ArrayList

- Storing list of users fetched from database.
- Holding search results.
- Storing order history.
- Maintaining dynamic list in REST APIs.
- Temporary list processing.

4. Common Operations & Time Complexity

Operation	Time Complexity
add(E e) at end	O(1) amortized
add(int index, E e)	O(n)
get(int index)	O(1)
set(int index, E e)	O(1)
remove(int index)	O(n)
remove(Object o)	O(n)
contains(Object o)	O(n)
size()	O(1)
isEmpty()	O(1)
clear()	O(n)
iterator()	O(1)

Amortized O(1) means → most of the time O(1), but sometimes (during resizing) takes O(n).

5. ArrayList vs Array

	ArrayList	Array
Size	Dynamic (grows/shrinks)	Fixed
Capacity	Increases automatically	Fixed
Insertion	Slower (shifting may happen)	Faster (if space available)
Deletion	Slower (shifting may happen)	Faster (if index known)
Memory	Extra memory for capacity	No extra memory
Feature	Many utility methods	No methods

6. ArrayList vs LinkedList

	ArrayList	LinkedList
Data Structure	Dynamic Array	Doubly Linked List
Access (get/set)	Fast (O(1))	Slow (O(n))
Insertion	Slow (O(n))	Fast (O(1))
Deletion	Slow (O(n))	Fast (O(1))
Memory	Less	More (extra pointers)
Cache Performance	Better	Poor
Best Use	Frequent access	Frequent insert/delete in middle

Interview Tip :

Always mention internal array, size, capacity, resizing & time complexity.

Common Mistakes

- Thinking ArrayList is synchronized. (It is not. Use Collections.synchronizedList())
- Ignoring resizing cost in performance.
- Using ArrayList for heavy insert/delete in middle.
- Not understanding capacity vs size.

Phase 2 - ArrayList Deep Dive (Continue...)

8. ArrayList vs Vector

Feature	ArrayList	Vector
Synchronization	Not Synchronized	Synchronized (Thread-safe)
Performance	Faster	Slower
Growth	1.5 times	2 times
Legacy	Modern	Legacy Class
Iterator	Fail-Fast	Fail-Safe (Enumeration)
Best Use	Single Threaded	Multi Threaded (Legacy)

Note : In modern development, prefer ArrayList over Vector.

11. Iterator & ListIterator in ArrayList

Iterator (Forward Only) :

```
List<String> list = new ArrayList<>(List.of("A","B","C"));
Iterator<String> it = list.iterator();
while(it.hasNext()) {
    System.out.println(it.next());
}
```

Used to
traverse
elements.

ListIterator (Forward + Backward) :

```
ListIterator<String> lit = list.listIterator();
while(lit.hasNext()) {
    System.out.println(lit.next());
}
while(lit.hasPrevious()) {
    System.out.println(lit.previous());
}
```

Used when
we need both
directions.

9. ArrayList vs LinkedHashSet (Ordering + Duplicates)

Feature	ArrayList	LinkedHashSet
Duplicates	Allows Duplicates	Does not allow
Order	Insertion Order	Insertion Order
Performance (add)	$O(1)$ amortized	$O(1)$
Search (contains)	$O(n)$	$O(1)$
Best Use	When duplicates are allowed	When unique elements with order

10. Real Example

Maintain list of products in e-commerce application.

Why ArrayList ?

- Need fast access by index.
- Need ordered list.
- Duplicates like same product with different seller allowed.
- Frequent read operations (display products).
- Occasional insert/delete.

Example Code :

```
List<String> products = new ArrayList<>();
products.add("Laptop");
products.add("Mobile");
products.add("Tablet");
products.add("Mobile"); // duplicate allowed

System.out.println(products.get(1)); // Mobile
System.out.println(products);
// [Laptop, Mobile, Tablet, Mobile]
```

12. Fail-Fast Behavior

- ArrayList iterator is fail-fast.
- If list is modified structurally after iterator creation, it throws `ConcurrentModificationException`.

Example :

```
List<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
Iterator<Integer> it = list.iterator();
list.add(30); // structural modification
it.next(); // throws ConcurrentModificationException
```

Structural Modifications :

- `add()`
 - `remove()`
 - `clear()`
- (except via iterator)

13. Important Points to Remember

- ☆ ArrayList uses dynamic array internally.
- ☆ When full, capacity grows automatically.
- ☆ Growth factor = $\text{oldCapacity} \times 1.5 + 1$
- ☆ Insertion and deletion in middle is costly.
- ☆ Best for read-heavy applications.
- ☆ Not thread-safe. Use `Collections.synchronizedList()` if required.
- ☆ Prefer ArrayList over Vector.
- ☆ Use proper initial capacity if size is known.
- ☆ Iterator is fail-fast.

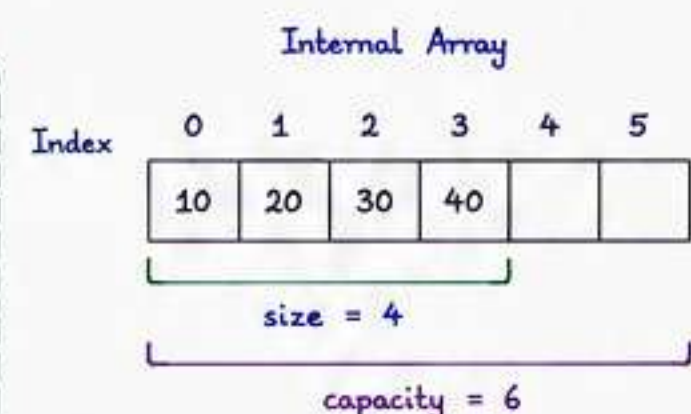
14. Interview Questions (ArrayList)



- Q1. What is ArrayList in Java?
- Q2. How ArrayList works internally?
- Q3. What is the default capacity of ArrayList?
- Q4. How ArrayList grows when full?
- Q5. What is the time complexity of `get()` in ArrayList?
- Q6. What is the time complexity of `add()` in ArrayList?
- Q7. What happens when we insert element in the middle?
- Q8. How is ArrayList different from Vector?
- Q9. How is ArrayList different from LinkedList?
- Q10. What is fail-fast behavior in ArrayList?
- Q11. What is the use of Iterator and ListIterator?
- Q12. How to make ArrayList thread-safe?
- Q13. Can we store primitives in ArrayList?
- Q14. How to set initial capacity of ArrayList?
- Q15. Write a program to remove duplicates from ArrayList.

15. Short Summary

- ArrayList is a dynamic array based list.
- It allows duplicates and maintains insertion order.
- It resizes automatically (growth factor 1.5).
- Fast random access ($O(1)$).
- Slow insert/delete in middle ($O(n)$).
- Not synchronized (not thread-safe).
- Prefer ArrayList unless specific need arises.



Elements count = size
Internal array length = capacity
size <= capacity

☆ Remember : Use ArrayList when → fast access by index is needed
Use LinkedList when → frequent insert/delete in middle is needed 😊

Next Page :
HashMap Deep Dive →

Phase 2 - ArrayList Deep Dive (Continue...)

16. Code Snippet - Growth / Resizing

```
import java.util.*;

public class ArrayListGrowth {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>(2);
        // initial capacity = 2
        // Adding elements and printing capacity
        for(int i=1; i<=10; i++) {
            list.add(i);
            System.out.println("After adding " + i +
                " size = " + list.size() +
                ", capacity = " + getCapacity(list));
        }
    }
    // Using reflection to get capacity
    private static int getCapacity(ArrayList<?> list) {
        try {
            java.lang.reflect.Field f = ArrayList.class
                .getDeclaredField("elementData");
            f.setAccessible(true);
            Object[] arr = (Object[]) f.get(list);
            return arr.length;
        } catch (Exception e) {
            return -1;
        }
    }
}
```

Output

After adding 1
size = 1 , capacity = 2

After adding 2
size = 2 , capacity = 2

After adding 3
size = 3 , capacity = 3
($2 \times 1.5 + 1 = 3$)

After adding 4
size = 4 , capacity = 4
($3 \times 1.5 + 1 = 4$)

After adding 5
size = 5 , capacity = 6
($4 \times 1.5 + 1 = 7 \rightarrow$ but next int so 6)

After adding 7
size = 7 , capacity = 9
($6 \times 1.5 + 1 = 10 \rightarrow 9$)

After adding 10
size = 10 , capacity = 13
($9 \times 1.5 + 1 = 14 \rightarrow 13$)

- ★ **Note :**
- Growth factor is 1.5 times.
 - New capacity = $\text{oldCapacity} \times 1.5 + 1$
 - Copying old elements to new array takes $O(n)$.

17. ArrayList Growth Visual (Example)

Initial capacity = 2

Add 1

1	—
---	---

 size=1 capacity=2

Add 2

1	2
---	---

 size=2 capacity=2

Add 3 → resize ($2 \times 1.5 + 1 = 3$)

1	2	3		
---	---	---	--	--

 size=3 capacity=3

Add 4 → resize ($3 \times 1.5 + 1 = 4$)

1	2	3	4	
---	---	---	---	--

 size=4 capacity=4

Add 5 → resize ($4 \times 1.5 + 1 = 7 \rightarrow 6$)

1	2	3	4	5		
---	---	---	---	---	--	--

 size=5 capacity=6

Add 7 → resize ($6 \times 1.5 + 1 = 10 \rightarrow 9$)

1	2	3	4	5	6	7		
---	---	---	---	---	---	---	--	--

 size=7 capacity=9

Add 10 → resize ($9 \times 1.5 + 1 = 14 \rightarrow 13$)

1	2	3	4	5	7	8	9	10	
---	---	---	---	---	---	---	---	----	--

 size=10 capacity=13

18. ArrayList Summary

Topic	Summary
Internal Structure	Uses dynamic array internally.
Default Capacity	10
Grow Factor	$\text{oldCapacity} \times 1.5 + 1$
Add / Get	$O(1)$ amortized / $O(1)$
Remove by Index	$O(n)$ (shifts elements)
Search (contains)	$O(n)$
Insertion in Middle	$O(n)$ (shifts elements)
Best Use	Frequent reads, random access
Not Good For	Frequent insert/delete in middle

19. ArrayList vs LinkedList (Detailed)

	ArrayList	LinkedList
Data Structure	Dynamic Array	Doubly Linked List
Access (get/set)	Fast $O(1)$	Slow $O(n)$ (traverse)
Insertion (end)	Amortized $O(1)$	$O(1)$
Insertion (middle)	$O(n)$	$O(1)$ (if position known)
Deletion (end)	Amortized $O(1)$	$O(1)$
Deletion (middle)	$O(n)$	$O(1)$ (if position known)
Memory Overhead	Less	More (extra pointers)
Cache Performance	Better	Poor
Best Use	Random access, Read heavy	Frequent insert/delete in middle

20. When NOT to Use ArrayList?

- When there are very frequent insertions/deletions in the middle.
- When we don't know approximate size (causes many resizes).
- When memory is extremely limited and objects are small (overhead of unused capacity).

Think :

ArrayList is not always the best choice. Choose the right tool for the job.

21. Mini Practice Questions (ArrayList)

1. What is default capacity of ArrayList?
2. How ArrayList grows internally?
3. What is time complexity of add() at end?
4. What is time complexity of add() in middle?
5. What happens when ArrayList becomes full?
6. Difference between ArrayList and LinkedList.
7. Can ArrayList store duplicate elements?
8. Can ArrayList store null values?
9. Is ArrayList synchronized?
10. Which is better for frequent reads : ArrayList or LinkedList?

★ Quick Revision (Key Points)

- ArrayList uses dynamic array.
- It resizes using $(\text{oldCapacity} \times 1.5) + 1$.
- add() at end is amortized $O(1)$.
- get() is $O(1)$.
- Insert/delete in middle is $O(n)$.
- Best for random access & read heavy operations.

22. Small Real World Example - Storing Products

```
import java.util.*;

class Product {
    int id; String name; double price;
    Product(int id, String name, double price) {
        this.id = id; this.name = name; this.price = price;
    }
    public String toString() {
        return id + " - " + name + " - " + price;
    }
}

public class ProductExample {
    public static void main(String[] args) {
        ArrayList<Product> products = new ArrayList<>();
        products.add(new Product(1, "Laptop", 55000));
        products.add(new Product(2, "Mouse", 500));
        products.add(new Product(3, "Keyboard", 1200));
        // Iterate
        for(Product p : products)
            System.out.println(p);
        // Remove
        products.remove(1); // removes Mouse
        System.out.println("After removing index 1");
        for(Product p : products)
            System.out.println(p);
    }
}
```

Output

1 - Laptop - 55000.0
2 - Mouse - 500.0
3 - Keyboard - 1200.0

After removing index 1
1 - Laptop - 55000.0
3 - Keyboard - 1200.0

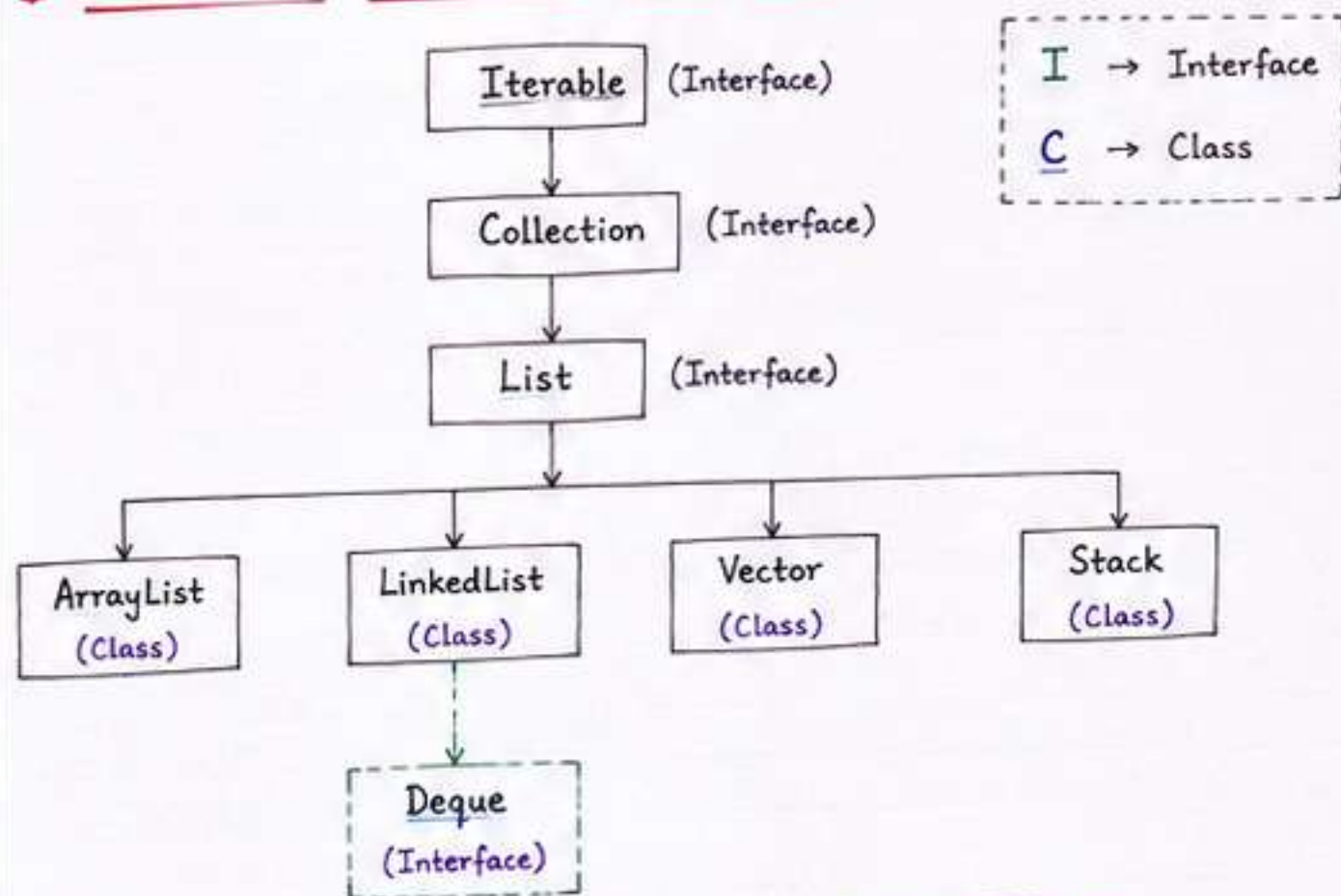
Interview Tip :

Always explain internal working with diagram, complexity & real world use cases.

★ Remember : Understand internally → Practice more → Solve problems → Crack interviews !

JAVA COLLECTIONS FRAMEWORK - LIST (PART-1)

(1) COLLECTION HIERARCHY (List Part)



IMPORTANT RELATIONSHIPS

- ArrayList → implements List
- LinkedList → implements List, Deque
- Vector → implements List
- Stack → extends Vector

List is an Interface present in java.util package. It is an ordered collection (sequence).

(2) ARRAYLIST (Class)

- Implements → List Interface
- Reskeal-it-array implementation of List.
- Elements stored in contiguous memory.
- Index based. Duplicate values allowed.
- Provides random access.

Example:

```
ArrayList<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
list.add(30);
System.out.println(list); // [10, 20, 30]
```

Commonly Used Methods

Method	Description
add(e)	adds element at last
add(index, e)	adds element at specific index
get(index)	returns element at index
set(index, e)	replaces element at index
remove(index)	removes element at index
remove(Object o)	removes first occurrence of object
size()	returns number of elements
isEmpty()	checks if list is empty
contains(o)	checks if element exists

Key Points

- Fast random access.
- Slower insert/delete in middle.
- Uses more memory than LinkedList.

Internal Working

Uses a dynamic array. When size > capacity, a new array (1.5x old capacity) is created and old elements are copied.

Array -

10	20	30
----	----	----

size → 3 capacity → 10

Complexity

add(e) at last → $O(1)$ (amortized)
get(index) → $O(1)$
add(index, e) → $O(n)$
remove(index) → $O(n)$

When to Use?

- When random access is required.
- When frequent traversal is needed.

(3) LINKEDLIST (Class)

- Implements → List, Deque Interface
- Doubly Linked List implementation.
- Elements are not stored in contiguous memory.
- Each element (node) contains → data, previous address, next address.
- Index based. Duplicate values allowed.
- Good for frequent insertions/deletions.

Example:

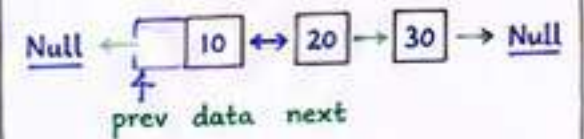
```
LinkedList<String> list = new LinkedList<>();
list.add("A");
list.add("B");
list.addFirst("Start");
list.addLast("End");
System.out.println(list); // [Start, A, B, End]
```

Commonly Used Methods (More than ArrayList)

Method	Description
addFirst(e)	adds element at beginning
addLast(e)	adds element at end
getFirst()	returns first element
getLast()	returns last element
removeFirst()	removes first element
removeLast()	removes last element
add(index, e)	adds element at given index
get(index)	returns element at index

Internal Working

Implemented as Doubly Linked List.



Complexity

addFirst(e) → $O(1)$
addLast(e) → $O(1)$
removeFirst() → $O(1)$
removeLast() → $O(1)$
get(index) → $O(n)$
add/remove at index → $O(n)$

When to Use?

- When many insertions/deletions at beginning/end are required.
- When memory is not a big concern.

(4) VECTOR (Class)

- Implements → List Interface
- Similar to ArrayList but synchronized.
- Thread-safe (methods are synchronized).
- Legacy class (from Java 1.0).
- Less used in modern applications.

Example:

```
Vector<Integer> v = new Vector<>();
v.add(10);
v.add(20);
v.add(30);
System.out.println(v); // [10, 20, 30]
```

Key Points

- Thread-safe (synchronized methods).
- Slower than ArrayList (due to synchronization).
- Grows by 100% of old capacity (default).

Internal Working

Uses dynamic array just like ArrayList. But all methods are synchronized.

Complexity

add(e) at last → $O(1)$ (amortized)
get(index) → $O(1)$
add(index, e) → $O(n)$
remove(index) → $O(n)$

(5) STACK (Class)

- Extends → Vector Class
- Represents LIFO (Last In First Out) data structure.
- Provides push(), pop(), peek(), search() methods.
- Legacy class (Not recommended for new code).
- Use Deque (ArrayDeque/LinkedList) instead.

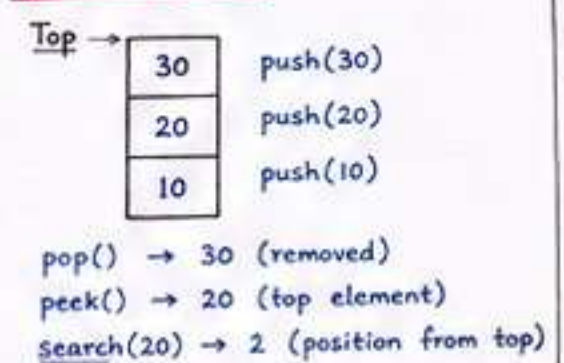
Example:

```
Stack<Integer> s = new Stack<>();
s.push(10);
s.push(20);
s.push(30);
System.out.println(s.pop()); // 30
System.out.println(s.peek()); // 20
```

Key Points

- Stack is a subclass of Vector.
- It is legacy class.
- Use Deque for stack operations.

Operation (LIFO)



Complexity

push(e) → $O(1)$
pop() → $O(1)$
peek() → $O(1)$

JAVA COLLECTIONS FRAMEWORK - LIST (PART-1)

- Sandeep Chaurasiya *

① ARRAYLIST (Class)

- Implements List Interface.
- Resizable-array implementation.
- Elements are stored in contiguous memory.
- Duplicate values allowed.
- Maintains insertion order.
- Provides random access (get/set) in $O(1)$.

Interface or Class?

ArrayList is a Class
It implements
List Interface.

Internal Working:

- Uses a dynamic array internally.
- Has a default capacity of 10.
- When capacity is full, a new array of (old capacity * 1.5) + 1 is created and old elements are copied.

Initial Array (capacity = 10)

0	1	2	3	4	5	6	7	8	9
10	20	30	40						

After adding more elements

0	1	2	3	4	5	6	7	8	9
10	20	30	40	50	60	70	80	90	100

If we add 11th element → new array of size $(10 \times 1.5) + 1 = 16$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
10	20	30	40	50	60	70	80	90	100	110					

Important Methods:

Method	Description
add(E e)	adds element at last
add(int index, E e)	adds element at given index
get(int index)	returns element at index
set(int index, E e)	replaces element at index
remove(int index)	removes element at index
remove(Object o)	removes first occurrence of object
size()	returns number of elements
isEmpty()	checks if list is empty
contains(Object o)	checks if element exists
clear()	removes all elements

Complexity:

add() at last → $O(1)$
(amortized)
add(index, e) → $O(n)$
get(index) → $O(1)$
set(index, e) → $O(1)$
remove(index) → $O(n)$
remove(Object) → $O(n)$

When to Use?

- When we need fast random access.
- When insertions/deletions are less frequent in middle.

Key Points:

- Fast in accessing elements.
- Slower in insertion/deletion at middle.
- Uses more memory when resizing.
- Not synchronized (not thread-safe).

Example:

```
ArrayList<Integer> list = new ArrayList<>();  
list.add(10);  
list.add(20);  
list.add(1, 15); // [10, 15, 20]  
System.out.println(list.get(1)); // 15
```

ARRAYLIST vs LINKEDLIST

Feature	ArrayList	LinkedList
Implementation	Resizable Array	Doubly Linked List
Memory	Less memory per element	More memory per element
Random Access	Yes (get/set $O(1)$)	No (get $O(n)$)
Insertion/Deletion	Slow in middle ($O(n)$)	Fast ($O(1)$ at ends, $O(n)$ in middle)
Performance	Better for read (get/set)	Better for insert/delete
Use Case	When more reads, less writes	When more writes, less reads

② LINKEDLIST (Class)

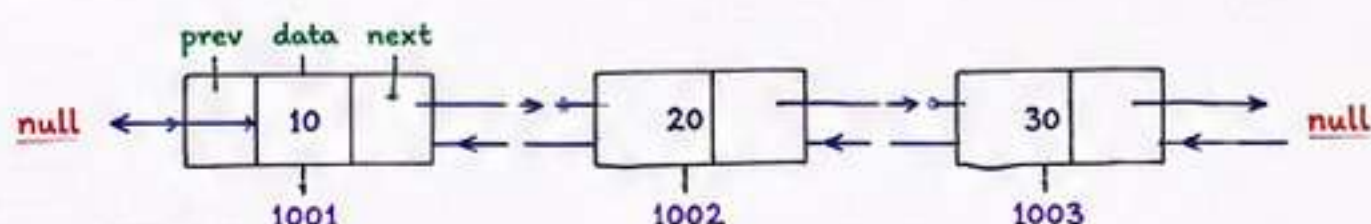
- Implements List and Deque Interfaces.
- Doubly Linked List implementation.
- Elements are not stored in contiguous memory.
- Insertion/Deletion are fast.
- Duplicate values allowed.
- Maintains insertion order.
- Does not provide random access (get is $O(n)$).

Interface or Class?

LinkedList is a Class
It implements
List and Deque
Interfaces.

Internal Working:

- Uses Doubly Linked List.
- Each element is stored in a Node.
- Each Node contains → data, previous & next.



Important Methods:

Method	Description
add(E e)	adds at last
addFirst(E e)	adds at beginning
addLast(E e)	adds at end
get(int index)	returns element at index ($O(n)$)
remove(int index)	removes element at index
removeFirst()	removes first element
removeLast()	removes last element
size()	returns number of elements
contains(Object o)	checks if element exists
clear()	removes all elements

Complexity:

add() at last → $O(1)$
addFirst() → $O(1)$
addLast() → $O(1)$
get(index) → $O(n)$
remove(index) → $O(n)$
removeFirst() → $O(1)$
removeLast() → $O(1)$

When to Use?

- When frequent insertions/deletions are required.
- When we need Deque operations.
- When memory is not a big concern.

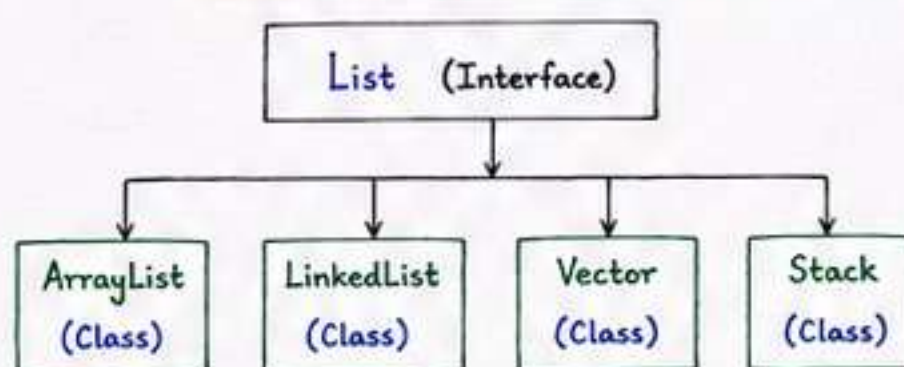
Key Points:

- Fast in insertions/deletions.
- Slow in accessing elements.
- Implements List + Deque.
- More memory per element (stores 2 extra refs).

Example:

```
LinkedList<String> list = new LinkedList<>();  
list.add("A");  
list.addFirst("Start");  
list.addLast("End");  
System.out.println(list);  
// [Start, A, End]
```

QUICK SUMMARY OF LIST CLASSES



I → Interface
C → Class

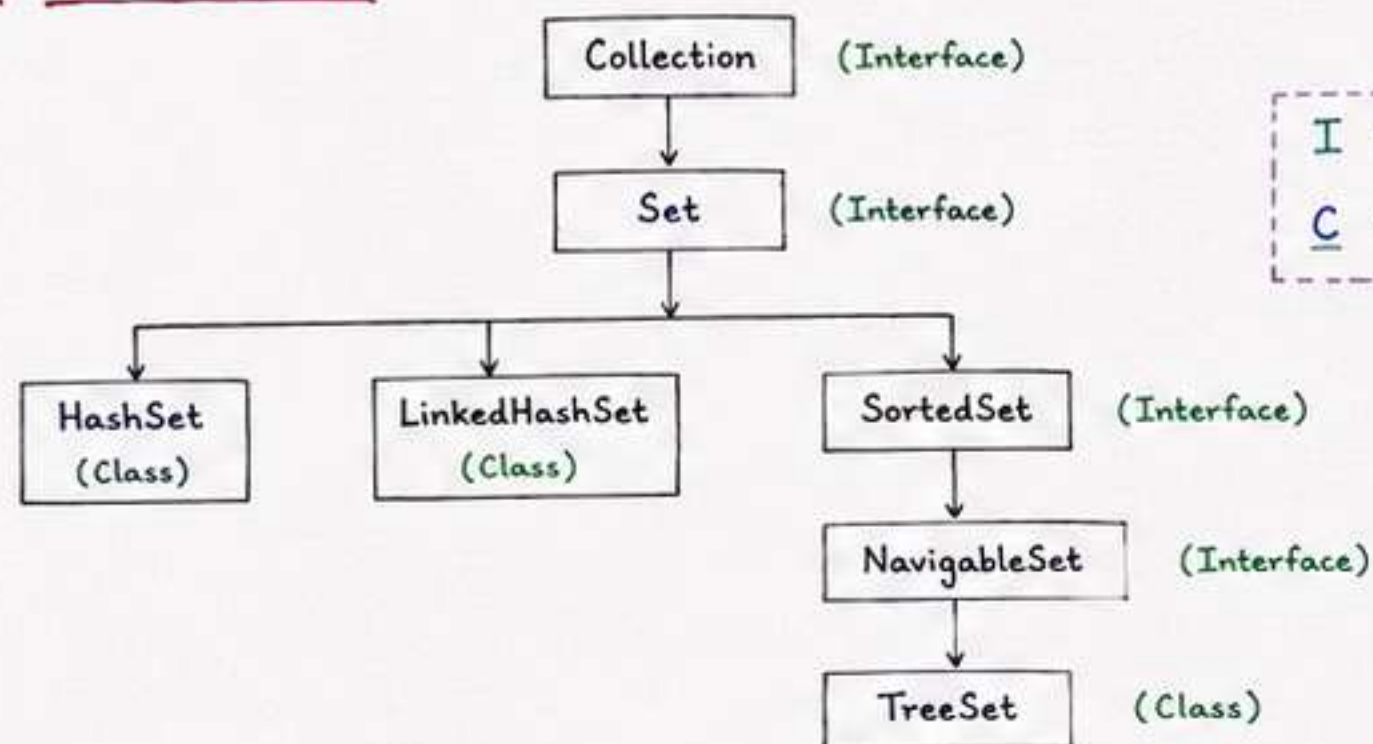
Notes:

- ★ All above classes are in java.util package.
- ★ List maintains insertion order & allows duplicates.

1) SET INTERFACE

- Set is a subinterface of Collection.
- It does not allow duplicate elements.
- At most one null element is allowed.
- It is unordered in nature (except SortedSet/TreeSet).
- Best for search operations.

Set is an Interface.
We cannot create object of Set directly.
We use its implementation classes.

2) SET HIERARCHY

I → Interface
C → Class

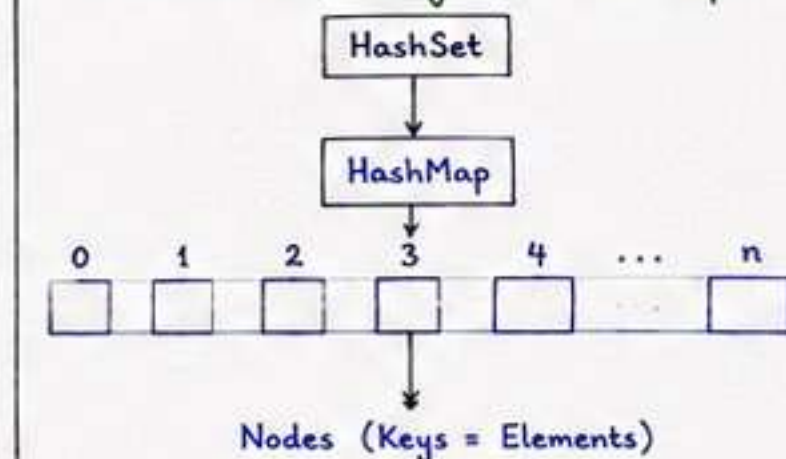
3) HASHSET (CLASS)

- Implements Set interface.
- Does not maintain any order.
- Allows only one null element.
- Uses Hashing internally (HashMap).
- Best performance for add, remove, contains.
- Not synchronized (not thread-safe).

Internal Working :

- Internally, HashSet uses a HashMap.
- Elements are stored as keys in HashMap.
- A dummy object (PRESENT) is used as value.
- Duplicate elements are not allowed because keys in HashMap are unique.

HashSet Internally Uses HashMap

Commonly Used Methods

Method	Description
add(E e)	adds element, returns true if added
remove(Object o)	removes element, returns true if removed
contains(Object o)	returns true if element exists
size()	returns number of elements
isEmpty()	checks if set is empty
iterator()	returns iterator

Complexity

add() → O(1) (avg)
remove() → O(1) (avg)
contains() → O(1) (avg)

Worst Case (many collisions)
→ O(n)

4) LINKEDHASHSET (CLASS)

- Extends HashSet.
- Maintains insertion order (doubly linked list).
- Uses HashMap + LinkedList internally.
- Allows only one null element.
- Slightly slower than HashSet due to order maintenance.

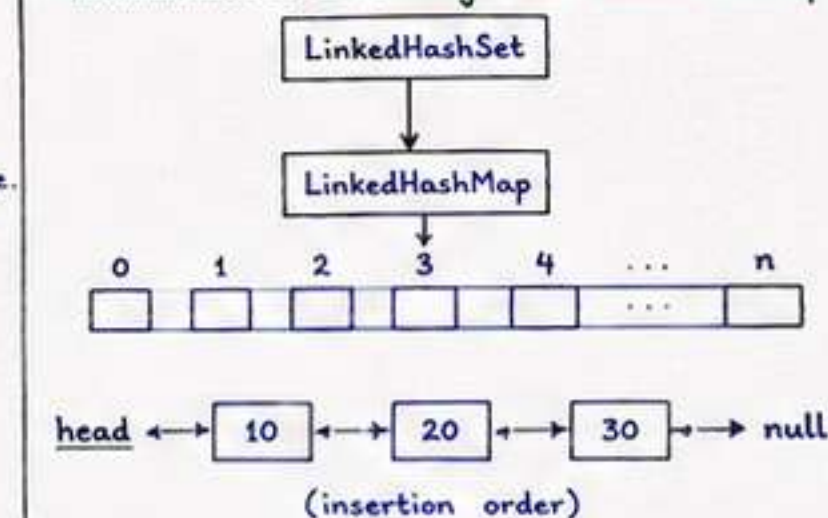
Internal Working :

- LinkedHashSet extends HashSet.
- Internally, it uses LinkedHashMap.
- LinkedHashMap maintains a doubly linked list along with hash table to maintain insertion order.

Commonly Used Methods

Method	Description
add(E e)	adds element, maintains insertion order
remove(Object o)	removes element
contains(Object o)	checks element exists or not
iterator()	returns elements in insertion order

LinkedHashSet Internally Uses LinkedHashMap

Complexity

add() → O(1) (avg)
remove() → O(1) (avg)
contains() → O(1) (avg)
iterator() → O(n)

We cannot create object of SortedSet, we use classes like TreeSet.

5) SORTEDSET (INTERFACE)

- Extends Set interface.
- Maintains elements in sorted (natural) order.
- Does not allow duplicate elements.
- Acts as a parent interface for NavigableSet.

6) NAVIGABLESET (INTERFACE)

- Extends SortedSet interface.
- Provides navigation methods to traverse elements.
- Useful for finding elements like lower, higher, floor, ceiling etc.

7) TREESSET (CLASS)

- Implements NavigableSet interface.
- Elements are stored in sorted (natural) order.
- Uses Red-Black Tree internally.
- Does not allow duplicate elements.
- Not synchronized.

Internal Working :

- TreeSet uses Red-Black Tree (self-balancing BST).
- So operations are log(n) time.

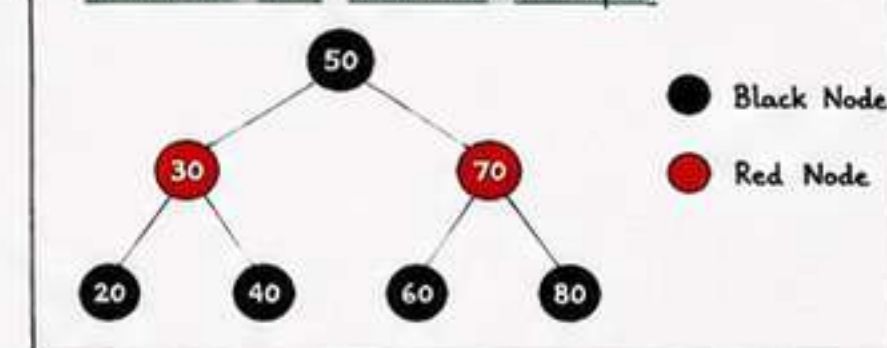
Commonly Used Methods

Method	Description
add(E e)	adds element in sorted order
remove(Object o)	removes element
first()	returns first (lowest) element
last()	returns last (highest) element
headSet(E to)	returns elements less than to
tailSet(E from)	returns elements greater than or equal to from
subSet(E from, E to)	returns elements in given range

Important Methods

lower(E e) → strictly less than e
floor(E e) → less than or equal to e
higher(E e) → strictly greater than e
ceiling(E e) → greater than or equal to e
pollFirst() → removes first element
pollLast() → removes last element

Red-Black Tree Structure (Example)

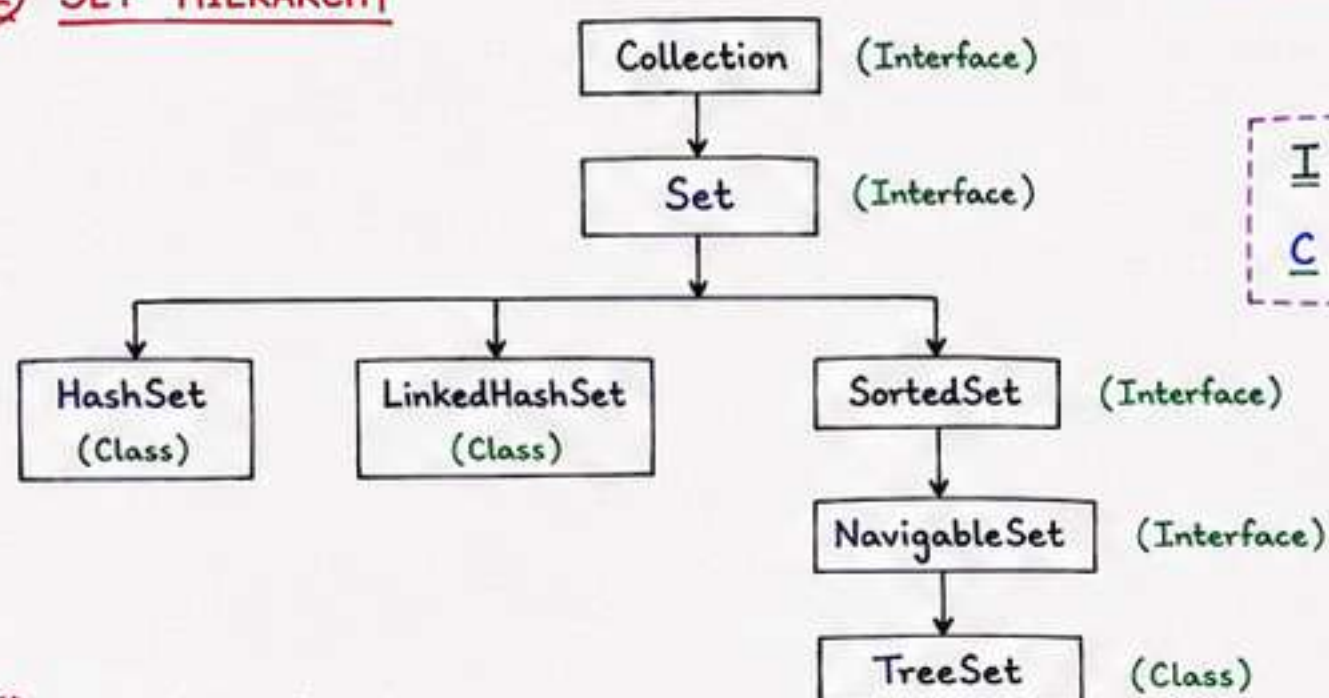
Complexity

add() → O(log n)
remove() → O(log n)
contains() → O(log n)
navigation ops → O(log n)

(1) SET INTERFACE

- Set is a subinterface of Collection.
- It does not allow duplicate elements.
- At most one null element is allowed.
- It is unordered in nature (except SortedSet / TreeSet).
- Best for search operations.

Set is an Interface.
We cannot create object of Set directly.
We use its implementation classes.

(2) SET HIERARCHY

I → Interface
C → Class

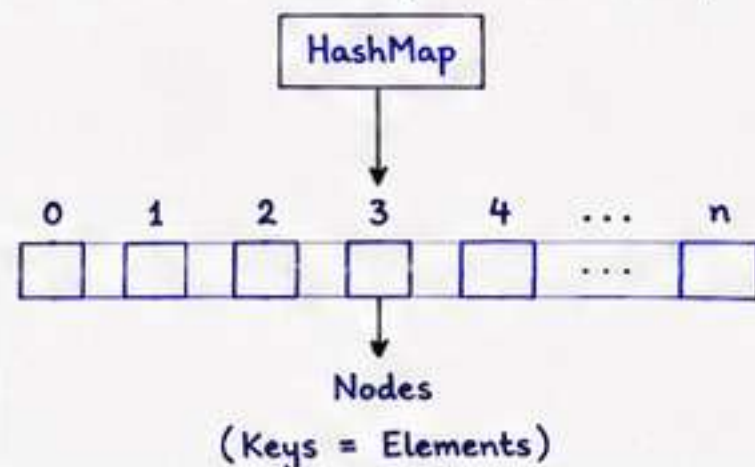
(3) HASHSET (Class)

- Implements Set interface.
- Does not maintain any order.
- Allows only one null element.
- Uses Hashing internally (HashMap).
- Best performance for add, remove, contains.
- Not synchronized (not thread-safe).

Internal Working :

- HashSet uses a HashMap internally.
- Elements are stored as keys in HashMap and a dummy object (PRESENT) is used as value.
- Duplicate elements are not allowed because keys in HashMap are unique.

HashSet Internally Uses HashMap

Commonly Used Methods (HashSet)

Method	Description
add(E e)	adds element, returns true if added
remove(Object o)	removes element, returns true if removed
contains(Object o)	returns true if element exists
size()	returns number of elements
isEmpty()	checks if set is empty
iterator()	returns iterator

When to Use?

- When we need fast operations.
- When order is not important.
- When we need to store unique elements.

Key Points

- Uses hashing.
- Best performance.
- Unordered.
- Allows one null.

- Sandeep Chaurasiya

(4) LINKEDHASHSET (Class)

- Extends HashSet.
- Maintains insertion order (doubly linked list).
- Uses HashMap + LinkedList internally.
- Allows only one null element.
- Slightly slower than HashSet due to order maintenance.

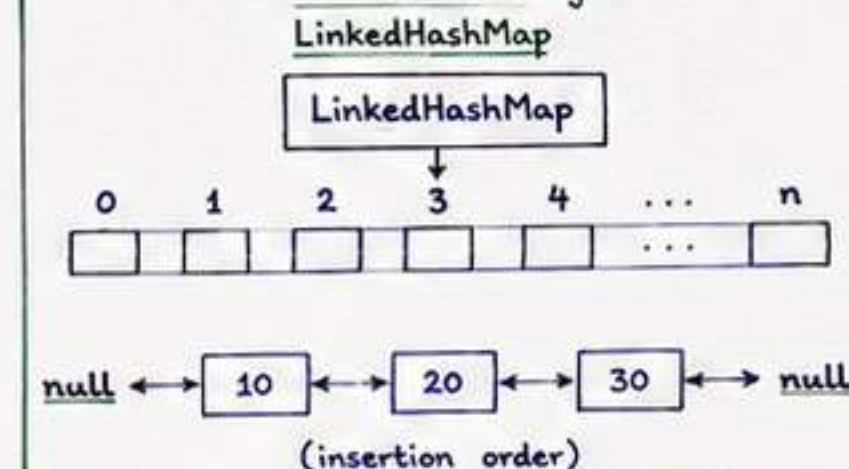
Internal Working :

- LinkedHashSet uses a HashMap internally.
- Each entry is linked in a doubly linked list.
- Iteration is done in the order of insertion.

Commonly Used Methods (LinkedHashSet)

Method	Description
add(E e)	adds element, maintains insertion order
remove(Object o)	removes element
contains(Object o)	checks if element exists
iterator()	returns elements in insertion order

LinkedHashSet Internally Uses

Complexity (LinkedHashSet)

add() → O(1) (avg)
remove() → O(1) avg
contains() → O(1) (avg)
iterator() → O(n)

(5) SORTEDSET (Interface)

- Extends Set interface.
- Maintains elements in sorted (natural) order.
- Does not allow duplicate elements.
- Acts as a parent interface for NavigableSet.
- We cannot create object of SortedSet, we use classes like TreeSet.

Important Methods (SortedSet)

first() → returns first (lowest) element
last() → returns last (highest) element
headSet(E to) → elements < to
tailSet(E from) → elements >= from
subSet(E from, E to) → elements in range

(6) NAVIGABLESET (Interface)

- Extends SortedSet interface.
- Provides navigation methods.
- Useful for finding elements like lower, higher, floor, ceiling etc.

Important Methods (NavigableSet)

lower(E e) → strictly less than e
floor(E e) → less than or equal to e
higher(E e) → strictly greater than e
ceiling(E e) → greater than or equal to e
pollFirst() → removes first element
pollLast() → removes last element

(7) TREESSET (Class)

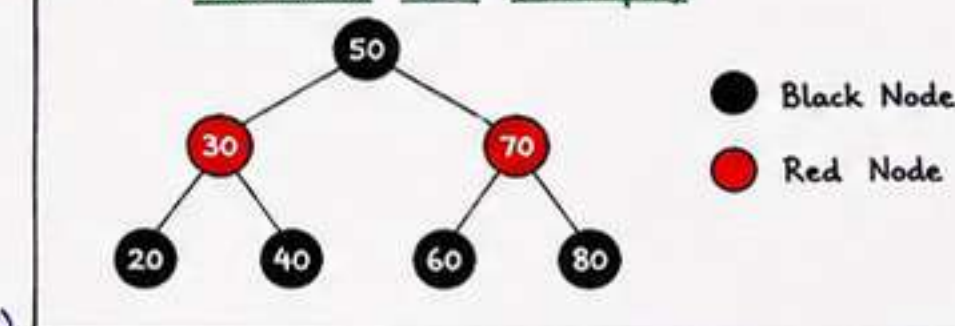
- Implements NavigableSet interface.
- Maintains elements in sorted order.
- Uses Red-Black Tree internally.
- Does not allow duplicate elements.
- Not synchronized.

Internal Working :

- TreeSet uses Red-Black Tree (self-balancing BST).
- So operations are O(log n) time.

Commonly Used Methods (TreeSet)

Method	Description
add(E e)	adds element in sorted order
remove(Object o)	removes element
contains(Object o)	checks if element exists
first()	returns first (lowest) element
last()	returns last (highest) element
iterator()	returns elements in sorted order

Red-Black Tree (Example)Complexity (TreeSet)

add() → O(log n)
remove() → O(log n)
contains() → O(log n)
iterator() → O(n)

- Sandeep Chaurasiya

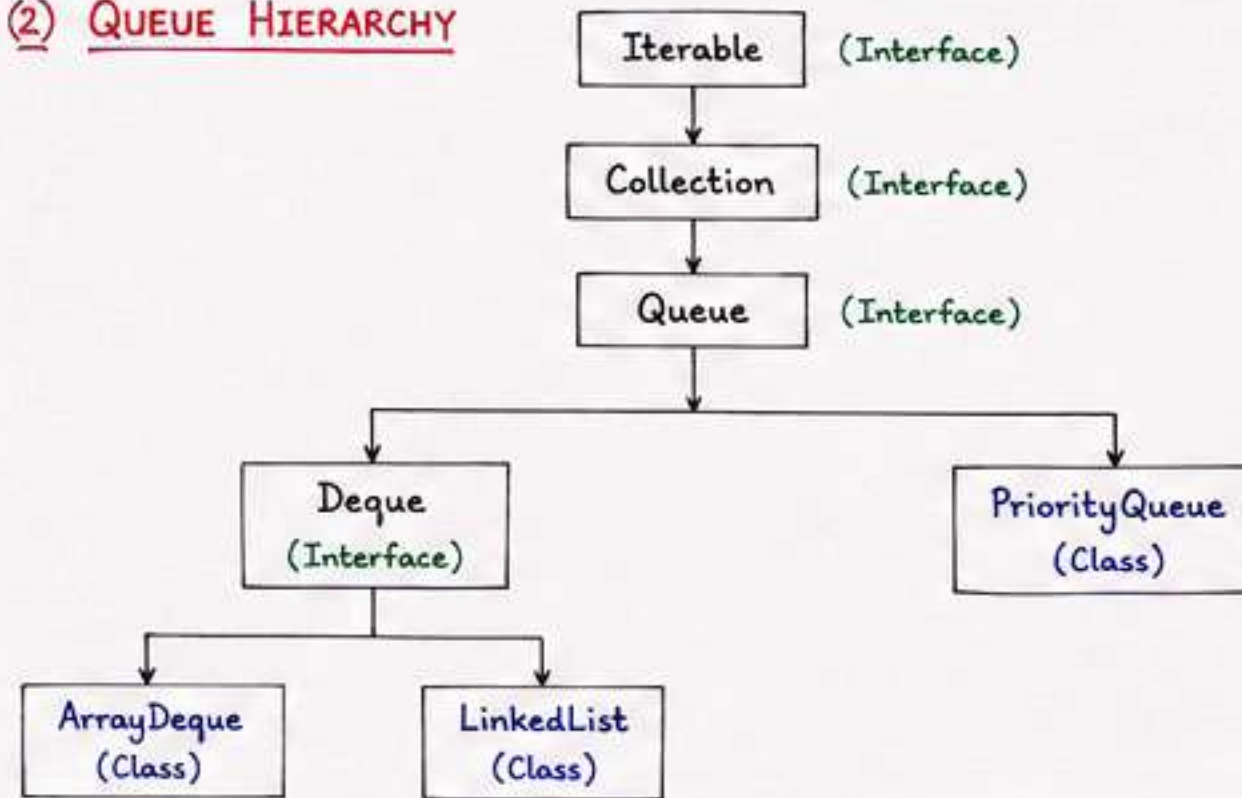
JAVA COLLECTIONS FRAMEWORK - QUEUE (PART-1)

(1) QUEUE INTERFACE

- Queue is a part of Collection Framework.
- It follows FIFO (First In First Out) order.
- Used when elements are processed in the order they arrive.
- Basic operations: add/offer, remove/poll, element/peek.

Queue is an Interface.
It provides the basic operations for FIFO data structures.

(2) QUEUE HIERARCHY



I → Interface
C → Class

(3) DEQUE INTERFACE

- Deque stands for "Double Ended Queue".
- Extends Queue Interface.
- Allows insertion & deletion from both front and rear.
- Can be used as Queue or Stack.

(4) IMPLEMENTING CLASSES OF DEQUE

- ArrayDeque (Resizable array implementation)
- LinkedList (Doubly Linked List implementation)
(LinkedList also implements List Interface)

Important Methods

addFirst(e)	→ inserts at front
addLast(e)	→ inserts at rear
removeFirst()	→ removes from front
removeLast()	→ removes from rear
peekFirst()	→ retrieves front element
peekLast()	→ retrieves last element

Complexity

- add/remove at both ends → $O(1)$
- peek at both ends → $O(1)$

Example

```

ArrayDeque<Integer> dq = new ArrayDeque<>();
dq.addLast(10); dq.addLast(20); dq.addFirst(5);
dq.addLast(30); // Deque: [5, 10, 20, 30]
    
```

B) LINKEDLIST (as Deque)

- LinkedList is a Doubly Linked List.
- It implements Deque Interface.
- Allows insertion & deletion at both ends in $O(1)$.
- Can store null elements.
- More memory overhead than ArrayDeque.

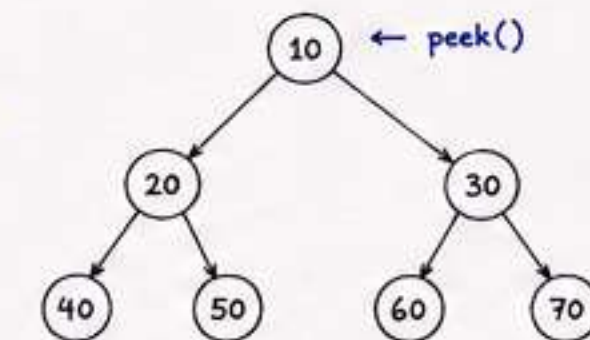
Complexity

- add/remove at both ends → $O(1)$
- peek at both ends → $O(1)$
- get(index) → $O(n)$

(5) PRIORITYQUEUE (Class)

- PriorityQueue is not a Queue in strict FIFO sense.
- Elements are ordered according to their priority.
- By default, it is a Min-Heap (smallest element has highest priority).
- Does not allow null elements.
- Not synchronized.
- Natural ordering or Comparator can be used.

Internal Working (Min-Heap)



Smallest element (10) is always at the root.

Important Methods

Method	Description
offer(e)	inserts element (returns true)
poll()	retrieves & removes head
peek()	retrieves head (does not remove)
size()	returns number of elements

Complexity

- offer(e) → $O(\log n)$
- poll() → $O(\log n)$
- peek() → $O(1)$

Example

```

PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(30); pq.offer(10);
pq.offer(20);
System.out.println(pq.peek());
// 10
System.out.println(pq.poll());
// 10 (removed)
    
```

(6) COMPARISON SUMMARY

Class	Implements	Order	Insertion	Deletion	Null Allowed	Internal Structure	Use When
ArrayDeque	Deque	No order (FIFO/LIFO)	$O(1)$ both ends	$O(1)$ both ends	No	Resizable Array	Fast operations at both ends
LinkedList	Deque List	No order (FIFO/LIFO)	$O(1)$ both ends	$O(1)$ both ends	Yes	Doubly Linked List	Frequent insertions/deletions at both ends
PriorityQueue	Queue	Priority order (Min-Heap)	$O(\log n)$	$O(\log n)$	No	Heap	When order is based on priority, not arrival

Key Points

- Queue interface follows FIFO.
- Deque allows operations at both ends.
- ArrayDeque is faster & preferred over LinkedList as Deque.
- PriorityQueue orders elements by priority (Min-Heap by default).

Interview Tip

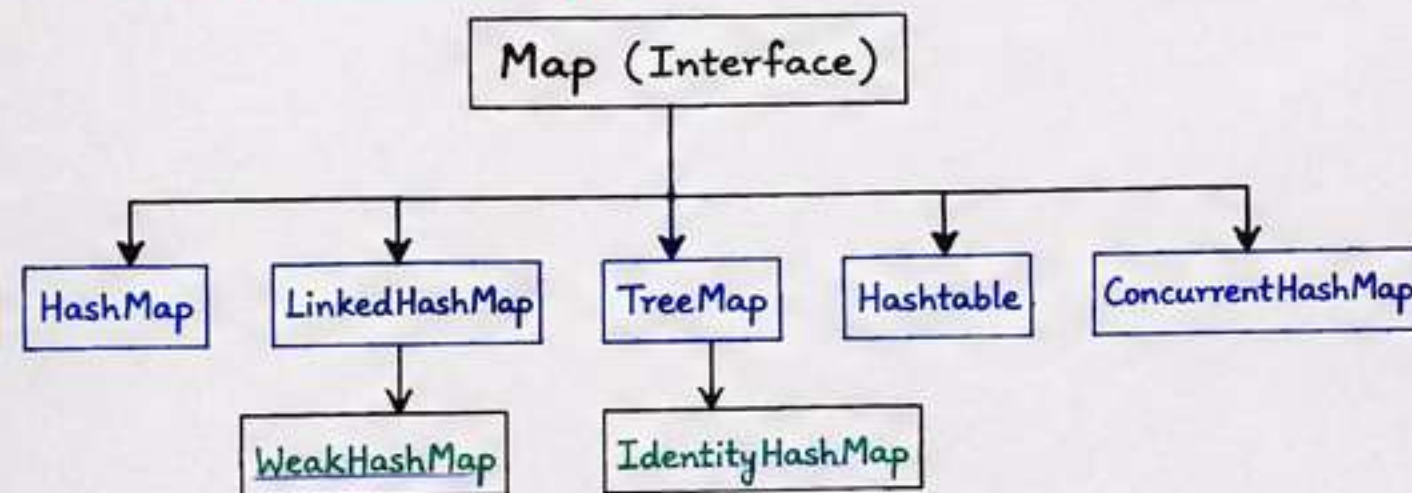
Know the difference between Queue, Deque and PriorityQueue clearly with examples.

* What is Map?

Map is an interface of java.util package. It represents a collection of Key-value pairs. Each key is unique and maps to exactly one value.

- Map does **NOT** extend Collection interface.
- Map is a separate hierarchy.
- Key is unique, but Value can be duplicate.
- One key can map to only one value.
- Null key is allowed by some implementations (HashMap, LinkedHashMap), but not by others (TreeMap, Hashtable).

* Map Hierarchy



* Important Points :

1. Map is not a sub-interface of Collection.
2. Map contains key-value pairs.
3. Key must be unique.
4. Value can be duplicate.
5. Map is widely used in real-world applications like caching, lookups, storing user-session, configuration data etc.

- Sandeep Chaurasiya

* Common Map Implementations

Class	Underlying	Order	Null Key	Null Value	Thread	Best For
HashMap	Hash Table	Not Maintained	1 Null Key Allowed	Multiple Allowed	<u>Not Synchronized</u>	General Purpose (Fast)
LinkedHashMap	Hash Table + Doubly Linked List	<u>Insertion Order</u>	1 Null Key Allowed	Multiple Allowed	<u>Not Synchronized</u>	When Order is Important
TreeMap	Red-Black Tree	Sorted (Natural/Custom)	Null Key Not Allowed	Multiple Allowed	<u>Not Synchronized</u>	Sorted Data, Range Queries
Hashtable	Hash Table	Not Maintained	Null Key Not Allowed	Null Value Not Allowed	Synchronized (Thread Safe)	Legacy, Thread-Safe
ConcurrentHashMap	Segmented Hash Table	Not Maintained	Null Key Not Allowed	Null Value Not Allowed	Thread Safe (High Concurrency)	High Concurrency Applications

* Key Methods of Map Interface

1. V put(K key, V value)
→ Inserts the key-value pair.
2. V get(Object key)
→ Returns value for the given key.
3. V remove(Object key)
→ Removes the key and its value.
4. boolean containsKey(Object key)
→ Checks if key exists.
5. boolean containsValue(Object value)
→ Checks if value exists.
6. int size()
→ Returns number of key-value pairs.
7. boolean isEmpty()
→ Checks if map is empty.
8. void clear()
→ Removes all key-value pairs.
9. Set<K> keySet()
→ Returns all keys.
10. Collection<V> values()
→ Returns all values.
11. Set<Map.Entry<K,V>> entrySet()
→ Returns set of key-value mappings.

* Example : (HashMap)

```

import java.util.*;

public class MapExample {
    public static void main(String[] args) {
        Map<Integer, String> map = new HashMap<>();
        map.put(1, "Sandeep");
        map.put(2, "Chaurasiya");
        map.put(3, "Java Developer");
        System.out.println("Map : " + map);           // {1=Sandeep, 2=Chaurasiya, 3=Java Developer}
        System.out.println("Value for key 2 : " + map.get(2)); // Chaurasiya
        System.out.println("Contains key 3 ? " + map.containsKey(3)); // true
        System.out.println("All Keys : " + map.keySet()); // [1, 2, 3]
        System.out.println("All Values : " + map.values()); // [Sandeep, Chaurasiya, Java Developer]
    }
}
    
```

Output :
 Map : {1=Sandeep, 2=Chaurasiya, 3=Java Developer}
 Value for key 2 : Chaurasiya
 Contains key 3 ? true
 All Keys : [1, 2, 3]
 All Values : [Sandeep, Chaurasiya, Java Developer]

MAP IN JAVA - DETAILED NOTES

By: Sandeep Chaurasiya

* Why Map is Important?

- Stores data in Key-Value pairs.
- Key is unique, but multiple keys can have same value.
- Very fast search, insert, delete based on key.
- Widely used in real-world applications like caching, indexing, user sessions, configuration, counting frequency, group by operations etc.

* Map Interface Declaration

```
public interface Map<K, V>
```

K → Type of Key

V → Type of Value

* Key Characteristics in Map

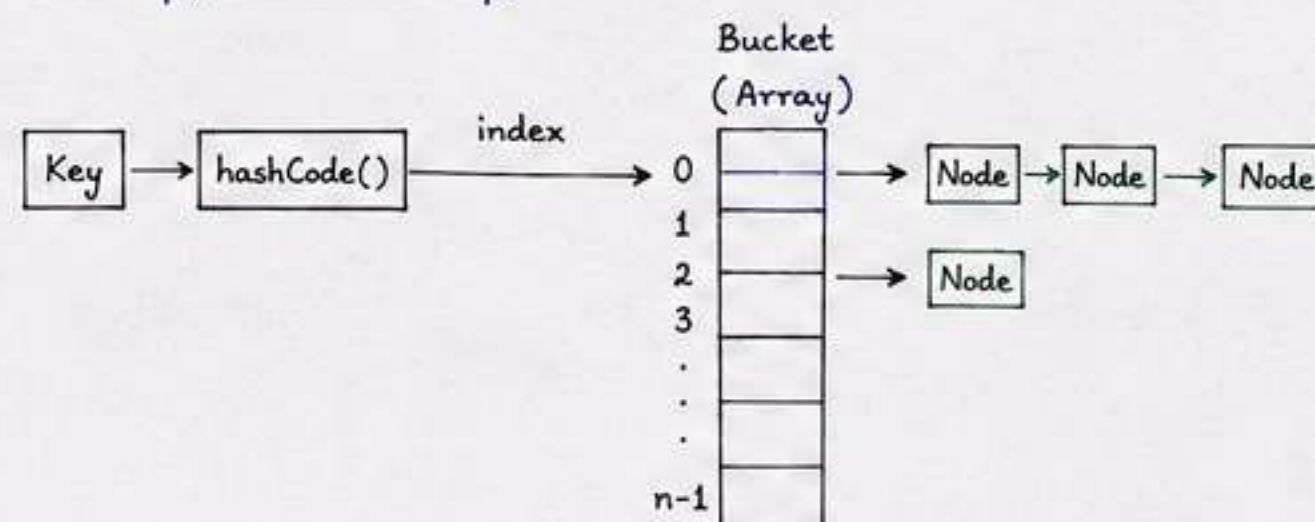
1. Keys must be UNIQUE.
2. At most one null key is allowed (in HashMap, LinkedHashMap, TreeMap).
3. Key is used to perform operations like put, get, remove, containsKey etc.
4. If two keys are equal (according to equals()), they are treated as same key.
5. Internally key is used to calculate hashCode().

* Value Characteristics in Map

1. Values can be duplicate.
2. Any number of null values are allowed.
3. Value does not participate in uniqueness.
4. Value is stored and retrieved using its key.

* How Map Works Internally (General Idea)

Map works internally using Hashing (in most implementations like HashMap, LinkedHashMap).



- Key is converted to hashCode().
- hashCode() is converted to index using (n-1) & hashCode.
- Index points to a bucket.
- If bucket is empty, node is inserted.
- If bucket already has nodes, nodes are linked (Collision Handling).
- Node contains (Key, Value, Hash, Next).

* Map.Entry<K, V>

Map.Entry is a nested interface inside Map. It represents a key-value pair.

Methods:

1. K getKey() → Returns key.
2. V getValue() → Returns value.
3. V setValue(V value) → Updates value and returns old value.
4. boolean equals(Object o) → Compares two entries.
5. int hashCode() → Returns hash code.
6. String toString() → Returns key=value.

* Example : Using Map.Entry

```
Map<Integer, String> map = new HashMap<>();
map.put(1, "One");
map.put(2, "Two");
map.put(3, "Three");

for (Map.Entry<Integer, String> e : map.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}
```

Output :

```
1 -> One
2 -> Two
3 -> Three
```

* Commonly Used Map Implementations

1. HashMap → Unordered, Fast, Allows null key & null value.
2. LinkedHashMap → Maintains insertion order.
3. TreeMap → Sorted by keys (Natural order or Comparator).
4. Hashtable → Legacy class, Synchronized, No null key/value.
5. ConcurrentHashMap → Thread-safe, High concurrency.

* When to Use Which Map?

Map	Use When	Reason
HashMap	General purpose use	Fastest for most operations.
LinkedHashMap	When insertion order must be preserved	Maintains order using doubly linked list.
TreeMap	When keys must be sorted	Keys are stored in Red-Black Tree.
Hashtable	Legacy systems	Synchronized, Not recommended.
ConcurrentHashMap	Multi-threaded environment	High concurrency, Better performance.

* Internal Nodes in HashMap (Java 8+)

Each bucket stores nodes of type Node<K,V>.

Node structure:

```
static class Node<K,V> {
    final int hash;
    final K key;
    V value;
    Node<K,V> next;
}
```

In case of high collisions, Linked List is converted to Red-Black Tree (TreeNode).

* Important Points

1. Map is not a child of Collection. It is a separate hierarchy.
2. Key is unique, Value can be duplicate.
3. HashMap allows 1 null key and multiple null values.
4. TreeMap does not allow null key.
5. HashMap is not synchronized (Not thread-safe).
6. ConcurrentHashMap is the best choice for multi-threading.

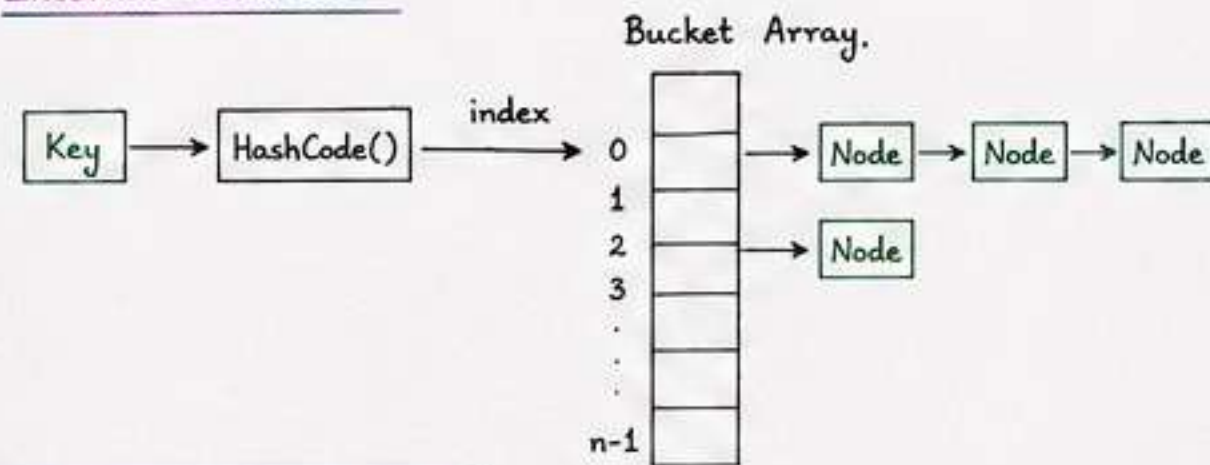
MAP IMPLEMENTATIONS - DEEP DIVE

By : Sandeep Chaurasiya

* HashMap (Most Important)

- Based on Hash Table.
- Allows null key and multiple null values.
- Not synchronized. (Not thread-safe)
- Very fast for most operations.

Internal Structure :



Node Structure (Java 8+)

```
int hash
K key
V value
Node<K,V> next
```

If more than 8 nodes in a bucket, Linked List is converted to Red-Black Tree. (TreeBin)

Important Methods :

- `put(K,V)` → Inserts or updates.
- `get(Object key)` → Returns value.
- `remove(Object key)` → Removes key-value pair.
- `containsKey(Object key)` → Checks key exists.
- `containsValue(Object value)` → Checks value exists.
- `keySet()` → Returns all keys.
- `values()` → Returns all values.
- `entrySet()` → Returns set of key-value pairs.
- `size()`, `isEmpty()`, `clear()` → Basic operations.

Time Complexity (Average)

```
put() : O(1)
get() : O(1)
remove() : O(1)
containsKey() : O(1)
Worst Case (Before Java 8) : O(n)
Worst Case (Java 8 with Tree) : O(log n)
```

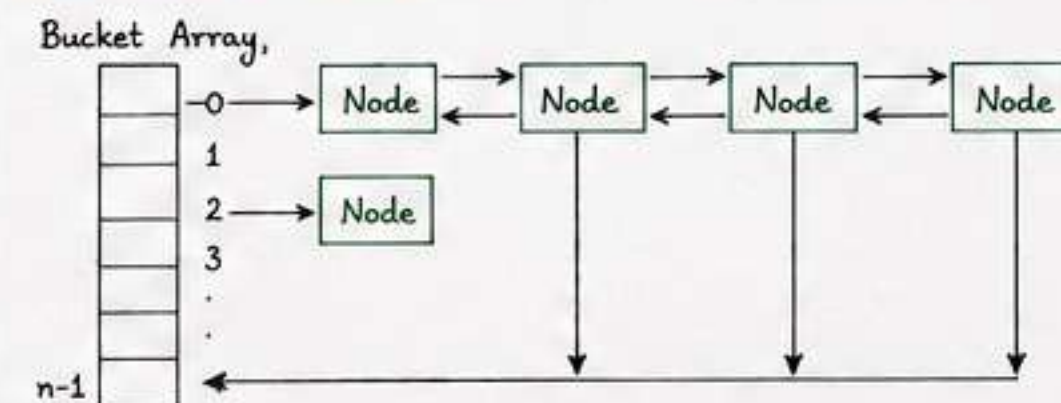
When to use :

- When fast insertion, deletion and search are required.
- General purpose Map.
- Most used Map.

* 2. LinkedHashMap (Important)

- Maintains insertion order.
- Based on Hash Table + Doubly Linked List.
- Allows null key and multiple null values.
- Not synchronized.

Internal Structure :



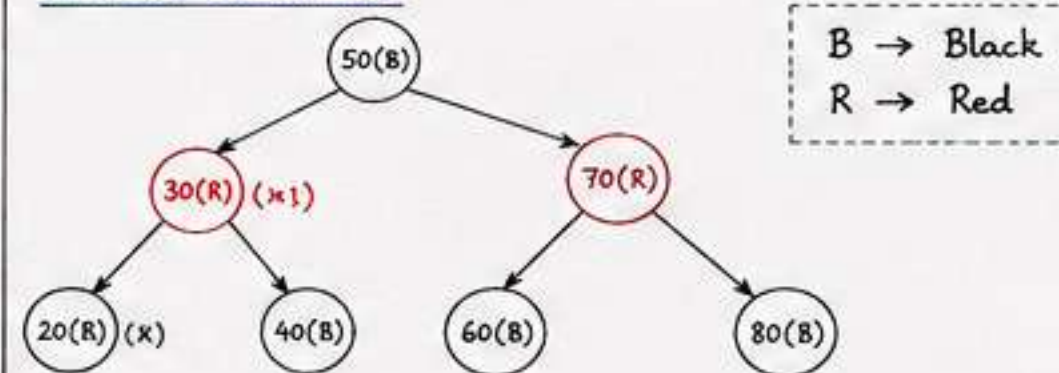
When to use :

- When we need order of insertion.
- LRU Cache implementation.
- Maintaining order is important.

3. TreeMap (Important)

- Based on Red-Black Tree.
- Keys are sorted in natural order (Ascending).
- Does not allow null key (NullPointerException).
- Allows multiple null values.
- Slower than HashMap but provides sorted data.

Internal Structure :



Important Methods :

```
put(), get(), remove(), containsKey(), containsValue(),
firstKey(), lastKey(), headMap(), tailMap(),
subMap(), descendingMap(), navigableKeySet() etc.
```

When to use :

- When data must be sorted by keys.
- Range queries (for example, from key1 to key2).
- Leader/Floor/Ceiling key operations.

4. Hashtable (Awareness)

- Legacy class.
- Synchronized.
- Does not allow null key or null value.
- Slower than HashMap & ConcurrentHashMap.
- Use only in old legacy code.

5. ConcurrentHashMap (Very Important for Projects)

- Thread-safe and high performance.
- Does not allow null key or null value.
- Uses Segmented Locking / CAS (Java 8+).
- Best choice for multi-threaded applications.

When to use :

- When multiple threads access Map concurrently.
- High concurrency applications.
- Real-world projects (Caching, Sessions, etc.)

MAP IMPLEMENTATIONS COMPARISON

Map	Null Key	Null Value	Order	Thread-Safe	Sorted	Best For
HashMap	1 Allowed	Multiple Allowed	No Order	No	No	General Purpose Fast Access
LinkedHashMap	1 Allowed	Multiple Allowed	Insertion Order	No	No	When order is required
TreeMap	Not Allowed	Multiple Allowed	Sorted Order	No	Yes	Sorted Data Range Queries
Hashtable	Not Allowed	Not Allowed	No Order	Yes	No	Legacy (Old Code)
ConcurrentHashMap	Not Allowed	Not Allowed	No Order	Yes	No	High Concurrency

Map.Entry<K,V> Interface

Map.Entry represents a key-value pair.

Methods :

```
getKey() → Returns key.
getValue() → Returns value.
setValue(V value) → Updates value.
```

Example :

```
Map<Integer, String> map = new HashMap<>();
map.put(1, "One");
map.put(2, "Two");
map.put(3, "Three");

for (Map.Entry<Integer, String> e : map.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}
```

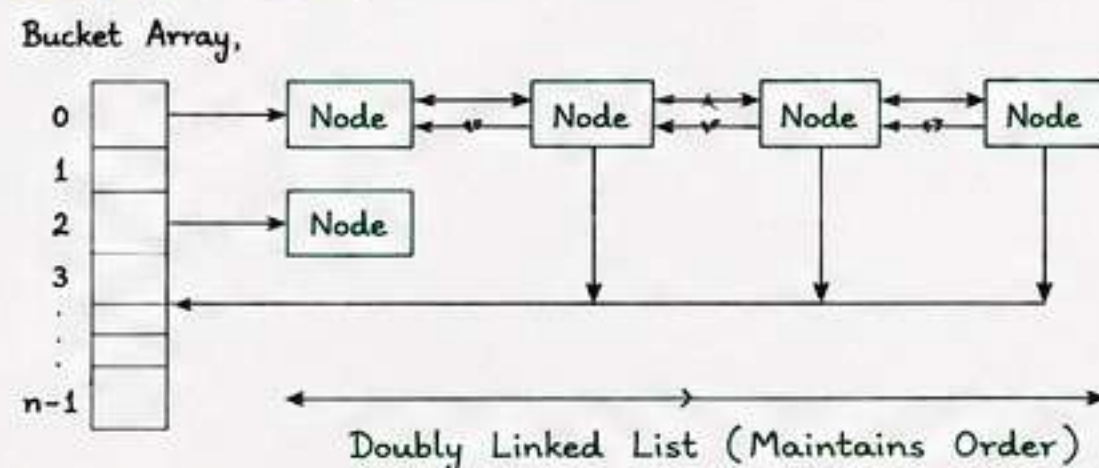
Output :

```
1 → One
2 → Two
3 → Three
```

6. LinkedHashMap (Medium-Deep)

- Combination of Hash Table + Doubly Linked List.
- Maintains insertion order (or access order if required).
- Allows null key and multiple null values.
- Not synchronized.

Internal Structure :



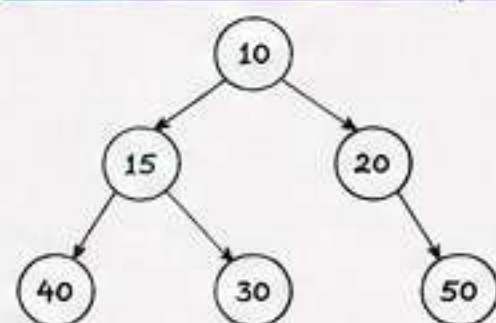
When to use :

- When we need insertion order.
- Implementing LRU Cache (remove eldest entry).
- Order of keys is important.

7. PriorityQueue (Medium-Deep)

- Based on Heap (Min-Heap by default).
- Does not allow null.
- Not synchronized.
- Elements are ordered according to natural order or by Comparator.
- Head is the smallest element (Min-Heap).

Internal Structure (Heap) :



Array Representation
[10, 15, 20, 40, 30, 50]

Parent at i
Left child = $2i + 1$
Right child = $2i + 2$

Important Methods :

- offer() → Inserts.
- poll() → Removes head (smallest element).
- peek() → Returns head.
- size() → Returns size.

When to use :

- Scheduling
- Task scheduling
- Dijkstra's Algorithm
- Sorting top K elements

8. Collections Utility Methods (Medium-Deep)

Very Important Class : `java.util.Collections`

Frequently Used Methods :

- sort(List) → Sorts list.
- reverse(List) → Reverses list.
- shuffle(List) → Shuffles list randomly.
- swap(List, i, j) → Swaps elements.
- rotate(List, k) → Rotates list by k positions.
- binarySearch(List, key) → Binary search (sorted list).
- max(Collection) → Maximum element.
- min(Collection) → Minimum element.
- frequency(Collection, obj) → Returns frequency of element.
- disjoint(Collection1, C2) → Returns true if no element common.

9. Arrays Utility Methods (Medium-Deep)

Class : `java.util.Arrays`

Frequently Used Methods :

- sort(array) → Sorts array.
- binarySearch(array, key) → Binary search.
- fill(array, value) → Fills array with value.
- copyOf(array, newLength) → Copies array.
- copyOfRange(array, f, t) → Copies range.
- equals(array1, array2) → Compares arrays.
- toString(array) → Returns string representation.

10. Map vs Collection

Collection	Map
Part of <code>java.util</code> hierarchy	Not part of Collection hierarchy
Stores single values	Stores key-value pairs
Inherits from Collection interface	Does NOT inherit from Collection
Allows duplicate values (depends on implementation)	Keys must be unique
Iterator is used	Iterator / EntrySet is used
Examples : List, Set, Queue	Examples : Map, HashMap, TreeMap

11. When NOT to Use Map ?

- When we only need to store single values.
- When key is not required.
- When order is not important and simple list is enough.

12. Map Interview Questions (Important)

1. Why Map does not extend Collection ?
2. Difference between HashMap and Hashtable.
3. What happens if we put null key in HashMap ?
4. How HashMap works internally ?
5. What is the difference between equals() and hashCode() ?
6. Why is hashCode() important in HashMap ?
7. How does ConcurrentHashMap achieve thread-safety ?
8. Difference between TreeMap and HashMap.
9. What is the default initial capacity of HashMap ?
10. What is the load factor in HashMap ?
11. How LinkedHashMap maintains order ?
12. How to implement LRU Cache using LinkedHashMap ?
13. What is the time complexity of get() in HashMap ?
14. What is collision handling in HashMap ?

13. Real Project Usage (Spring Boot)

- Storing configuration key-value pairs.
- Caching data (User sessions, Tokens, OTPs etc.).
- Storing entity data in Map before database operations.
- Grouping data (e.g. `Map<String, List<Order>>`).
- Counting frequencies (`Map<String, Integer>`).
- Building lookups for fast access.
- Implementing LRU Cache.

Summary (Choose the Right Map)

Use This Map	When to Use
HashMap	General purpose, fastest.
LinkedHashMap	When order is important / LRU Cache.
TreeMap	When sorted keys are required.
Hashtable	Legacy, synchronized.
ConcurrentHashMap	High concurrency applications.

MAP IMPLEMENTATIONS - CONCLUDED

By : Sandeep Chaurasiya

14. Important Points (Map)

1. Map does NOT extend Collection.
2. Key is unique, Value can be duplicate.
3. At most one null key (in HashMap, LinkedHashMap, TreeMap) but multiple null values allowed.
4. HashMap is not synchronized.
5. TreeMap keeps keys sorted.
6. Hashtable is legacy class (use ConcurrentHashMap).
7. ConcurrentHashMap is best for multi-threading.
8. equals() and hashCode() are very important in Map.

15. equals() and hashCode() in Map (Very Important)

Used in : HashMap, LinkedHashMap, HashSet (Same Logic)

1. Two keys are considered same if both equals() and hashCode() are same.
2. If hashCode() is same but equals() is false, they will go to same bucket but treated as different.
3. Always override both while using custom objects as keys.

Example :

```
class Student {
    int id;
    String name;
    Student(int id, String name) {
        this.id = id; this.name = name;
    }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Student s = (Student) o;
        return this.id == s.id && this.name.equals(s.name);
    }
    @Override
    public int hashCode() {
        return Objects.hash(id, name);
    }
}
```

Note :

- Equal objects MUST have same hashCode.
- Same hashCode objects MAY or MAY NOT be equal.
- Good hashCode() improves performance.

16. Time Complexity Summary

Operation	HashMap	LinkedHashMap	TreeMap	Hashtable	ConcurrentHashMap
put()	O(1)	O(1)	O(log n)	O(1)	O(1)
get()	O(1)	O(1)	O(log n)	O(1)	O(1)
remove()	O(1)	O(1)	O(log n)	O(1)	O(1)
containsKey()	O(1)	O(1)	O(log n)	O(1)	O(1)
size()	O(1)	O(1)	O(1)	O(1)	O(1)
iteration	O(n)	O(n)	O(n)	O(n)	O(n)

17. Real Interview Tricky Questions

1. What if we use Mutable key in HashMap? (Avoid)
2. What is Load Factor and why 0.75 is default in HashMap?
3. How does HashMap handle collision internally?
4. Difference between fail-fast and fail-safe iterators.
5. Why ConcurrentHashMap does not allow null key/value?
6. How LRU Cache can be implemented using LinkedHashMap?
7. How TreeMap maintains sorted order internally?
8. What happens if hashCode() is bad implementation?
9. Why equals() must be consistent with hashCode()?

18. Interview One-Liner Answers

HashMap	→ Best for general purpose (fast).
LinkedHashMap	→ Best when order matters (insertion / access order).
TreeMap	→ Best when keys must be sorted.
Hashtable	→ Legacy class, synchronized (use ConcurrentHashMap).
ConcurrentHashMap	→ Best for multi-threading.
HashSet	→ Uses HashMap internally.
TreeSet	→ Uses TreeMap internally.
PriorityQueue	→ Based on Heap (Min-Heap by default).
ArrayDeque	→ Best for stack / queue operations.
LinkedList	→ Use when frequent insert/delete in middle.
ArrayList	→ Use when frequent read & random access.

Golden Rule for Choosing Map

- Need order + fast access → LinkedHashMap
- Need sorted keys → TreeMap
- Need multi-threading → ConcurrentHashMap
- Need general fast operations → HashMap
- Legacy system → Hashtable (Avoid)

19. Mini Project Ideas Using Map

- * Student Management System (Map< id, Student >)
- * User Session Management (Map< userId, Session >)
- * Caching System (Map< key, value >)
- * Count Frequency of Words (Map< word, Integer >)
- * LRU Cache Implementation (LinkedHashMap + removeEldestEntry)
- * Leaderboard System (Map< User, Score >)
- * Inventory Management (Map< productId, Product >)
- * Authentication System (Map< username, User >)

20. Map Cheat Sheet (Quick Revision)

Map Declaration

```
Map<KeyType, ValueType> map = new HashMap<>();
```

Add / Update

```
map.put(key, value);
```

Get Value

```
map.get(key);
```

Check Key

```
map.containsKey(key);
```

Check Value

```
map.containsValue(value);
```

Remove

```
map.remove(key);
```

Iterate (Key-Value)

```
for (Map.Entry<KeyType, ValueType> e : map.entrySet()) {
    System.out.println(e.getKey() + " = " + e.getValue());
}
```

Get All Keys

```
map.keySet();
```

Get All Values

```
map.values();
```

Clear Map

```
map.clear();
```

Size

```
map.size();
```

* Master these concepts with practice, problems and real projects. You will be interview ready and project ready. All the Best! - Sandeep Chaurasiya

1. What is Map?

Map is a part of Java Collections Framework which stores data in key-value pairs. It maps unique keys to corresponding values.

`Map<K, V>` → Key-Value pair

Example :

```
Map<Integer, String> map = new HashMap<>();
map.put(101, "Sandeep");
map.put(102, "Rahul");
map.put(103, "Amit");
```

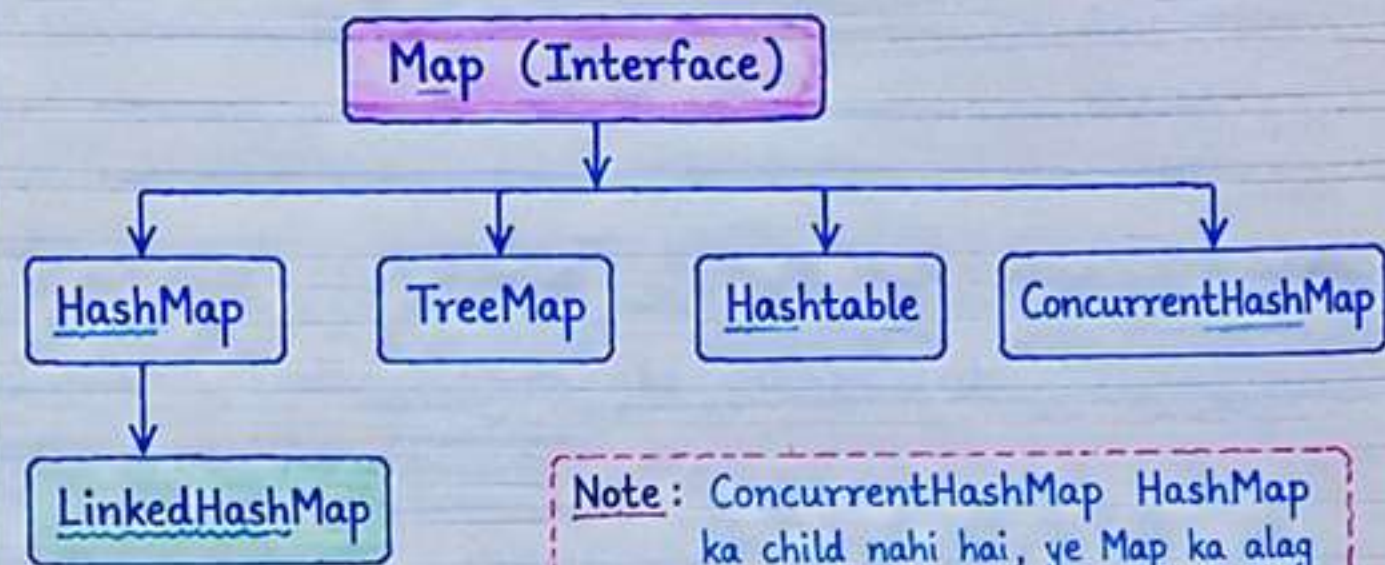
Output :

```
101 → Sandeep
102 → Rahul
103 → Amit
```

Important Points :

- Key is unique (duplicate key not allowed).
- Same key par old value replace ho jaati hai.
- Values duplicate ho sakti hain.
- Map directly Collection interface ko implement nahi karta.
- Main purpose → key ke through value access karna.

2. Map Hierarchy



Note : ConcurrentHashMap HashMap ka child nahi hai, ye Map ka alag implementation hai.

Important Implementations :

Implementation	Main Feature
HashMap	Fast, no guaranteed order
LinkedHashMap	Insertion / access order
TreeMap	Sorted keys
Hashtable	Legacy, synchronized
ConcurrentHashMap	Thread-safe, concurrent

3. Map vs Collection

Collection

- Single values store karta hai.
- Extends Collection interface.
- Example: List, Set, Queue

```
List<String> list = new ArrayList<>();
list.add("Sandeep");
list.add("Rahul");
```

Output :

```
Sandeep
Rahul
```

Map

- Key + Value store karta hai.
- Does not extend Collection.
- Example: HashMap, TreeMap

```
Map<Integer, String> map = new HashMap<>();
map.put(101, "Sandeep");
map.put(102, "Rahul");
```

Output :

```
101 → Sandeep
102 → Rahul
```

VS

Interview Line : "Collection stores individual elements, whereas Map stores data in key-value pairs."

4. Important Map Methods

<code>put(K, V)</code>	Key-value pair add karta hai.
<code>get(K)</code>	Value return karta hai.
<code>remove(K)</code>	Key remove karta hai.
<code>containsKey(K)</code>	Key exist karta hai.
<code>containsValue(V)</code>	Value exist karta hai.
<code>size()</code>	Total number of entries.
<code>isEmpty()</code>	Check if map is empty.
<code>clear()</code>	Remove all entries.
<code>keySet()</code>	All keys return karta hai.
<code>values()</code>	All values return karta hai.
<code>entrySet()</code>	All key-value pairs return karta hai.

Example :

```
Map<Integer, String> map = new HashMap<>();
map.put(101, "Sandeep");
map.put(102, "Rahul");
System.out.println(map.get(101));
System.out.println(map.containsKey(102));
System.out.println(map.size());
```

Output :

```
Sandeep
true
2
```

INTERVIEW QUESTIONS (Page 1)

Q1. What is Map in Java?

Ans: Map is an interface in Java Collections Framework that stores data in key-value pairs. Keys are unique, values can be duplicated.

Q2. Is Map a child of Collection?

Ans: No. Map does not extend the Collection interface. It is a separate hierarchy.

Q3. Can Map have duplicate keys?

Ans: No. Duplicate key is not allowed. If same key is put again, old value gets replaced.

Q4. Can Map have duplicate values?

Ans: Yes. Multiple keys can have the same value.

Q5. What is the difference between Key and Value?

Ans: Key → unique identifier, Value → actual data.
Key duplicate nahi ho sakti, Value duplicate ho sakti hai.

Q6. Name some Map implementations.

Ans: HashMap, LinkedHashMap, TreeMap, Hashtable, ConcurrentHashMap.

Key Points to Remember :

- ★ Map = Key + Value
- ★ Key → Unique
- ★ Value → Duplicate allowed
- ★ Map ≠ Collection
- ★ Same key → old value replace

— Sandeep Chaurasiya

2. Map Implementations

1. HashMap

- HashMap is the most commonly used Map implementation.

```
Map<Integer, String> map = new HashMap<>();
map.put(101, "Sandeep");
map.put(102, "Rahul");
map.put(103, "Amit");
System.out.println(map);
```

Important Points :

- Key-value pair store karta hai.
- No guaranteed order.
- One null key allowed.
- Multiple null values allowed.
- Duplicate key allowed nahi.
- Same key par old value replace hoti hai.
- Fast $O(1)$ for get/put/remove - average).

2. LinkedHashMap

LinkedHashMap = HashMap + Order

```
Map<Integer, String> map = new LinkedHashMap<>();
map.put(103, "Amit");
map.put(101, "Sandeep");
map.put(102, "Rahul");
System.out.println(map);
```

Output :

{103=Amit, 101=Sandeep, 102=Rahul}

Important Points :

- Insertion order maintain karta hai.
- HashMap jaisa hi, but order ke saath.
- Access-order mode bhi available hai.
- LRU Cache ke liye useful.

HashMap → No guaranteed order

LinkedHashMap → Insertion order

3. Hashtable

Hashtable ek legacy Map implementation hai.

```
Map<Integer, String> map = new Hashtable<>();
map.put(101, "Sandeep");
map.put(102, "Rahul");
```

Important Points :

- Thread-safe / synchronized.
- Null key allowed nahi.
- Null value allowed nahi.
- Legacy class hai.
- Modern applications me generally ConcurrentHashMap preferred hai.

```
map.put(null, "Sandeep"); // ✗
map.put(101, null); // ✗
```

4. ConcurrentHashMap

ConcurrentHashMap multithreaded applications ke liye use hota hai.

```
Map<Integer, String> map = new ConcurrentHashMap<>();
map.put(101, "Sandeep");
map.put(102, "Rahul");
```

Important Points :

- Thread-safe.
- Multiple threads safely access kar sakte hain.
- Null key allowed nahi.
- Null value allowed nahi.
- Hashtable se better scalability provide karta hai.

```
map.put(null, "Sandeep"); // ✗
```

```
map.put(101, null); // ✗
```

Quick Comparison

Feature	HashMap	LinkedHashMap	Hashtable	ConcurrentHashMap
Order	No guarantee	Insertion order	No guarantee	No guarantee
Thread-safe	No	No	Yes	Yes
Null Key	1 allowed	1 allowed	No	No
Null Values	Yes	Yes	No	No
Legacy	No	No	Yes	No
Main Use	General purpose	Ordered Map	Legacy code	Multithreading

★ Interview Priority

HashMap → ★★★ (Must Know)
 LinkedHashMap → ★★
 ConcurrentHashMap → ★★★
 Hashtable → ★★ (Legacy concept)

Keep Learning
 Keep Growing
 - Sandeep

Interview Questions (Page 2)

Q1. HashMap aur LinkedHashMap mein difference?

Ans: HashMap order guarantee nahi karta, whereas LinkedHashMap insertion order maintain karta hai.

Q2. HashMap mein null key allowed hai?

Ans: Yes, one null key aur multiple null values.

Q3. Hashtable mein null allowed hai?

Ans: No, null key aur null value dono nahi.

Q4. Hashtable vs ConcurrentHashMap?

Ans: Dono thread-safe hain, but ConcurrentHashMap modern applications ke liye better hai.

Q5. HashMap thread-safe hai?

Ans: No.

Q6. ConcurrentHashMap mein null kyun nahi?

Ans: Concurrency ke context mein null ko handle karna complex hota hai, isliye allowed nahi.

3. TreeMap

TreeMap stores data in sorted order of keys.

```
TreeMap<Integer, String> map = new TreeMap<>();
map.put(103, "Amit");
map.put(101, "Sandeep");
map.put(102, "Rahul");
System.out.println(map);
```

Output:

{101=Sandeep, 102=Rahul, 103=Amit}

Important Methods:

- firstKey() → smallest key
- lastKey() → largest key
- higherKey(key) → strictly greater key
- lowerKey(key) → strictly smaller key
- ceilingKey(key) → equal or greater
- floorKey(key) → equal or smaller

Important Points:

- Keys are sorted (natural ordering).
- Duplicate key not allowed.
- Duplicate values allowed.
- Null key not allowed (in natural order).
- Null values allowed.
- Not thread-safe.

TreeMap Example

```
TreeMap<Integer, String> map = new TreeMap<>();
map.put(103, "Amit");
map.put(101, "Sandeep");
map.put(102, "Rahul");

System.out.println("firstKey: " + map.firstKey());
System.out.println("lastKey: " + map.lastKey());
System.out.println("higherKey(101): " + map.higherKey(101));
System.out.println("lowerKey(102): " + map.lowerKey(102));
System.out.println("ceilingKey(102): " + map.ceilingKey(102));
System.out.println("floorKey(102): " + map.floorKey(102));
```

Interview line:

"TreeMap maintains sorted order of keys using a Red-Black Tree."

4. Hashtable

Hashtable is a legacy class and thread-safe (synchronized).

```
Hashtable<Integer, String> table = new Hashtable<>();
table.put(101, "Sandeep");
table.put(102, "Rahul");
System.out.println(table);
```

```
table.put(null, "Java"); // NullPointerException
table.put(101, null); // NullPointerException
```

Important Points:

- Thread-safe / synchronized.
- Null key not allowed.
- Null value not allowed.
- Legacy class.
- Order not guaranteed.
- Same key → old value replace.
- Duplicate values allowed.

5. ConcurrentHashMap

ConcurrentHashMap is for multithreaded applications.

```
ConcurrentHashMap<Integer, String> map = new ConcurrentHashMap<>();
map.put(101, "Sandeep");
map.put(102, "Rahul");
System.out.println(map);
```

Important Points:

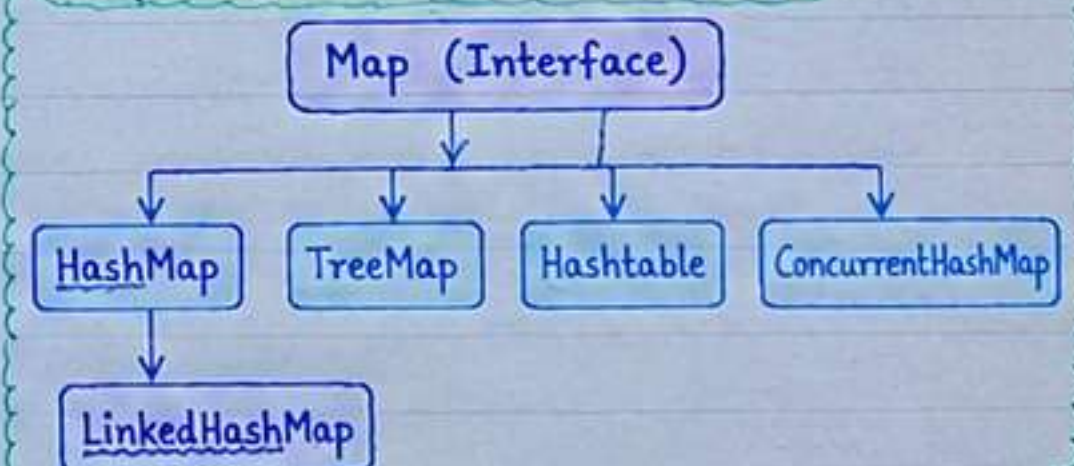
- Thread-safe.
- Null key not allowed.
- Null value not allowed.
- Better scalability than Hashtable.
- Modern and preferred in concurrent applications.

```
map.put(null, "Java"); // ✗
map.put(101, null); // ✗
```

Quick Comparison of Map Implementations

Feature	HashMap	LinkedHashMap	TreeMap	Hashtable	ConcurrentHashMap
Order	No guarantee	Insertion order	Sorted keys	No guarantee	No guarantee
Thread-safe	No	No	No	Yes	Yes
Null Key	1 allowed	1 allowed	No	No	No
Null Value	Yes	Yes	Yes	No	No
Legacy	No	No	Yes	Yes	No
Main Use	General purpose	Ordered Map	Sorted Map	Legacy code	Multithreading

Map Implementations Hierarchy



Interview Questions (Page 3)

- Q1. Which Map implementation maintains insertion order?
Ans: LinkedHashMap.
- Q2. Which Map implementation keeps keys sorted?
Ans: TreeMap.
- Q3. Is HashMap thread-safe?
Ans: No.
- Q4. Can Hashtable store null key or value?
Ans: No.
- Q5. What is the difference between Hashtable and ConcurrentHashMap?
Ans: Both are thread-safe, but ConcurrentHashMap is modern, provides better scalability and does not allow null key/value.

- Q6. Can TreeMap have null key?
Ans: No (in natural ordering).
- Q7. Can ConcurrentHashMap have null key/value?
Ans: No.
- Q8. What is the main feature of LinkedHashMap?
Ans: Maintains insertion order (or access order).
- Q9. Which Map is preferred for LRU cache?
Ans: LinkedHashMap (access-order).
- Q10. What happens when you put a duplicate key in Map?
Ans: Old value is replaced by new value.

Key Takeaways (Page 3)

- ✓ TreeMap → Sorted keys
- ✓ LinkedHashMap → Insertion order (or access order)
- ✓ Hashtable → Legacy + synchronized + no null
- ✓ ConcurrentHashMap → Modern + thread-safe + no null

Remember:

Map ≠ Collection

5. TreeMap

TreeMap stores data in sorted order of keys.

```
TreeMap<Integer, String> map = new TreeMap<>();
map.put(103, "Amit");
map.put(101, "Sandeep");
map.put(102, "Rahul");
System.out.println(map);
```

Output :

```
{101 = Sandeep,
102 = Rahul,
103 = Amit}
```

Important Points :

- Keys are sorted (natural ordering / Comparator).
- Duplicate key not allowed.
- Null key not allowed (in natural ordering).
- Null values allowed.
- Not thread-safe.

Special Methods :

- firstKey()
- lastKey()
- higherKey()
- lowerKey()
- ceilingKey()
- floorKey()

8. Interview Questions (Page 4)

Q1. What is the difference between HashMap and LinkedHashMap?

Ans: HashMap does not maintain any order, whereas LinkedHashMap maintains insertion order.

Q2. Can HashMap have null key?

Ans: Yes, HashMap allows one null key.

Q3. Can Hashtable have null key/value?

Ans: No, Hashtable does not allow null key or null value.

Q4. What is the difference between Hashtable and ConcurrentHashMap?

Ans: Both are thread-safe, but ConcurrentHashMap is modern and provides better scalability and performance.

Q5. Is HashMap thread-safe?

Ans: No, HashMap is not thread-safe.

Q6. What is the use of TreeMap?

Ans: TreeMap is used when keys need to be sorted.

Q7. What is the difference between floorKey() and lowerKey()?

Ans: floorKey() returns key $\leq$ given key, whereas lowerKey() returns key $<$ given key.

Q8. Why ConcurrentHashMap does not allow null?

Ans: To avoid ambiguity in concurrent environment (null can be used to represent absence of value).

Q9. Which implementation is best for LRU Cache?

Ans: LinkedHashMap (access-order mode).

Q10. What are the advantages of ConcurrentHashMap over Hashtable?

Ans: Better scalability, higher performance, modern design, supports concurrent access.

6. Map.Entry & Iteration**Map.Entry :**

Map.Entry is a nested interface of Map, which represents a single key-value pair.

```
for (Map.Entry<Integer, String> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
```

Iteration Methods :

- keySet() → gives all keys
- values() → gives all values
- entrySet() → gives key-value pairs (best for iteration)

Example :

```
for (Integer key : map.keySet())
    System.out.println(key);
for (String value : map.values())
    System.out.println(value);
```

7. HashMap Internal Working (Brief)

- Uses array + linked list / tree (since Java 8).
- Hash function is used to find the bucket.
- If collision occurs → linked list (after 8 nodes → tree).
- Time complexity :
 - get() / put() → $O(1)$ (average)
 - $O(n)$ (worst case)

Key Points :

- Initial capacity : 16
- Load factor : 0.75
- Rehashing : when $\text{size} > \text{capacity} * \text{loadFactor}$

9. Important Programs (Snippet)**1. Remove Duplicate from Map**

```
Map<Integer, String> map = new HashMap<>();
map.put(101, "Sandeep");
map.put(102, "Sandeep");
map.put(103, "Rahul");
System.out.println(map);
(Duplicate value allowed)
```

2. Check Key Exists

```
if (map.containsKey(101)) {
    System.out.println("Key Exist");
} else {
    System.out.println("Key Not Exist");
}
```

10. Real Time Use Cases

- HashMap → Caching, lookups, general purpose.
- LinkedHashMap → LRU Cache, recent access tracking.
- TreeMap → Range search, sorted data.
- Hashtable → Legacy systems, synchronized access.
- ConcurrentHashMap → Multithreading, high performance systems.

Quick Revision

- ✓ Map → Key + Value
- ✓ Key → Unique
- ✓ Value → Duplicate allowed
- ✓ Map $\neq$ Collection
- ✓ Same key → old value replace
- ✓ Order → depends on implementation
- ✓ Thread-safe → Hashtable, ConcurrentHashMap

"Choose the right Map implementation based on your requirement - speed, order, thread-safety and null handling."

- Sandeep Chaurasiya

Java - Generics

(Part 1)

Sandeep Chaurasiya

Generics = Type safety + Reusability + Clean Code

1. Type Parameters MUST KNOW

- **Hinglish:** Hum $\langle T \rangle$ likh ke ek placeholder banate hain jo baad me kisi bhi specific type se replace ho jata hai.
- **Why:** Code ko flexible aur type-safe banata hai.
- **Project Relevance:** Spring Boot me repository, service, controller me generic types use hote hain.
- **Example:**

```
class Box<T> {  
    private T value;  
    public void setValue(T value) { this.value = value; }  
    public T getValue() { return value; }  
}
```

```
Box<String> box = new Box<>();  
box.setValue("Hello");  
System.out.println(box.getValue()); // Hello
```

- **Output:** Hello

• Interview Qs:

1. Type parameter kya hota hai?
2. T ko hum kuch bhi naam de sakte hain?
3. Generic class aur normal class me farq?

• Common Mistakes:

1. Raw type use karna (`Box box = new Box();`)
2. Wrong type pass kar dena.

• Remember:

- $\langle T \rangle$ ek placeholder hai.
- Type safety deta hai.
- Mostly T, E, K, V use hote hain.

Type Parameters
(T, E, K, V etc.)

Use when:

- Generic class/method banana ho.
- Reusable aur flexible code likhna ho.

Tag:  MUST KNOW

2. Generic Class MUST KNOW

- **Hinglish:** Class ke level par type parameter use karte hain taaki class kisi bhi type ke data ke saath kaam kar sake.
- **Why:** Same class ko alag-alag type ke liye use kar sakte hain.
- **Project Relevance:** Repository, DAO, Response wrapper, Service classes me use hota hai.
- **Example:**

```
class Pair<T, U> {  
    private T first;  
    private U second;  
    // getters & setters  
}
```

```
Pair<String, Integer> p = new Pair<>("Age", 25);  
System.out.println(p.getFirst()); // Age
```

- **Output:** Age

• Interview Qs:

1. Generic class kya hota hai?
2. Kaise multiple type parameters use karte hain?

• Common Mistakes:

1. Generic class ko bina type ke use karna (raw type).
2. Galat type pass karna.

• Remember:

- $\langle T, U \rangle$ multiple types.
- Class level par define hota hai.
- Type safety + reusability.

Generic Class

```
class ClassName<T> { ... }
```

Use when:

- Ek hi class ko multiple types ke liye use karna ho.
- Reusable components banana ho.

Tag:  MUST KNOW

3. Generic Method MUST KNOW

- **Hinglish:** Method level par type parameter use karte hain.
- **Why:** Method ko flexible banata hai, class ke type se independent.
- **Project Relevance:** Utility methods, service methods, helper methods me use hota hai.
- **Example:**

```
public static <T> void printData(T data) {  
    System.out.println(data);  
}
```

```
printData("Hello");  
printData(100);
```

- **Output:** Hello
100

• Interview Qs:

1. Generic method kya hota hai?
2. Class level aur method level generics me farq?

• Common Mistakes:

1. Type parameter ko hardcode karna.
2. Wrong type pass karna.

• Remember:

- $\langle T \rangle$ method ke return type ya parameter me hota hai.
- Class ke type se independent.

Generic Method

```
<T> returnType methodName(T param)
```

Use when:

- Utility methods banana ho.
- Generic class ke bina bhi generic logic chahiye ho.

Tag:  MUST KNOW

4. Generic Interface MUST KNOW

- **Hinglish:** Interface me type parameter use karte hain.
- **Why:** Multiple implementation ko support karta hai.
- **Project Relevance:** Repository, DAO, Service, Comparator, Factory, etc.
- **Example:**

```
interface Repository<T> {  
    void save(T entity);  
    T findById(int iid);  
}
```

```
class UserRepository implements Repository<User> { ... }
```

• Interview Qs:

1. Generic interface kya hota hai?
2. Interface me multiple type parameter kaise use karte hain?

• Common Mistakes:

1. Raw type use karna.
2. Wrong type return karna.

• Remember:

- Interface me $\langle T \rangle$ use hota hai.
- Implementation me specific type mention karte hain.

Generic Interface

```
interface InterfaceName<T> { ... }
```

Use when:

- Contract define karna ho.
- Different type ke saath same behavior dena ho.

Tag:  MUST KNOW

Generic Class = Class jisme type parameter use karke flexible (reusable) code likhte hain.

1. Simple Explanation (Hinglish)

- Generic class ka matlab hota hai ki aap class ko kisi specific type ke saath bind nahi karte, balki `<T>` jaise type parameter use karte ho.
- Isse same class ko alag-alag data types ke saath use kar sakte ho.
- Code reusable, type-safe aur clean banta hai.

2. Why is it used?

- Code reuse ke liye (same class multiple types ke liye).
- Type safety milta hai (runtime error kam hote hain).
- Readability aur maintainability improve hoti hai.

3. Project / Spring Boot Relevance

- Repository pattern me (Spring Data JPA) generic interfaces/classes use hote hain.
- Custom utility classes (like `ResponseWrapper<T>`, `ApiResponse<T>`) banane me kaam aata hai.
- Service layer me reusable components ke liye.

4. Java Example

```
// Generic Class
class Box<T> {
    private T value;

    public void setValue(T value) {
        this.value = value;
    }
    public T getValue() {
        return value;
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Box<String> stringBox = new Box<>();
        stringBox.setValue("Hello");
        System.out.println(stringBox.getValue());

        Box<Integer> intBox = new Box<>();
        intBox.setValue(100);
        System.out.println(intBox.getValue());
    }
}
```

5. Output

```
Hello
100
```

6. Real Project / Spring Boot Example

Suppose hum ek generic Response class banate hain jo kisi bhi type ka data wrap kar sake.

```
class ApiResponse<T> {
    private T data;
    private String message;

    public ApiResponse(T data, String message){
        this.data = data;
        this.message = message;
    }
    // getters and setters
}
```

Use in Spring Boot (example):

```
@RestController
public class UserController {
    @GetMapping("/user")
    public ResponseEntity<ApiResponse<User>>
    getUser() {
        User user = userService.getUser();
        ApiResponse<User> response = new ApiResponse<>
        (user, "User fetched successfully");

        return ResponseEntity.ok(response);
    }
}
```

- ✓ Yahan `ApiResponse<T>` generic class hai, jisme T alag-alag type ke data ke liye use ho sakta hai.

7. Interview Relevance

- Generic class ka concept bahut common hai, especially in Spring Boot, repositories, services, DTOs, response classes.
- Poochte hain: "Generic class ka benefit kya hai?", "Isme type safety kaise milti hai?", "T ka kya matlab hai?"

8. Common Interview Questions

1. Generic class kya hota hai? Example do.
2. T ka full form kya hota hai?
3. Generic class ka benefit kya hai?
4. Spring Boot me kaha-kaha generic class use hoti hai?
5. Agar aap generic class ko bina type ke use karein to kya hoga?

9. Common Mistakes

- ✗ Type parameter ke jagah direct data type use karna.
- ✗ Raw type use karna.
(`Box box = new Box();`).
- ✗ Different types ke data ko same generic class me store karne ki koshish.
- ✗ Type safety rules ko ignore karna.

10. What to Remember

- ✓ `<T>` type parameter hota hai.
- ✓ Class ko reusable aur type-safe banata hai.
- ✓ Generic class me method bhi generic ho sakte hain (next topic).
- ✓ Spring Boot me Response, Repository, Custom Utility classes me use hota hai.



MUST KNOW

Quick Summary

- Generic class = reusable + type-safe.
- `<T>` use hota hai.
- Spring Boot me response, repository etc. me commonly use hota hai.
- Raw type mat use karo.

Part 1 - Generic Class : Interview + Project Revision Sheet

Key Points

- ✓ `<T>` se generic class banta hai.
- ✓ Reusable aur type-safe code.
- ✓ Spring Boot me Response, Repository, Utility classes me use hota hai.
- ✓ Raw type mat use karo.

Important Interview Questions

1. Generic class kya hota hai?
2. T ka full form kya hai?
3. Benefit kya hai?
4. Spring Boot me kaha use hota hai?
5. Without type use karne par kya hoga?

Project Use Cases

- `ApiResponse<T>` (custom response)
- Generic Repository (Spring Data JPA)
- Utility / Helper classes
- Service layer components

Next: 3. Generic Method? →

Java - Generics (Part 3)

Sandeep Chaurasiya

Generic Interface, Bounded Types, Wildcards, ? extends, ? super, PECS

4. Generic Interface 🔥 MUST KNOW

- Hinglish:** Interface bhi generic ho sakti hai. Isme type parameter use karte hain.
- Why:** Reusability + flexibility.
- Project Relevance:** Repository / Service layer me generic interface use hoti hai (e.g. Spring Data JPA).
- Example:**

```
Interface Repository<T> {  
    T save(T entity);  
    T findById(Long id);  
}
```

- Output:** (Interface hai, output nahi)
- Interview Qs:** Generic interface ka fayda? T ka kya hota hai? Spring Data JPA me use?
- Common Mistakes:** T galat jagah use karna.
- What to Remember:** Interface me <T> likho aur generic method define karo.

Generic Interface (Syntax)

```
Interface InterfaceName<T> {  
    returnType methodName(T param);  
}
```

Use when:

- Reusable interface chahiye.
- Different types ke liye same contract chahiye.

Tag: 🔥 MUST KNOW

5. Bounded Types 🔥 MUST KNOW

- Hinglish:** Type parameter ko limit karna (jaise Number, Comparable, etc.).
- Why:** Type safety + specific functionality use karna.
- Project Relevance:** Sorting, comparison, etc. me use hota hai.
- Example:**

```
class MaxFinder<T extends Comparable<T>> {  
    T getMax(T a, T b) {  
        return a.compareTo(b) > 0 ? a : b;  
    }  
}
```

- Output:** Maximum value (e.g. 20).
- Interview Qs:** Bounded type ka use case? extends vs super?
- Common Mistakes:** Sirf class ka naam likhna (T extends Object).
- What to Remember:** extends use karo jab compare karna ho ya specific class ke methods chahiye.

Bounded Types (Syntax)

<T extends ClassName>

Example:

```
<T extends Number>  
<T extends Comparable<T>>
```

Use when:

- Type ko limit karna hai.
- Specific class ke methods chahiye.

Tag: 🔥 MUST KNOW

6. Wildcard ? BASIC

- Hinglish:** ? ek unknown type hai. Use kabhi bhi type ke liye placeholder ki tarah karte hain.
- Why:** Flexibility in method parameters.
- Project Relevance:** Collections, utility methods, etc.
- Example:**

```
void printList(List<?> list) {  
    for (Object o : list) {  
        System.out.println(o);  
    }  
}
```

- Output:** List ke elements print ho jayenge.
- Interview Qs:** Wildcard kya hota hai? Kab use karte hain?
- Common Mistakes:** ? ko actual type ki tarah treat karna.
- What to Remember:** ? means "unknown type".

Wildcard (Syntax)

List<?> → any type

List<? extends Number> → Number ya uska subclass

List<? super Integer> → Integer ya uska superclass

Use when:

- Type unknown ho.
- Read-only type chahiye.

Tag: 🟡 BASIC

7. ? extends 🔥 MUST KNOW

- Hinglish:** Upper bound. Sirf given type ya uske subclass ko allow karta hai. ? tarah karte hain.
- Why:** Read-only operation ke liye best.
- Project Relevance:** Collection se data read karne ke time.
- Example:**

```
List<? extends Number> list = new ArrayList<Integer>();  
list.add(10); // allowed  
// list.add(20.5); // not allowed (since Integer is fixed)
```

- Output:** 10 add ho jayega, Double nahi.
- Interview Qs:** extends vs super difference?
- Common Mistakes:** ? extends me object add karnd.
- What to Remember:** extends = read only?

Extends (Syntax)

List<? extends ClassName>

Use when:

- Data ko READ karna hai.
- Parent class ya uske subclass allowed.

Tag: 🔥 MUST KNOW

8. ? super 🔥 MUST KNOW

- Hinglish:** Lower bound. Given type ya uske parent class ko allow karta hai. ? tarah karte hain.
- Why:** Write operation ke liye best.
- Project Relevance:** Data add karne ke time (e.g. saving to DB).
- Example:**

```
List<? super Integer> list = new ArrayList<Number>();  
list.add(10); // allowed  
// list.add(10.5); // not allowed (Double is not super of Integer)
```

- Output:** 10 add ho jayega, 10.5 nahi.
- Interview Qs:** extends vs extends difference?
- Common Mistakes:** ? super me read karne ki koshish.
- What to Remember:** super = write + (parent class allowed).

Super (Syntax)

List<? super ClassName>

Use when:

- Data ko WRITE karna hai.
- Given type ya uska parent allowed.

Tag: 🔥 MUST KNOW

9. PECS (Producer Extends, Consumer Super) 🔥 MUST KNOW

- Hinglish:** Ye ek simple rule hai jo batata hai ki kab extends aur kab super use karna hai.
- Why:** Type safety + flexibility.
- Project Relevance:** Collections, generics, utility methods.
- Rule:**

```
Producer (read) → ? extends T  
Consumer (write) → ? super T
```

- Example:** Methods me use hota hai (List<? extends T>, List<? super T>).
- Interview Qs:** PECS ka full form? Real use case?
- Common Mistakes:** Galat jagah extends/super use karnd.
- What to Remember:** Read = extends, Write = super.

Quick Revision (Part 1)

- ☐ Generics = flexibility + type safety
- ☐ extends = read | super = write
- ☐ PECS rule yaad rakho!

Next Steps

Type 1 → Welcome...
🔥 Ready for Part 2?
(Functional Programming)

💡 Summary - Part 1 (Generics)

Topic	Use When	Key Point
Type Parameters	Method/Class/Interface generic	<T> se type flexible.
Generic Class	Reusability	Generic class <T>.
Generic Method	Method level flexibility	<T> method me.
Generic Interface	Reusable contract	<T> in interface.
Bounded Types	Type limit	extends / Comparable.
Wildcard ?	Unknown type	flexible, read-only.
? extends	Read operation	upper bound.
? super	Write operation	lower bound.
PECS	Rule of thumb	extends = read, super = write.

Part 1 Complete! → Next: Part 2 (Functional Programming)

Keep coding, Keep learning! 😊
- Sandeep Chaurasiya

1. Functional Interface

- Interface with exactly 1 abstract method is called Functional Interface.

Example:

```
interface Calculator {
    int add(int a, int b);
}
```

Benefit:

- Used with Lambda
- Cleaner code
- Easy to understand

Real Project Example:

```
@FunctionalInterface
interface Validator {
    boolean validate(String value);
}
```

- Functional Interface is mainly used when we want to use Lambda or method reference.

2. @FunctionalInterface

- This is an annotation.
- It tells the compiler that this interface should be a Functional Interface.
- If we add more than 1 abstract method, compiler will give an error.

Wrong Example:

```
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
    int subtract(int a, int b);
} X Error
```

Why use it?

- Compile-time safety
- Better code clarity
- Helps in future maintenance

Important:

- Not mandatory, but recommended.
- It does not mean you must use Lambda. It just checks for 1 abstract method.

3. Lambda Expression

- Lambda = short form of a method, without method name.

Syntax:

(parameters) → expression

Examples:

1) No parameter:

() → System.out.println("Hello");

2) One parameter:

name → System.out.println(name);

3) Two parameters:

(a, b) → a + b;

4) Multiple statements:

```
(a, b) → {
    int result = a + b;
    return result;
}
```

Real Project Example:

```
@FunctionalInterface
interface Greeting {
    void sayHello(String name);
}
```

```
public class Main {
    public static void main(String[] args) {
        Greeting g = name → System.out.println
            ("Hello " + name);
        g.sayHello("Sandeep");
    }
}
```

Quick Recap

- Functional Interface → 1 abstract method
- @FunctionalInterface → compile-time check
- Lambda → short implementation

Remember

- Functional Interface → 1 method
- @FunctionalInterface → check
- Lambda → () →
- Used with → Predicate / Consumer / Supplier / Function / Method Reference

Keep it Simple 😊

1. Predicate

- Predicate ek Functional Interface hai jo condition check karke boolean return karta hai.
- Main method: `test()`
- Use: `filter()`, validation, condition checking.

Syntax:

```
Predicate<T> p = value → condition;
```

Example:

```
Predicate<Integer> adult = n → n >= 18;

System.out.println(adult.test(20)); // true
System.out.println(adult.test(15)); // false
```

Real Project Example:

```
List<Integer> ages = List.of(15, 20, 17, 25);
Predicate<Integer> adult = age → age >= 18;

ages.stream().filter(adult)
    .forEach(System.out::println);

// 20, 25
```

* Remember: `test()` → boolean

2. Consumer

- Consumer ek Functional Interface hai jo input leta hai, lekin kuch return nahi karta (void).
- Main method: `accept()`
- Use: `forEach()`, logging, printing, notification, update, etc.

Syntax:

```
Consumer<T> c = value → {
    // action
};
```

Example:

```
Consumer<String> print = name →
    System.out.println(name);

print.accept("Sandeep"); // Sandeep
```

Real Project Example:

```
List<String> names = List.of("Amit",
    "Rahul", "Sandeep");
names.forEach(name →
    System.out.println(name));

// ya phir names.forEach(System.out::println);
```

* Remember: `accept()` → void

3. Supplier

- Supplier ek Functional Interface hai jo koi input nahi leta, bas ek value return karta hai.
- Main method: `get()`
- Use: default value, random value, current time, lazy value generation.

Syntax:

```
Supplier<T> s = () → value;
```

Example:

```
Supplier<String> s = () → "Hello";

System.out.println(s.get()); // Hello
```

Real Project Example:

```
Supplier<Long> currentTime =
    () → System.currentTimeMillis();

System.out.println(currentTime.get());
```

* Remember: `get()` → value

Quick Comparison

Interface	Input	Output	Main Method	Main Use
Predicate<T>	1	boolean	test()	Check / condition
Consumer<T>	1	void	accept()	Action / forEach()
Supplier<T>	0	1 value	get()	Provide / generate

Shortcut:

Predicate → Check | Consumer → Do | Supplier → Give

Important Points

- Predicate → `filter()` / validation
- Consumer → `forEach()` / logging
- Supplier → default value / lazy init
- All 3 are Functional Interfaces.
- Use lambda for simple cases.
- Method reference bhi use kar sakte ho (e.g. `System.out::println`).

Keep
Going
😊

1. Function

- Input lekar usko process karke ek output return karta hai.

Main method: `apply()`

• Syntax:

```
Function<T, R> f = value → result;
```

• Example:

```
Function<String, Integer> length = s → s.length();
System.out.println(length.apply("Java")); // 4
```

• Real Project Use:

- Stream map()
- DTO/Entity se value nikalna
- Data transformation

• Remember:

T = Input type
R = Return type

* Shortcut: Transform → apply()

2. Method Reference

- Lambda ka short & clean form.
- Jab existing method ko directly refer kar sakte ho.

• Syntax:

```
ClassName::methodName
object::methodName
ClassName::new (constructor)
```

• Example:

```
List<String> names = List.of("Amit", "Rahul");
names.forEach(System.out::println);
// same as name → System.out.println(name)
```

• Common Examples:

- System.out::println
- String::toUpperCase
- User::new (constructor)

• Remember:

Agar lambda sirf kisi existing method ko call kar raha hai, to Method Reference use kar sakte ho.

* Not possible: $x \rightarrow x + 10$ (no existing method)

3. Quick Revision

Functional Interfaces

Interface	Input	Output	Method	Use
<u>Predicate<T></u>	1	boolean	test()	Check
<u>Consumer<T></u>	1	void	accept()	Action
<u>Supplier<T></u>	0	1 value	get()	Provide
<u>Function<T,R></u>	1	1 value	apply()	Transform

Lambda → Short implementationMethod Reference Types

ClassName::methodName (e.g. System.out::println)
object::methodName (e.g. list::size)
ClassName::new (e.g. User::new)

Key Patterns (Real Code)

```
users.stream()
    .filter(user → user.isActive()) // Predicate
    .map(user → user.getName()) // Function
    .forEach(name → System.out.println(name)); // Consumer
```

* This pattern is very important for projects + interviews.

Important Interview Questions

- Functional Interface kya hai?
→ 1 abstract method.
- Lambda kya hai?
→ Functional interface ka short form.
- Predicate ka main method?
→ test() (boolean).
- Consumer kya return karta hai?
→ void.
- Function ka main method?
→ apply().
- Method Reference kya hai?
→ :: (existing method ka short form).

Coding Practice (3 Questions)

- Q1 Names ko upper case me convert karo.
(List<String> → map(String::toUpperCase))
- Q2 18+ users filter karo.
(Predicate + filter())
- Q3 Names print karo using method reference.
(forEach(System.out::println))

Quick Code Snippets

- Predicate: $n \rightarrow n > 10$
- Consumer: $n \rightarrow \text{System.out.println}(n)$
- Supplier: $() \rightarrow \text{"Hello"}$
- Function: $s \rightarrow s.length()$
- Method Ref: `System.out::println`

★ Timeline / Importance

- 🔥 Lambda → MUST KNOW
- 🔥 Predicate → MUST KNOW
- 🔥 Function → MUST KNOW
- 🔥 Method Reference → IMPORTANT
- 🟡 Consumer → IMPORTANT
- 🟡 Supplier → BASIC
- 🟡 Functional Interface → BASIC
- 🟡 @FunctionalInterface → BASIC

Last Reminder

- ✓ Focus on practical usage only.
- ✓ No deep theory / internal details.
- ✓ Streams + Spring Boot me kaafi use hoga.

Keep Going 😊

1. Stream Creation / Stream()

Java 8+
(Stream API)

① Definition :

Stream creation means creating a **Stream** from a source (like a collection, array, or other data source) using the `stream()` method or other factory methods.

② Why we use it :

- To process data in a functional style.
- Helps to perform operations like filter, map, sorted, collect etc.
- Makes code cleaner, shorter and easy to read.
- Useful in real-world Java/Spring Boot projects for data processing, transformation and reporting.

③ Basic Syntax :

```
List<T> list = ...;  
Stream<T> stream = list.stream();
```

→ Creates a sequential stream from the list.

④ Very Simple Example :

```
List<String> names = Arrays.asList("Amit", "Ravi", "Neha");  
Stream<String> stream = names.stream();  
System.out.println(stream); // just to show stream is created
```

⑤ Example with List :

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);  
Stream<Integer> stream = numbers.stream();  
Stream.forEach(n -> System.out.println(n));
```

Input : numbers = [1, 2, 3, 4, 5]

Output : 1
2
3
4
5

⑥ Step-by-Step Execution / Flow :

1. Collection/array se data milta hai.
2. `stream()` method call karte hi ek Stream object create hota hai.
3. Ab is stream par aage operations (filter, map etc.) chain kar sakte hai.
4. `collect()` ya terminal operation ke baad result milta hai.



(`Stream()` just creates the stream, it doesn't process anything)

⑦ Real-world Use Case (Java / Spring Boot) :

Suppose you have a list of users from a database and you want to get the names of only active users and show them in a web page (Spring Boot controller).

```
List<User> users = userRepository.findAll();  
List<String> activeUserNames = users.stream()  
    .filter(User::isActive)  
    .map(User::getName)  
    .collect(Collectors.toList());
```

→ Here `stream()` is used to start the processing on the list of users.

⑧ Interview-Important Points :

- `Stream()` creates a sequential stream (default).
- It is an intermediate operation.
- It does not modify the original collection.
- You can create a stream from:
 - Collection (List, Set, etc.)
 - Array (`Arrays.stream()`)
 - Specific values (`Stream.of()`)
 - Files (`Files.lines()`)

⑨ Common Mistakes :

- ✗ Bhoolna ki `stream()` original list ko modify nahi karta.
- ✗ Terminal operation (like `collect`, `forEach`) ke bina result nahi milta.
- ✗ Stream ko baar-baar reuse nahi kar sakte.
- ✗ Null list par `stream()` call karne se `NullPointerException` aayega.

⑩ Important Coding Examples :

1. From List

```
List<String> list = Arrays.asList("a", "b", "c");  
Stream<String> s = list.stream();
```

2. From Array

```
String[] arr = {"Java", "Spring", "Boot"};  
Stream<String> s = Arrays.stream(arr);
```

3. From `Stream.of()`

```
Stream<Integer> s = Stream.of(10, 20, 30);
```

⑪ Practice Question :

1. Create a stream from a list of 5 product names and print them using `forEach()`.
2. Create a stream from an array of integers and print the square of each number.

⑫ Revision / Memory Trick :

"`Stream()` = Stream ka gate, aage operations se hota hai work!"

Sandeep Chaurasiya 😊

2. filter()

① Definition :

`filter()` ek intermediate operation hai jo stream ke elements ko check karta hai aur sirf un elements ko aage bhejta hai jo specified condition (predicate) ko satisfy karte hain.

② Why we use it :

- Unwanted data ko remove karne ke liye.
- Sirf required elements ko process karne ke liye.
- Code ko clean aur readable banane ke liye.
- Real-world data filtering (e.g. active users, valid records, etc.) ke liye.

③ Basic Syntax :

```
Stream<T> filter(Predicate<? super T> predicate);
```

→ Predicate ek condition check karta hai (true/false).

④ Very Simple Example :

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> evenNumbers = numbers.stream()
    .filter(n -> n % 2 == 0) // sirf even numbers
    .collect(Collectors.toList());
System.out.println(evenNumbers);
```

Input : [1, 2, 3, 4, 5]

Output : [2, 4]

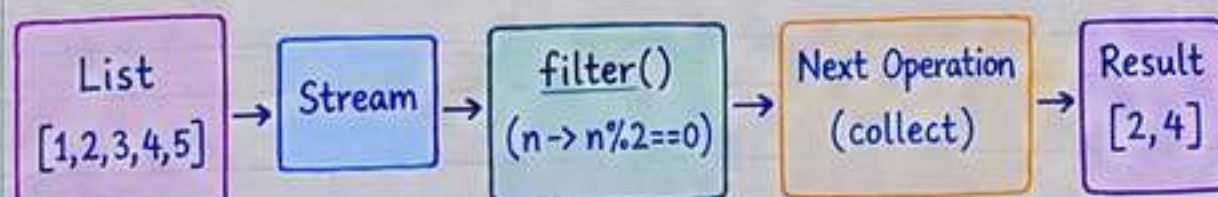
⑤ Example with List (Students) :

```
List<Student> students = Arrays.asList(
    new Student(1, "Amit", 82),
    new Student(2, "Neha", 56),
    new Student(3, "Ravi", 91),
    new Student(4, "Pooja", 45)
);
List<Student> passed = students.stream()
    .filter(s -> s.getMarks() >= 60) // sirf 60+
    .collect(Collectors.toList());
System.out.println(passed);
```

Output : [Student{id=1, name='Amit', marks=82},
Student{id=3, name='Ravi', marks=91}]

⑥ Step-by-Step Execution / Flow :

1. Stream banega (list se).
2. `filter()` condition check karega (predicate).
3. Jo true honge, sirf wahi elements aage jayenge.
4. Baaki elements skip ho jayenge.



(`filter()` unwanted elements ko remove kar deta hai)

⑦ Real-world Use Case (Java / Spring Boot) :

Suppose hum user list se sirf active users ko fetch karna chahte hain (status = "ACTIVE").

```
List<User> activeUsers = users.stream()
    .filter(u -> "ACTIVE".equals(u.getStatus()))
    .collect(Collectors.toList());
```

→ Ye usually Spring Boot services, controllers ya service layer me use hota hai.

⑧ Interview-Important Points :

- `filter()` ek intermediate operation hai.
- Ye `Predicate<T>` accept karta hai.
- Ye lazy evaluation karta hai (jab terminal operation call hota hai tab execute hota hai).
- Predicate me multiple conditions use kar sakte hain (&&, ||, !).

⑨ Common Mistakes :

- ✗ Predicate return type galat dena.
- ✗ null values ko handle na karna.
- ✗ `filter()` ke baad `collect()` na lagana (agar result chahiye ho).
- ✗ String comparison me `==` use karna (`equals()` use karo).

⑩ Important Coding Examples :

1. Multiple Conditions :

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
List<Integer> result = numbers.stream()
    .filter(n -> n > 2 && n < 6)
    .collect(Collectors.toList());
// Output: [3, 4, 5]
```

2. Method Reference (with filter) :

```
List<Employee> employees = getEmployees();
List<Employee> deptList = employees.stream()
    .filter(Employee::getDepartment) // method ref
    .collect(Collectors.toList());
// Sirf department wali values aayengi (non-null).
```

3. Removing Null Values :

```
List<String> names = Arrays.asList("Amit", null, "Ravi", null);
List<String> cleaned = names.stream()
    .filter(Objects::nonNull)
    .collect(Collectors.toList());
// Output: [Amit, Ravi]
```

⑪ Practice Question :

1. Ek list of integers se sirf even numbers nikal kar list banao using `filter()`.
2. Ek student list se sirf un students ko nikalo jinke marks 70 se zyada hain.

⑫ Revision / Memory Trick :

"`filter()` = Condition lagao, unwanted hatao."

Sandeep Chaurasiya 😊

3. map()

① Definition :

map() ek intermediate operation hai jo har element ko ek naye form me convert karta hai. Ye original elements ko modify nahi karta, balki unke upar diye gaye function ko apply karke naye values return karta hai.

② Why we use it :

- Data ko transform karne ke liye (e.g. String → Length).
- Ek type ko dusre type me convert karne ke liye (e.g. Entity → DTO, String → Uppercase).
- Clean aur readable code likhne ke liye.
- Real-world projects me data formatting, mapping aur conversion ke liye bahut useful hai.

③ Basic Syntax :

```
List<T> list = ...;  
List<R> result = list.stream().map(...).collect(Collectors.toList());  
                                ↙  
                                Function<T, R>  
                                (lambda ya method reference)
```

④ Very Simple Example :

```
List<String> names = Arrays.asList("Amit", "Ravi", "Neha");  
List<Integer> lengths = names.stream()  
    .map(name -> name.length()) // har naam ki length nikal rahi hai  
    .collect(Collectors.toList());
```

Input : ["Amit", "Ravi", "Neha"]

Output : [4, 4, 4]

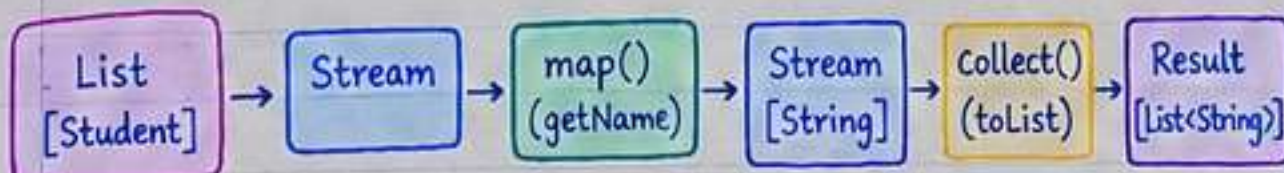
⑤ Example with List (Students) :

```
List<Student> students = Arrays.asList(  
    new Student(1, "Amit", 82),  
    new Student(2, "Neha", 56),  
    new Student(3, "Ravi", 91)  
);  
List<String> names = students.stream()  
    .map(Student::getName) // method reference  
    .collect(Collectors.toList());  
System.out.println(names);
```

Output : [Amit, Neha, Ravi]

⑥ Step-by-Step Execution / Flow :

1. Stream banega (list se).
2. map() method har element par function apply karega.
3. Har element ka result ek naye value me convert hoga.
4. collect() final list banayega.



(map() original data ko modify nahi karta, sirf naye values create karta hai.)

⑦ Real-world Use Case (Java / Spring Boot) :

Suppose hum ek user list se unke email addresses nikalna chahte hain aur unhe uppercase me convert karna chahte hain (e.g. notifications ke liye).

```
List<User> users = userRepository.findAll();  
List<String> emails = users.stream()  
    .map(User::getEmail)  
    .map(String::toUpperCase)  
    .collect(Collectors.toList());
```

→ Pehle email nikala, fir usse uppercase me convert kiya.

⑧ Interview-Important Points :

- map() ek intermediate operation hai.
- Ye har element ko transform karta hai.
- Function<T, R> accept karta hai (lambda ya method ref).
- Ye lazy evaluation karta hai (jab terminal operation call hota hai tab execute hota hai).
- Parallel stream me map() thread-safe hona chahiye (agar shared state use kar rahe ho to).

⑨ Common Mistakes :

- ✗ map() ko filter() ki tarah use karna.
- ✗ Function ka sahi return type na dena.
- ✗ Null values handle na karna (agar data me null ho).
- ✗ collect() lagana bhool jana (terminal operation).

⑩ Important Coding Examples :

1. String to Length :

```
List<String> words = Arrays.asList("Java", "Spring", "Boot");  
List<Integer> lengths = words.stream()  
    .map(String::length)  
    .collect(Collectors.toList());  
System.out.println(lengths); // [4, 6, 4]
```

2. Employee Name to Uppercase :

```
List<Employee> list = employees.stream()  
    .map(Employee::getName)  
    .map(String::toUpperCase)  
    .collect(Collectors.toList());  
System.out.println(list); // [AMIT, NEHA, RAVI]
```

3. Student to Name + Marks (DTO/Custom Object) :

```
List<StudentDTO> result = students.stream()  
    .map(s -> new StudentDTO(s.getName(), s.getMarks()))  
    .collect(Collectors.toList());  
System.out.println(result); // [StudentDTO(...), ...]
```

⑪ Practice Question :

1. Ek list of integers se unke square nikal kar ek new list banao using map().
2. Ek list of strings se unke first character nikal kar ek new list banao using map().

⑫ Revision / Memory Trick :

"map() = Modify / Transform"
(Data change nahi hota, sirf new value milti hai.)

Sandeep Chaurasiya 😊

4. sorted()

① Definition :

sorted() ek intermediate operation hai jo stream ke elements ko natural order (ascending) me sort karta hai. Agar hume reverse order chahiye to hum Comparator bhi pass kar sakte hain.

② Why we use it :

- Data ko sorted order me chahiye hota hai.
- Report, list display, top-N etc. ke liye useful.
- Clean aur readable code likhne me help karta hai.
- Java 8 se pehle hume manually collection sort karna padta tha, ab stream se easy ho gaya.

③ Basic Syntax :

```
Stream<T> sorted() → natural order
Stream<T> sorted(Comparator<? super T> comparator) → custom order
```

④ Very Simple Example :

```
List<String> names = Arrays.asList("Ravi", "Amit", "Neha");
List<String> sortedNames = names.stream()
    .sorted()
    .collect(Collectors.toList());
System.out.println(sortedNames);
```

Input : [Ravi, Amit, Neha] → (Alphabetical order me sort ho gaya)
Output : [Amit, Neha, Ravi]

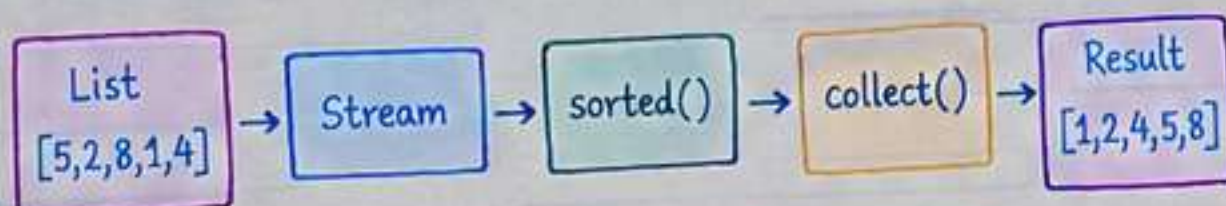
⑤ Example with List (Numbers) :

```
List<Integer> numbers = Arrays.asList(5, 2, 8, 1, 4);
List<Integer> sortedNumbers = numbers.stream()
    .sorted()
    .collect(Collectors.toList());
System.out.println(sortedNumbers);
```

Input : [5, 2, 8, 1, 4] (Ascending order)
Output : [1, 2, 4, 5, 8]

⑥ Step-by-Step Execution / Flow :

1. Stream banega (list se).
2. sorted() method har element ko compare karega.
3. Elements ko ascending order me arrange karega.
4. collect() se final list milega.



(sorted() original data ko modify nahi karta, new sorted list return karta hai.)

⑦ Real-world Use Case (Java / Spring Boot) :

Suppose hum ek user list ko name ke basis par alphabetical order me dikhana chahte hain (e.g. admin panel, user search, reports).

```
List<User> users = userRepository.findAll();
List<User> sortedUsers = users.stream()
    .sorted(Comparator.comparing(User::getName))
    .collect(Collectors.toList());
```

→ User objects ko name ke basis par sort karte hain.

⑧ Interview-Important Points :

- sorted() ek intermediate operation hai.
- Natural order ke liye no argument.
- Custom order ke liye Comparator pass karte hain.
- Objects ko sort karne ke liye Comparable interface ya Comparator use hota hai.
- sorted() stream ke elements ko sort karta hai, original list me change nahi karta.

⑨ Common Mistakes :

- ✗ sorted() ke baad collect() na lagana.
- ✗ Comparator me galat logic likhna.
- ✗ Null values ko handle na karna (NPE aa sakta hai).
- ✗ Assuming ki input list modify ho jayegi (original list change nahi hoti).

⑩ Important Coding Examples :

1. Natural Order (String) :

```
List<String> names = Arrays.asList("Ravi", "Amit", "Neha");
List<String> sorted = names.stream()
    .sorted()
    .collect(Collectors.toList());
System.out.println(sorted); // [Amit, Neha, Ravi]
```

2. Reverse Order (String) :

```
List<String> names = Arrays.asList("Ravi", "Amit", "Neha");
List<String> sorted = names.stream()
    .sorted(Comparator.reverseOrder())
    .collect(Collectors.toList());
System.out.println(sorted); // [Ravi, Neha, Amit]
```

3. Custom Order (Numbers) :

```
List<Integer> numbers = Arrays.asList(5, 2, 8, 1, 4);
List<Integer> sorted = numbers.stream()
    .sorted(Comparator.comparingInt(a -> a))
    .collect(Collectors.toList());
System.out.println(sorted); // [1, 2, 4, 5, 8]
```

⑪ Practice Question :

1. Ek list of strings dekar use reverse alphabetical order me sort karo.
2. Ek list of integers ko descending order me sort karo.

⑫ Revision / Memory Trick :

"sorted() = Arrange karo, natural ya custom order me."

Sandeep Chaurasiya 😊

5. collect() + 6. Collectors.groupingBy()

5. collect()

1. Simple Definition:

collect() ek **TERMINAL** operation hai jo Stream ke elements ko final result/collection me convert karta hai.

→ collect() stream ko consume karta hai.

2. Why we use it:

- Stream ka result List me chahiye
- Set me chahiye
- Map me chahiye
- Data ko group/count/join karna ho

3. Basic Syntax:

```
List<T> result = stream.collect(Collectors.toList());
Set<T> result = stream.collect(Collectors.toSet());
Map<K,V> result = stream.collect(Collectors.toMap(
    keyMapper, valueMapper));
```

4. Very Simple Example:

```
List<String> names = Arrays.asList("Amit", "Ravi", "Neha");
List<String> upperNames = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
System.out.println(upperNames);
```

Input: [Amit, Ravi, Neha]

Output: [AMIT, RAVI, NEHA]

6. Example with List:

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> evenNumbers = numbers.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());
```

Output: [2, 4]

7. Step-by-step flow:



→ collect() result ko collection me convert karta hai.

8. Real-world Java/Spring Boot use case:

```
List<Customer> customers = customerRepository.findAll();
List<CustomerDTO> dtoList = customers.stream()
    .map(c -> new CustomerDTO(c.getId(), c.getName()))
    .collect(Collectors.toList());
```

→ Spring Boot API response ke liye Entity → DTO List.

9. Interview-important points:

- collect() is a terminal operation.
- It uses Collector.
- Common collectors: toList(), toSet(), toMap(), groupingBy(), counting(), joining()
- Stream cannot be reused after a terminal operation.
- collect() does not modify the original collection.

Final List

10. Common mistakes:

- collect() ko intermediate operation samajhna
- Wrong Collector choose karna
- toMap() me duplicate keys handle na karna
- Terminal operation ke baad same Stream reuse karna

11. Important coding examples:

A. Collect to Set

```
Set<String> set = names.stream()
    .collect(Collectors.toSet());
```

B. Collect to Map

```
Map<String, Integer> map =
    names.stream()
    .collect(Collectors.toMap(
        String::toLowerCase,
        String::length));
```

C. counting()

```
long count = names.stream()
    .collect(Collectors.counting());
```

Comparison:

collect() → Stream ka final result collect karta hai.

groupingBy() → Elements ko key ke basis par groups me divide karta hai.

Important Relationship:



6. Collectors.groupingBy()

1. Simple Definition:

groupingBy() elements ko kisi condition/Key ke basis par groups me divide karta hai.

→ Ek key → us key ke andar matching elements.

2. Why we use it:

- Department-wise employees
- Category-wise products
- City-wise users
- Status-wise orders
- Count per category

3. Basic Syntax:

```
Map<K, List<T>> result =
    list.stream()
    .collect(Collectors.groupingBy(T::getSomething));
```

→ getSomething() grouping key decide karta hai.

4. Very Simple Example:

```
List<String> names = Arrays.asList("Amit", "Ravi", "Aman", "Neha");
Map<Integer, List<String>> grouped =
    names.stream()
    .collect(Collectors.groupingBy(String::length));
```

Output:

```
{
  3 = [Amit, Ravi, Aman],
  4 = [Neha]
}
```

5. Most important Employee example:

```
List<Employee> employees = Arrays.asList(
    new Employee(1, "Amit", "IT"),
    new Employee(2, "Neha", "HR"),
    new Employee(3, "Ravi", "IT")
);
Map<String, List<Employee>> result =
    employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));
```

Output:

```
IT → [Amit, Ravi]
HR → [Neha]
```

Employee::getDepartment
= employee → employee.getDepartment()

6. Step-by-step execution:



7. Real-world Java/Spring Boot use case:

```
Map<String, List<Product>> productsByCategory =
    products.stream()
    .collect(Collectors.groupingBy(Product::getCategory));
```

→ Reporting APIs aur dashboard data preparation me useful.

8. Interview-important points:

- groupingBy() is commonly used with collect().
- Return type is generally Map<K, List<T>>.
- Key decides the group.
- Multiple records can belong to the same group.
- groupingBy() can also be combined with counting().

9. Important counting example:

```
Map<String, Long> countByDepartment =
    employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.counting()));
```

Output:

```
IT → 2
HR → 1
```

→ groupingBy + counting = har group me kitne elements hain.

10. Common mistakes:

- groupingBy() ka result List samajhna; normally Map hota hai.
- Wrong grouping key choose karna.
- Duplicate data ko galat samajhna — same group me aa sakte hain.
- Employee::getDepartment ka meaning na samajhna.

11. Practice Questions:

1. Employee list ko department-wise group karo.
2. Product list ko category-wise group karo.
3. Department-wise employee count nikalo using: groupingBy() + counting().

FINAL MEMORY TRICK:

"collect() = RESULT jama karo"
"groupingBy() = GROUP banaao"

Sandeep Chaurasiya 😊

(Java 8+)

★ 1. **LocalDate** → Sif date ko represent karta hai (Time nahi).

→ Example:

```

    LocalDate date = LocalDate.now();
    System.out.println(date);    // 2024-09-08
  
```

→ Specific date:

```

    LocalDate date = LocalDate.of(2024, 9, 8);
    System.out.println(date);    // 2024-09-08
  
```

→ Important methods:

- `getYear()`, `getMonth()`, `getDayOfMonth()`
- `plusDays()`, `minusDays()`
- `isBefore()`, `isAfter()`, `isEqual()`

★ 2. **LocalTime** → Sif time ko represent karta hai (Date nahi).

→ Example:

```

    LocalTime time = LocalTime.now();
    System.out.println(time);    // 19:35:20
  
```

→ Specific time:

```

    LocalTime time = LocalTime.of(14, 30);
    System.out.println(time);    // 14:30
  
```

→ Important methods:

- `getHour()`, `getMinute()`, `getSecond()`
- `plusHours()`, `minusMinutes()`
- `isBefore()`, `isAfter()`

★ 3. **LocalDateTime** → Date + Time (but no timezone).

→ Example:

```

    LocalDateTime dt = LocalDateTime.now();
    System.out.println(dt);    // 2024-09-08T19:40:25
  
```

→ Specific date + time:

```

    LocalDateTime dt = LocalDateTime.of(2024, 9, 8, 19, 30);
    System.out.println(dt);    // 2024-09-08T19:30
  
```

→ Important methods:

- `getYear()`, `getMonth()`, `getDayOfMonth()`
- `getHour()`, `getMinute()`, `getSecond()`
- `plusDays()`, `plusHours()`, `minusMinutes()`

Use when you need date + time but timezone is not required.

* 4. ZonedDateTime → Date + Time + Timezone

→ Example:

```
ZonedDateTime now = ZonedDateTime.now();  
System.out.println(now); // 2025-09-08T19:45:00+05:30[Asia/Kolkata]
```

→ Specific timezone:

```
ZonedDateTime india = ZonedDateTime.now(ZoneId.of("Asia/Kolkata"));  
System.out.println(india);  
ZonedDateTime usa = ZonedDateTime.now(ZoneId.of("America/New-York"));  
System.out.println(usa);
```

→ Real use:

- International applications
- Meeting across countries

Difference:

- LocalDateTime → date + time (+/- timezone)
- ZonedDateTime → date + time + timezone

* 5. Instant → UTC timeline ka exact moment.

→ Example:

```
Instant now = Instant.now();  
System.out.println(now); // 2025-09-08T14:20:30Z
```

→ Use:

- Logs, database timestamps, distributed systems, API events
- Timezone-independent exact point in time

* 6. Duration → Time difference measure karne ka liye.

→ Example:

```
LocalTime start = LocalTime.of(10, 0);  
LocalTime end = LocalTime.of(12, 30);  
Duration d = Duration.between(start, end);  
System.out.println(d.toHours()); // 2  
System.out.println(d.toMinutes()); // 150
```

Use for: API execution time, performance, debugging.

- Duration → time-based difference

Common units:

- Hours
- Minutes
- Seconds
- Nanoseconds

* 7. Period → Date difference (Years + Months + Days)

→ Example:

```
LocalDate start = LocalDate.of(2025, 9, 8);  
LocalDate end = LocalDate.of(2026, 9, 8);  
Period p = Period.between(start, end);  
System.out.println(p.getYears()); // 1
```

Use:

- Employee experience
- Age calculation
- Date-based logic

Period → date-based difference

* 8. Date Formatting → Date ko string me convert karne.

→ Example:

```
LocalDate date = LocalDate.of(2025, 9, 8);  
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy");  
String result = date.format(formatter);  
System.out.println(result); // 08-09-2025
```

Common patterns:

- dd - day
- MM - month
- yyyy - year
- mm - minute
- ss - second

* 9. Date Parsing → String ko Date me convert karne.

→ Example:

```
String text = "08-09-2025";  
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy");  
LocalDate date = LocalDate.parse(text, formatter);  
System.out.println(date); // 2025-09-08
```

Trick:

- Format = Date → String
- Parse = String → Date

PPT or book me date ko LocalDate me convert kar ke use kar sakte hain

1. LocalDate - Concept

- LocalDate ek date ko represent karta hai (year, month, day).
- Isme time (hour, minute, second) nahi hota.
- Yeh immutable (change nahi hota) hota hai.
- Ye timezone aware nahi hota.
- Common use: birth date, due date, invoice date, etc.

2. Common Methods

- now() : current date
- of() : specific date
- getYear(), getMonth(), getDayOfMonth()
- plusDays(), minusDays()
- plusMonths(), minusMonths()
- isBefore(), isAfter(), isEqual()

3. Example - Getting and using LocalDate

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class LocalDateExample {
    public static void main(String[] args) {
        // 1. current date
        LocalDate today = LocalDate.now();
        System.out.println("Today: " + today);

        // 2. specific date
        LocalDate dob = LocalDate.of(2000, 5, 15);
        System.out.println("DOB: " + dob);

        // 3. add 5 days
        LocalDate future = today.plusDays(5);
        System.out.println("After 5 days: " + future);

        // 4. format date
        DateTimeFormatter formatter =
            DateTimeFormatter.ofPattern("dd-MM-yyyy");
        String formatted = today.format(formatter);
        System.out.println("Formatted: " + formatted);
    }
}
```

🔥 MUST KNOW

- ✓ LocalDate
- ✓ LocalTime
- ✓ LocalDateTime
- ✓ ZonedDateTime
- ✓ Instant
- ✓ Duration
- ✓ Period
- ✓ Date Formatting
- ✓ Date Parsing

When to use which?

- LocalDate : sirf date chahiye
- LocalTime : sirf time chahiye
- LocalDateTime : date + time
- ZonedDateTime : date + time + timezone
- Instant : exact point in time (UTC)
- Duration : time difference (hours/min)
- Period : date difference (days/months/years)

4. Line by Line Explanation

- ① import ... : Required classes.
- ② public class ... : Class start.
- ③ public static void main() :
Entry point of program.
- ④ LocalDate.now() : current date.
- ⑤ LocalDate.of() : create date
(year, month, day).
- ⑥ plusDays(5) : 5 days add.
- ⑦ DateTimeFormatter : format date
in given pattern.
- ⑧ format() : date ko string me
convert karta hai.
- ⑨ System.out.println() : output
- ⑩ } : end of class and method.

5. Real World / Project Example

- User registration me DOB store karna.
- Order/Invoice date save karna.
- Report generation me specific date range use karna.
- End date calculate karna (e.g. subscription expiry).

6. Spring Boot / API Use Cases

- REST API me date receive karna (JSON).
- Response me date ko format karke send karna.
- Database me LocalDate use karna (JPA).
- Date based filtering (e.g. /orders?date=2025-09-01).

7. Interview Points (Must Remember)

- ✓ LocalDate vs LocalDateTime difference.
- ✓ Date ko format aur parse kaise karte hain?
- ✓ ZoneId ka kya use hai?
- ✓ Immutability ka kya matlab hai?
- ✓ plusDays vs plusMonths difference.

8. Common Interview Questions

1. LocalDate aur LocalDateTime me kya difference hai?
2. Date ko string se kaise convert karte hain?
3. LocalDate me 1 month add karne ka tarika?
4. ZonedDateTime kya hota hai?
5. Instant ka kya use hai?

9. Practical Coding Questions

1. Aaj ki date se 7 days baad ki date nikaalo.
2. 2025-01-01 se 10 days pehle ki date nikaalo.
3. Do dates ke beech kitne days ka difference hai?

🔥 Quick Revision - MUST REMEMBER

- LocalDate = date only
- LocalTime = time only
- LocalDateTime = date + time
- ZonedDateTime = date + time + timezone
- Instant = exact point in time (UTC)
- Duration = time difference (hours/min)
- Period = date difference (days/months/years)

★ Next: LocalTime → (Page 2)

1. Process vs Thread

- Process : independent program, has its own memory space, resources.
- Thread : lightweight, shares same memory (heap) with other threads of same process.

Feature	Process	Thread
Memory	Separate	Shared
Communication	Difficult	Easy
Overhead	High	Low
Example	Chrome app	Chrome tabs

Real World Example:

Browser = multiple processes, but each tab is a thread (lightweight).

2. Thread Creation

1) Extending Thread class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread started");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

- run() → contains the code.
- start() → creates a new thread & calls run().

3. Runnable Interface

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Running");
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t = new Thread(new MyRunnable());
        t.start();
    }
}
```

- Runnable is better than Thread class (because we can extend other classes).

4. Callable & Future

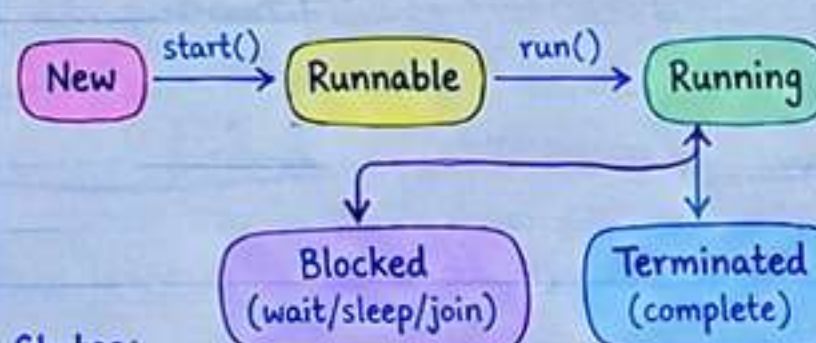
```
import java.util.concurrent.*;

class MyCallable implements Callable<String> {
    public String call() {
        return "Hello from Callable";
    }
}

public class Main {
    public static void main(String[] args) {
        throws Exception {
            ExecutorService executor = Executors.
                newSingleThreadExecutor();
            Future<String> future = executor.submit(
                new MyCallable());
            System.out.println(future.get());
            executor.shutdown();
        }
    }
}
```

- Callable → returns a value.
- Future → get() se result milta hai.

5. Thread Lifecycle



States:

1. New - object created.
2. Runnable → ready to run.
3. Running → executing.
4. Blocked → waiting for resource.
5. Terminated - finished.

6. Common Thread Methods

Method	Use
start()	Starts new thread (calls run()).
run()	Contains thread code.
sleep(ms)	Pause thread for given time.
join()	Wait for another thread to finish.
interrupt()	Stops a sleeping/waiting thread.
yield()	Suggests scheduler to switch.
setPriority()	Set thread priority (low...high).
setDaemon()	Make thread as daemon.

Example - sleep() & join()

```
Thread t = new Thread() → {
    for (int i = 1; i <= 5; i++) {
        System.out.println("T1: " + i);
        try { Thread.sleep(1000); }
        catch (InterruptedException e) {}
    }
};

t.start();
t.join(); // main thread waits
System.out.println("Done");
```

7. Thread Priority & Daemon Thread

- Priority: 1 (MIN) to 10 (MAX).
- Daemon: background thread (e.g. GC).

Good to know (Low Priority)

- yield() - hint to scheduler.
- Thread Priority → rarely used.
- Very deep JVM internals - skip.

8. Synchronization & synchronized

- synchronized → only one thread at a time can access the block/method.

```
class Counter {
    private int count = 0;
    public synchronized void increment() {
        count++;
    }
}
```

9. Race Condition

- When multiple threads access shared data without proper synchronization.

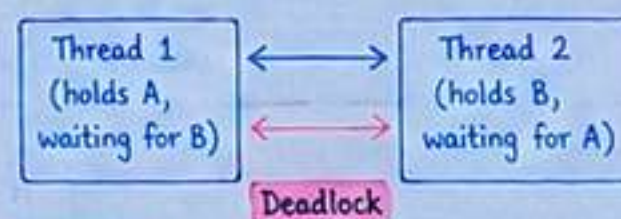
```
count = count + 1; // not safe
// fix: synchronized or atomic class
```

Real World Example:

Multiple users booking same seat at the same time.

10. Deadlock

- Two threads wait for each other and never finish.



Example:

- Lock a, then b in one thread,
- Lock b, then a in another thread.

11. wait(), notify(), notifyAll()

- wait() → releases lock & waits.
- notify() → wakes one thread.
- notifyAll() → wakes all threads.

```
synchronized (obj) {
    obj.wait(); // wait
    obj.notify(); // wake one
    obj.notifyAll(); // wake all
}
```

12. Executor Framework & ThreadPool

- ExecutorService → high level API.
- ThreadPool → reuses threads (better performance).

```
ExecutorService executor = Executors.newFixedThreadPool(5);
executor.submit(() → { /* task */ });
executor.shutdown();
```

Why use ThreadPool?

- ✓ Less overhead
- ✓ Better performance
- ✓ Manage multiple tasks
- ✓ Avoids creating new threads again and again.

13. Concurrent Collections

- Thread-safe collections.
- Examples: ConcurrentHashMap, CopyOnWriteArrayList, ConcurrentLinkedQueue.

Example:

```
ConcurrentHashMap<String, Integer> map =
    new ConcurrentHashMap<>();
map.put("user1", 1);
```

14. Atomic Classes

- For atomic (thread-safe) operations on variables.
- Examples: AtomicInteger, AtomicBoolean, AtomicLong.

```
AtomicInteger i = new AtomicInteger(0);
i.incrementAndGet();
```

15. Locks

- ReentrantLock, ReadWriteLock.
- More flexible than synchronized.

```
ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
    // critical section
} finally {
    lock.unlock();
}
```

Must Remember (for Interviews)

- ✓ Thread vs Process
- ✓ start() vs run()
- ✓ sleep()
- ✓ join()
- ✓ interrupt()
- ✓ synchronized
- ✓ race condition
- ✓ deadlock
- ✓ wait()/notify()
- ✓ ExecutorService
- ✓ ThreadPool
- ✓ Concurrent Collections.

Good to Know

- ✓ Atomic Classes
- ✓ Locks
- ✓ Daemon Thread

Skip / Low Priority

- yield()
- Thread Priority
- Deep JVM internals

Next →
File I/O
(Part 3)

Sandeep Chaurasiya

1. File

Concept:

- File class java.io package me hoti hai.
- Ye file ya directory ko represent karti hai.
- Isse hum file create, delete, check kar sakte hain aur file ka basic information (name, path, size etc.) le sakte hain.

Example: Create file and check properties

```

import java.io.File;
public class FileExample {
    public static void main(String[] args) {
        File file = new File("demo.txt");
        System.out.println("File name:" + file.getName());
        System.out.println("File path:" + file.getPath());
        System.out.println("Exists:" + file.exists());
        System.out.println("Is file:" + file.isFile());
        System.out.println("Size:" + file.length());
    }
}

```

Common Methods:

- new File(String path) → create file object
- exists() → check file exists or not
- getName() → file ka name
- getPath() → file ka path
- length() → file size (bytes)
- isFile() → file hai ya nahi
- isDirectory() → directory hai ya nahi

Real World Example:

- User data ko store karna (profile, settings etc.)
- Log files ko check karna
- File upload feature me file path use karna.

Spring Boot Use:

- File upload (MultipartFile)
- Configuration file read
- Log files handle karna

Priority: 🔥 Must Know 🔥 Good to Know ○ Skip / Low Priority

Remember:

File ke liye actual data read/write karne ke liye FileReader/FileWriter ya streams ka use hota hai.

2. FileReader & FileWriter

Concept:

- FileReader → character input (text read karne ke liye)
- FileWriter → character output (text write karne ke liye)
- Ye dono text files ke liye use hote hain. (binary ke liye streams)

A. FileReader (Read file using FileReader):

```

import java.io.*;
public class ReadFile {
    public static void main(String[] args) throws Exception {
        FileReader fr = new FileReader("data.txt");
        int ch;
        while ((ch = fr.read()) != -1) {
            System.out.print((char) ch);
        }
        fr.close();
    }
}

```

Line by Line:

1. FileReader object banaya (data.txt).
2. read() se ek-ek character padha.
3. -1 aane tak loop chalta hai.
4. Har character ko print kiya.
5. finally/last me close() kiya.

Real World / Spring Boot Use:

- Config file (application.properties) read karna.
- User data / text file read/write.
- Log files process karna.
- File based data export/import

Priority:

- 🔥 Must Know
- 🔥 Good to Know
- Skip

B. FileWriter (Write file using FileWriter):

```

import java.io.*;
public class WriteFile {
    public static void main(String[] args) throws Exception {
        FileWriter fw = new FileWriter("output.txt");
        fw.write("Hello Java!");
        fw.close();
    }
}

```

Line by Line:

1. FileWriter object banaya (output.txt).
2. write() se text likha.
3. close() kiya.

Difference (quick view):

Feature	FileReader	FileWriter
Type	Input (read)	Output (write)
Data	Characters (text)	Characters (text)
Use	Read file	Write file

Remember (Interview):

- FileReader/FileWriter character based hote hain.
- Binary data ke liye InputStream/OutputStream use hota hai.
- Always stream/reader/writer ko close karo (try-with-resources best hai).

10. Serialization

Concept :

- Serialization ka matlab hota hai object ko byte stream me convert karna, taaki use file ya network me store ya bheja ja sake.
- Iske liye class ko Serializable interface implement karna padta hai.
- Har field ko save nahi kar sakte, agar koi field transient ho to wo serialize nahi hoti.

Key Points :

- Serializable interface (java.io.Serializable)
- writeObject() / readObject() (custom serialization)
- transient keyword (field ko skip karne ke liye)

Interview Questions :

- Serialization kya hai?
- transient keyword ka use kab karte hain?
- Serializable interface me kaun sa method hota hai?
- Kya sabhi fields serialize hoti hain?
- Serialization ke advantages aur disadvantages?

Example - Serialization (Write to file)

```
import java.io.*;
class Student implements Serializable {
    private String name;
    private int age;
    transient private String password; // not serialized
    public Student(String name, int age, String password) {
        this.name = name;
        this.age = age;
        this.password = password;
    }
    public String toString() {
        return "Student{name=" + name + ", age=" + age +
            ", password=" + password + "}";
    }
}

public class SerializeExample {
    public static void main(String[] args) throws Exception {
        Student s = new Student("Sandeep", 22, "1234");
        ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream("student.ser"));
        oos.writeObject(s);
        oos.close();
        System.out.println("Object serialized");
    }
}
```

Coding Practice Questions :

1. Ek student object ko file me serialize karo.
2. File se object ko read karke print karo.
3. Ek class me transient field add karke check karo ki wo save nahi hota.

Line by Line :

1. Student class ko Serializable implement kiya.
2. transient field password hai (yeh save nahi hoga).
3. ObjectOutputStream se file me object likha gaya.
4. writeObject() method object ko byte stream me convert karta hai.
5. File "student.ser" me data save ho gaya.

Real World Example :

- User session save karna
- Configuration object save karna
- Cache data store karna
- Microservices me data exchange

Spring Boot Use :

- Session management, caching,
- file upload/download, object storage.

Priority :

-  Must Know
-  Good to Know
-  Skip

Remember :

- transient field serialize nahi hota.
- Serializable interface implement karna padta hai.

11. Deserialization

Concept :

- Deserialization ka matlab hai byte stream se object ko wapis Java object me convert karna.
- Iske liye ObjectInputStream ka use hota hai.
- File se data read karke object me convert kiya jata hai.

Key Points :

- ObjectInputStream (java.io.ObjectInputStream)
- readObject() method
- Class ko Serializable hona chahiye
- transient field ka value nahi aata

Interview Questions :

- Deserialization kya hai?
- transient keyword kaise kaam karta hai?
- ObjectInputStream vs FileInputStream me kya difference hai?
- Kya serialization ke bina deserialization possible hai?
- Kya hoga agar class Serializable implement nahi kare?

Example - Deserialization (Read from file)

```
import java.io.*;
class DeserializeExample {
    public static void main(String[] args) throws Exception {
        ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream("student.ser"));
        Student s = (Student) ois.readObject();
        ois.close();
        System.out.println("Deserialized object: " + s);
        System.out.println("Password (transient): " + s.password);
    }
}
```

Real World Example :

- User data load karna.
- File se configuration read karna
- Cached objects ko restore karna

Coding Practice Questions :

1. File se object ko read karke print karo.
2. Ek class me transient field ke saath test karo.
3. File se multiple objects read karo (list).

Line by Line :

1. File se ObjectInputStream banaya.
2. readObject() se object read kiya.
3. Type cast kiya (Student).
4. Object ko print kiya.
5. transient field ka value null aayega.

Priority :

-  Good to Know
-  Must to Know
-  Skip

Remember :

- readObject() ka use karo.
- transient field ka value nahi milta.
- Class ko Serializable hona zaroori hai.

Quick Summary :

- Serialization = Object → Byte Stream (save)
- Deserialization = Byte Stream → Object (read)
- Use: file, network, caching, session
- Interface: Serializable
- Skip: transient fields

8. ObjectInputStream

Concept :

- ObjectInputStream byte stream se object ko read karta hai.
- Ye deserialization ke liye use hota hai.
- Iske sath ObjectOutputStream ka pair hota hai.

Key Methods :

- readObject() → object read karta hai.
- close() → stream band karta hai.

Example - Read object using ObjectInputStream :

```

import java.io.*;
public class ReadObject {
    public static void main(String[] args) throws Exception {
        ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream("student.ser"));
        Student s = (Student) ois.readObject();
        System.out.println("Name : " + s.name);
        System.out.println("Age : " + s.age);
        ois.close();
    }
}

```

9. ObjectOutputStream

Concept :

- ObjectOutputStream object ko binary format me likhta hai.
- Ye serialization ke liye use hota hai.
- Iske sath FileOutputStream (ya koi OutputStream) use hota hai.

Key Methods :

- writeObject(Object obj) → object write karta hai.
- close() → stream band karta hai.

Example - Write object using ObjectOutputStream :

```

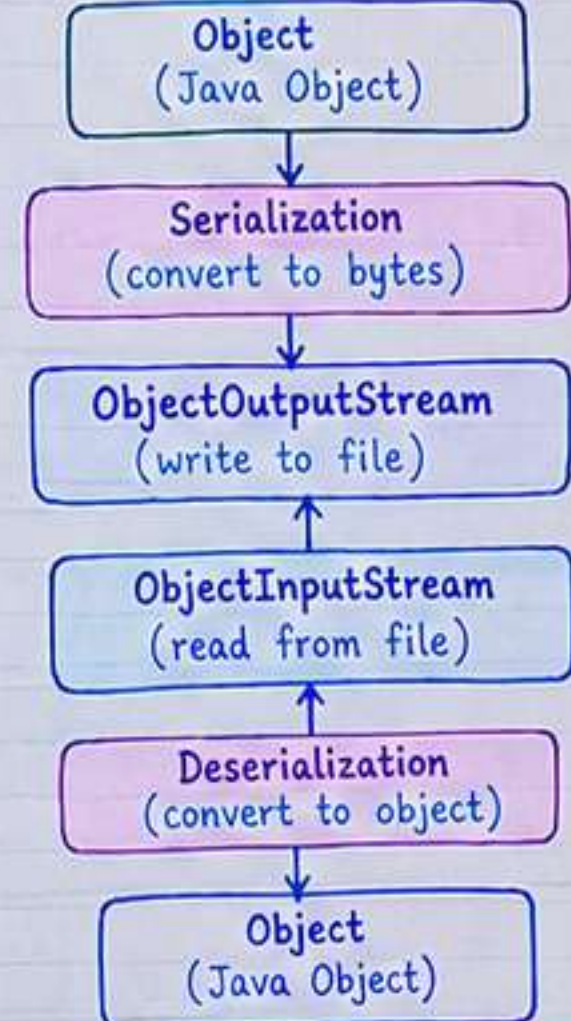
import java.io.*;
public class WriteObject {
    public static void main(String[] args) throws Exception {
        ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream("student.ser"));
        Student s = new Student("Sandeep", 22);
        oos.writeObject(s);
        oos.close();
        System.out.println("Object saved!");
    }
}

```

Real World Example :

- User session save karna.
- Cache data store karna.
- Configuration objects save/load karna.

How it works (Both together)



Priority :

- 🔥 Must Know
- 🔥 Good to Know
- Skip

10. Serialization

Concept :

- Object ko byte stream me convert karna.
- Iske liye class ko Serializable interface implement karna padta hai.
- Sirf non-transient fields hi serialize hote hain.

Key Points :

- Serializable interface
- SerialVersionUID
- transient keyword (exclude field)

Example - Serialization :

```

import java.io.*;
class Student implements Serializable {
    private String name;
    private int age;
    private transient String password; // not serialized
    public Student(String name, int age, String password) {
        this.name = name;
        this.age = age;
        this.password = password;
    }
}

```

Real World Example :

- User data save karna.
- Application state save karna.
- Cache / session management.

Important Points :

- transient field serialize nahi hota.
- static field serialize nahi hota.
- serialVersionUID version control ke liye.
- Custom serialization (advanced) - sirf interview / later.

Remember :

- Serializable → write (save)
- Deserializable → read (load)

Interview Question :

1. Serializable vs Externalizable?
2. transient keyword kya karta hai?
3. serialVersionUID kya hai?
4. kab serialization use karte hain?

Coding Question :

1. Ek class ko Serializable banao aur file me object save karo.
2. File se object read karke print karo.
3. transient field ko test karo.

11. Deserialization

Concept :

- Byte stream se object ko wapas banana.
- Iske liye ObjectInputStream use hota hai.
- Ye serialization ka reverse process hai.

Example - Deserialization :

```

import java.io.*;
public class ReadObject {
    public static void main(String[] args) throws Exception {
        ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream("student.ser"));
        Student s = (Student) ois.readObject();
        System.out.println("Name : " + s.name);
        ois.close();
    }
}

```

Real World Example :

- Saved user data load karna.
- Session restore karna.
- Configuration read karna.

Priority :

- 🔥 Must Know
- 🔥 Good to Know
- Skip

Interview Question :

1. Deserialization ka process explain karo.
2. Deserialization me kaun si class use hoti hai?
3. transient field ka value kya hota hai?

Coding Question :

1. Ek file se object read karke uska data print karo.
2. Multiple objects ko read karo.
3. Invalid file handle karo (exception).

Quick Summary :

- Serialization = object → byte stream (save) → Deserialization = byte stream → object (read)
- Needs : Serializable, ObjectOutputStream, ObjectInputStream → Use : file, session, cache, config

Remember :

- Non-serializable fields → transient
- Use try-with-resources for streams.

Practice + Projects
= Job Ready 😊



Java Annotations

(Metadata for your Code)

Write Clean Code
Think Like a Developer

Page 1/2



1. What is Annotation ?

- Annotation is a way to add metadata to your code.
- It provides extra information to the compiler, JVM or frameworks.
- Annotations don't directly change the program logic, but they are used by tools and frameworks (like Spring, Hibernate, etc).

Why use ?

- Reduces boilerplate code.
- Improves code readability.
- Helps in compile-time and runtime processing.
- Used in frameworks like Spring, Hibernate, JUnit, etc.

Example :

@Override

```
public String toString() {  
    return "Student";  
}
```

→ @Override
is an
annotation



Key Points to Remember

- ✓ Annotations start with @
- ✓ They are added above class, method, variable, etc.
- ✓ They do not change the logic directly.
- ✓ Widely used in real-world frameworks.

2. Types of Annotations

There are 3 main types :

1. Built-in (Predefined) Annotations

- Provided by Java (like @Override, @Deprecated, etc).

2. Meta-Annotations

- Used to create custom annotations (like @Retention, @Target, etc).

3. Custom Annotations

- Defined by the user as per need.

3. How Annotations Work ?

Source Code
(with annotation)



Compiler
(checks &
stores metadata)



Runtime /
Framework
(uses the metadata)

Example Flow :

- ① You write @Override in your code.
- ② Compiler checks it (gives error if wrong).
- ③ JVM / Framework uses it (e.g., Spring uses annotations for dependency injection).

1 Built-in Annotations (Examples)

Annotation	Use
@Override	Method overriding check
@Deprecated	Marks element as deprecated
@SuppressWarnings	Suppresses compiler warnings
@FunctionalInterface	Used for functional interfaces

2 Meta-Annotations (Examples)

Annotation	Use
@Retention	Defines how long annotation is kept
@Target	Specifies where it can be used
@Documented	Includes in Javadoc

3 Custom Annotation (Example)

```
@interface MyAnnotation {  
    String value(); // element  
}
```

```
@MyAnnotation("Hello")  
public class Test {  
    // ...  
}
```

↪ This is a
custom
annotation
with one
element
(value).

4. Common Use Cases

- Spring Framework (e.g., @Autowired, @Service, @RestController)
- Hibernate (e.g., @Entity, @Id)
- JUnit (e.g., @Test, @BeforeEach)
- Validation (e.g., @NotNull, @Size)

Annotations make
your code smarter,
cleaner and more powerful ! 😊

Sandeep Chaurasiya 😊

Annotation	Used For	Example
@Override	Method overriding check	@Override public String toString() { ... }
@Deprecated	Marks element as deprecated	@Deprecated public void oldMethod() { ... }
@SuppressWarnings	Suppresses compiler warnings	@SuppressWarnings("unchecked") List list = new ArrayList();
@FunctionalInterface	Used for functional interfaces	@FunctionalInterface interface MyFunc { void run(); }



Built-in annotations are already provided by Java. We just use them – no need to define them.



6. Creating Custom Annotations

- 1 Use @interface keyword.
- 2 Define elements (methods) inside it.
- 3 Provide default values (optional).

Example:

```
@interface MyAnnotation {
    String value(); // element
    int id() default 0; // default value
}
```

Using Custom Annotation

```
@MyAnnotation(value = "Test", id = 5)
public class Test {
    public static void main(String[] args) {
        System.out.println("Custom Annotation");
    }
}
```

Output:

Custom Annotation



We can add our own annotations for custom use cases (e.g., logging, validation, etc).

7. Meta-Annotations (Important)

- Annotations can be used to create other annotations.
- These are called meta-annotations.
- They help in defining behavior of custom annotations.

Common Meta-Annotations:

@Retention (how long the annotation is kept)
 @Target (where it can be used)
 @Documented (include in Javadoc)
 @Inherited (inherited by subclasses)

Example of Meta-Annotations

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@Documented
public @interface MyCustom {
    String value();
}
```



Meta-annotations tell Java how to handle your custom annotation.

Important Enums

- RetentionPolicy : SOURCE, CLASS, RUNTIME
- ElementType : TYPE, METHOD, FIELD, PARAMETER, CONSTRUCTOR, ...
- Target mentioned above.

← These enums are used inside meta-annotations.

8. Interview Questions (Annotation)

Q1. What is annotation in Java?

Ans: It is a way to add metadata to your code. It provides extra information to the compiler, JVM or frameworks.

Q2. What is the use of @Override?

Ans: It helps the compiler to check if you are correctly overriding a method from the parent class.

Q3. Difference between @Override and @Deprecated?

Ans: @Override → checks method overriding.
 @Deprecated → marks method/class as old (not used).

Q4. What is @SuppressWarnings?

Ans: It is used to suppress compiler warnings. e.g., @SuppressWarnings("unchecked")

Q5. What is the use of @FunctionalInterface?

Ans: It ensures the interface has only one abstract method (functional interface).

Q6. What is the difference between custom annotation and built-in annotation?

Ans: Built-in annotations are provided by Java. Custom annotations are user-defined using @interface.

→ Annotations make your code more organized, readable and powerful! 😊

Sandeep Chaurasiya 😊



Java Reflection

(Inspect and Modify Classes at Runtime)

Write Clean Code
Think Like a Developer

Page 1/2



1. What is Reflection?

- Reflection is a feature in Java that allows you to inspect and manipulate classes, methods, fields and constructors at runtime.
- It provides a way to examine the structure and behavior of a class without knowing it at compile time.
- It is part of the `java.lang.reflect` package.

Why use it?

- To create flexible and dynamic applications.
- To work with unknown classes (e.g. plugins, frameworks).
- To build generic utilities (e.g. dependency injection).
- Used in frameworks like Spring, Hibernate, JUnit, etc.

2. Important Classes and Interfaces

1. Class Class

- Represents a class at runtime.
- Used to get constructors, methods, fields, etc.
- Every class has a Class object.

Ways to get Class object:

- `Class.forName("com.example.User")`
- `User.class`
- `obj.getClass()`

2. AccessibleObject (Super class)

- Parent class of Field, Method, Constructor.
- Provides `setAccessible()` to access private members.

3. Field Class

- Represents a field (variable) of a class.
- Used to get/set field values.

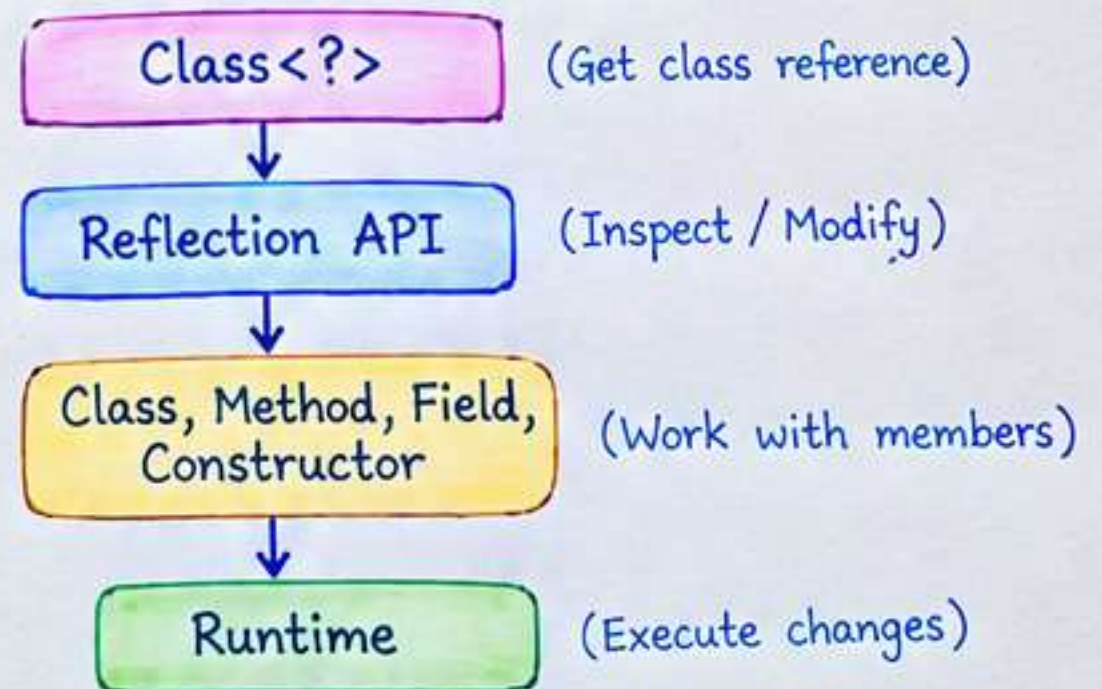
3. Common Methods (Class Class)

Method	Description
<code>getName()</code>	Returns the name of the class.
<code>getMethods()</code>	Returns all public methods.
<code>getDeclaredMethods()</code>	Returns all methods (including private).
<code>getFields()</code>	Returns all public fields.
<code>getDeclaredFields()</code>	Returns all fields (including private).
<code>newInstance()</code>	Creates a new instance (deprecated).



Reflection gives you power to work with classes dynamically. Use it wisely! 😊

How it works? (High Level)

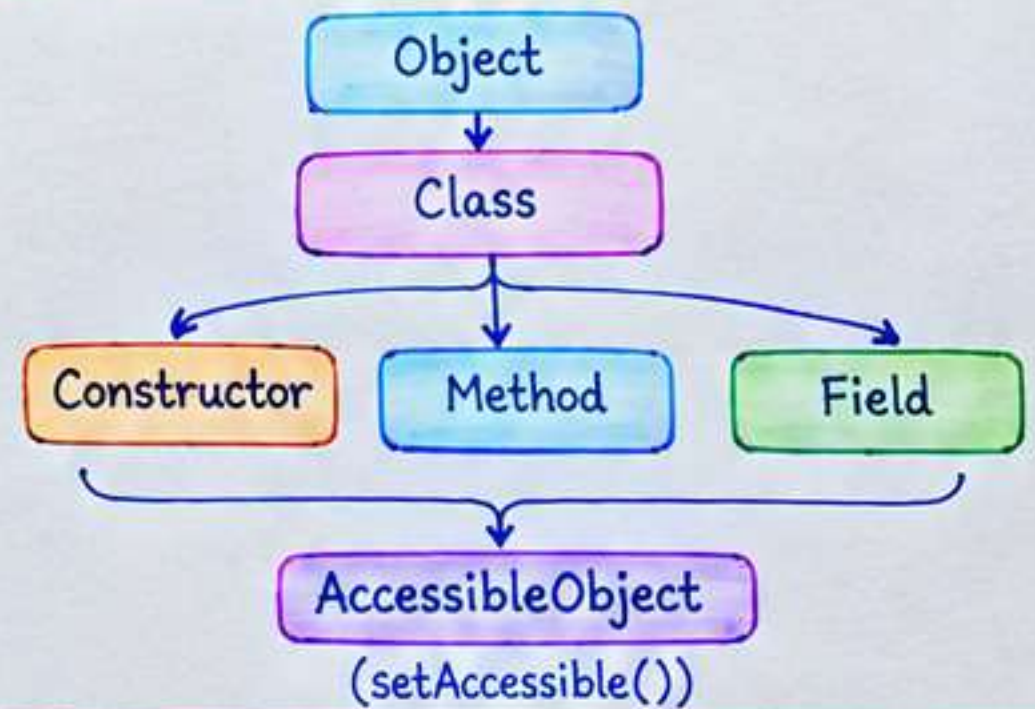


Example:

```

Class<?> cls = Class.forName("com.example.User");
Method m = cls.getMethod("display");
m.invoke(obj); // call method at runtime
  
```

Reflection API Hierarchy



Key Points to Remember

- ✓ Reflection works with Class, Method, Field, Constructor classes.
- ✓ Used for dynamic and flexible code.
- ✓ Requires `java.lang.reflect` package.
- ✓ By default, private members are not accessible (need `setAccessible(true)`).

4. Example - Get Class Info

```

Class<?> cls = Class.forName("com.example.User");
System.out.println("Class Name: " + cls.getName());
System.out.println("Simple Name: " + cls.getSimpleName());
System.out.println("Package: " + cls.getPackageName());
  
```

Output:

```

Class Name: com.example.User
Simple Name: User
Package: com.example
  
```

Sandeep Chaurasiya 😊



Java Reflection

(Inspect and Modify Classes at Runtime)

Write Clean Code
Think Like a Developer

Page 2/2



5. Example - Using Reflection

Here's a complete example to

- ✓ Load a class at runtime
- ✓ Create an object
- ✓ Call a method
- ✓ Get and modify a field value

```
import java.lang.reflect.*;

class ReflectionExample {
    public static void main(String[] args) throws Exception {
        // Load class
        Class<?> cls = Class.forName("com.example.User");
        // Create object
        Object obj = cls.getDeclaredConstructor().newInstance();
        // Call method
        Method method = cls.getMethod("display");
        method.invoke(obj);
        // Access field
        Field field = cls.getDeclaredField("name");
        field.setAccessible(true);
        field.set(obj, "Sandeep");
        System.out.println("Field value: " + field.get(obj));
    }
}
```

💡 Reflection is powerful but should be used carefully. It makes code more flexible but can also break encapsulation. 😊

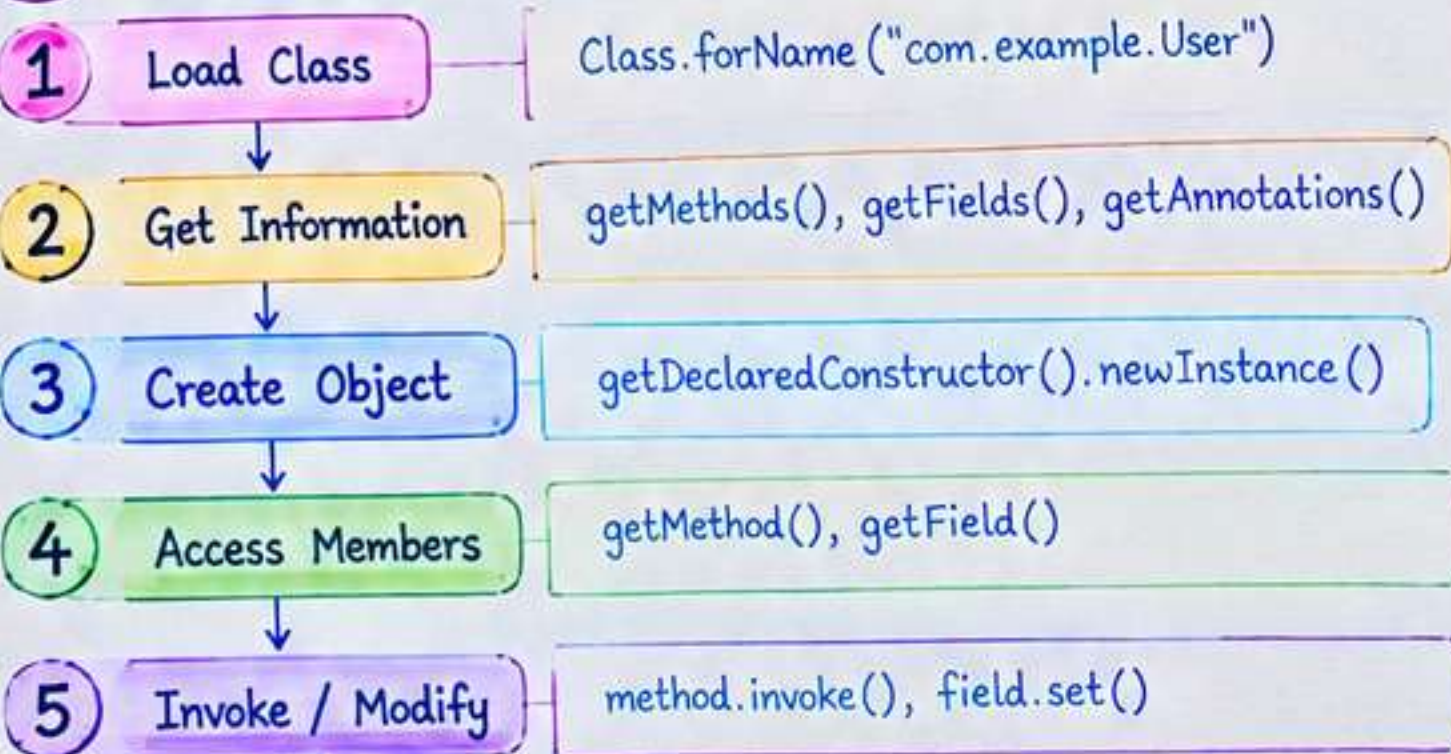
6. Important Methods of Class

Method	Return Type	Description
getName()	String	Returns the name of the class.
getSimpleName()	String	Returns simple name (without package).
getPackage()	Package	Returns package information.
getDeclaredMethods()	Method[]	Returns all declared methods (including private).
getDeclaredFields()	Field[]	Returns all declared fields (including private).
newInstance()	Object	Creates a new instance (using default constructor).



These methods help you inspect and work with classes at runtime. 😊

7. How Reflection Works (Step by Step)



9. Interview Questions (Reflection)

Q1. What is Java Reflection?

Ans: It is a feature in Java that allows us to inspect and manipulate classes, methods, fields and constructors at runtime.

Q2. Why is reflection used?

Ans: It is used to build flexible and dynamic applications, frameworks and to work with unknown classes at runtime.

Q3. What is `Class.forName()` used for?

Ans: It loads a class by its fully qualified name at runtime.

Q4. Can we modify private fields using reflection?

Ans: Yes, by using `setAccessible(true)`, we can access and modify private fields.

Q5. What is the difference between `getMethod()` and `getDeclaredMethod()`?

Ans: `getMethod()` returns only public methods, `getDeclaredMethod()` returns all methods (including private).

Q6. What is the use of `newInstance()`?

Ans: It creates a new instance of the class using default constructor.



Key Takeaways

- ✓ Reflection = inspect + modify classes at runtime.
- ✓ Used in frameworks, DI, ORM, testing, etc.
- ✓ Powerful but use it carefully.
- ✓ Common methods: `getName()`, `getMethods()`, `getField()`, `newInstance()`.

Reflection makes Java more dynamic and flexible! Use it wisely! 😊

Sandeep Chaurasiya 😊